

目 录

第 1 章 实验内容参考答案	1
第 2 章 实验内容参考答案	11
第 3 章 实验内容参考答案	18
第 4 章 实验内容参考答案	26
第 5 章 实验内容参考答案	34
第 6 章 实验内容参考答案	41
第 7 章 实验内容参考答案	47
第 8 章 实验内容参考答案	57
第 9 章 实验内容参考答案	64
第 10 章 实验内容参考答案	73
第 11 章 实验内容参考答案	81
第 12 章 实验内容参考答案	91
第 13 章 实验内容参考答案	99

第 1 章 实验内容参考答案

1. 已知拉马努金圆周率公式

$$\frac{1}{\pi} = \frac{2\sqrt{2}}{9801} \sum_{k=0}^{\infty} \frac{(4k)!}{(k!)^4} \frac{1103 + 26390k}{396^{4k}},$$

计算圆周率的近似值. 算法通过参数 k 值, 循环累加求和. 每项 $i = 0, 1, \dots, k$ 的计算, 均为标量计算, 可采用 `math` 库函数.

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_1_exp1.py
"""
import math
import time
from decimal import Decimal, getcontext

getcontext().prec = 100 # 高精度计算, 设置显示精度为 100 位

def calculate_pi(k):
    """
    拉马努金圆周率 pi 公式, 计算给定项数的 pi 近似值
    :param k: 计算圆周率所需的累加次数 k
    :return:
        approximate_pi, 即满足项数 k 的 pi 近似值
    """
    pi = Decimal(0.0) # 圆周率变量, 初始化为 0.0
    approximate_pi = Decimal(0.0) # 初始化最终计算的近似圆周率值
    coef = Decimal(2 * math.sqrt(2) / 9801) # 拉马努金公式系数
    # =====
    # 对于每一项, 分别按照拉马努金公式循环计算 k 此, 然后累加求解近似值
    # 其中 math.factorial() 为阶乘函数
    # Decimal() 为高精度计算类
    # 打印输出每次计算后的近似值
    # =====
    for i in range(k):
        # 拉马努金计算公式
        pi += Decimal(math.factorial(4 * i) / math.factorial(i) ** 4) * \
            (1103 + 26390 * i) / Decimal(396 ** (4 * i))
        approximate_pi = Decimal(1 / pi / coef) # 与系数乘积
    return approximate_pi

# 定义主函数, 并调用函数, 也可不使用主函数.
```

```

if __name__ == '__main__':
    k = int(input("请输入计算圆周率所需的累加次数 k = ")) # 接受用户输入且转换为整型
    start = time.time() # 计算消耗时间, 开始
    approximate_pi = calculate_pi(k) # 调用函数, 传递参数
    end = time.time() # 计算消耗时间, 结束
    print("消耗时间: %.15f" % (end - start)) # 消耗时间计算
    print(approximate_pi) # 直接打印输出
    print("指定输出小数点后 15 位: %.15f" % approximate_pi) # 占位符, 格式化输出

```

执行算法, 输入 $k=2$, 结果如下, 已具有相当高的精度.

```

请输入计算圆周率所需的累加次数 k = 2
消耗时间: 0.000000000000000
3.14159265358979342612963862481868208019403757270240261697677553953703549649127690004202740407
5451609
指定输出小数点后 15 位: 3.141592653589794

```

2. 完成算法“飞船软着陆模拟计算”和“曲边梯形近似积分计算”算法的可视化代码的内容, 并设置不同的参数, 验证可视化代码的正确性.

(1) 飞船软着陆模拟计算, 面向过程的程序设计可视化代码.

```

def plt_spacecraft_simulation(info_dict, rf_ths, i=0):
    """
    绘制飞船软着陆下降轨迹和速度变化曲线
    :param info_dict: 计算过程信息存储的字典
    :param rf_ths: 反向喷射的时刻、高度和速度
    """
    time, height, speed = info_dict["time"], info_dict["height"], info_dict["speed"]
    idx = np.argmin(height) # 飞船软着陆过程中最小的高度值索引
    time_min, height_min, speed_min = time[idx], height[idx], speed[idx]
    # plt.figure(figsize=(14, 5)) # 若独立绘图, 可取消注释; 绘制子图, 则注释
    plt.subplot(221 + 2 * i)
    plt.plot(time, height, "k-", lw=1.5, label="轨迹曲线")
    plt.plot(rf_ths[0], rf_ths[1], "ro", label="反向喷射点")
    plt.plot(time_min, height_min, "bs", label="最小高度: $%.2f$" % abs(height_min))
    plt.xlabel("时间", fontdict={"fontsize": 18})
    plt.ylabel("高度", fontdict={"fontsize": 18})
    if i == 0:
        plt.title("模拟飞船软着陆问题轨迹曲线 (成功)", fontdict={"fontsize": 18})
    else:
        plt.title("模拟飞船软着陆问题轨迹曲线 (失败)", fontdict={"fontsize": 18})
    plt.legend(frameon=False, fontsize=16, loc="best") # 添加图例
    plt.tick_params(labelsize=16) # 刻度字体大小 16
    plt.grid(ls=":")

```

```

plt.subplot(221 + 2 * i + 1)
plt.plot(time, np.abs(speed), "k-", lw=1.5, label="速度曲线")
plt.plot(rf_ths[0], rf_ths[2], "ro", label="反向喷射点")
plt.plot(time_min, speed_min, "bs", label="最小速度: %.2f$" % speed_min)
plt.xlabel("时间", fontdict={"fontsize": 18})
plt.ylabel("速度", fontdict={"fontsize": 18})
if i == 0:
    plt.title("模拟飞船软着陆问题速度曲线（成功）", fontdict={"fontsize": 18})
else:
    plt.title("模拟飞船软着陆问题速度曲线（失败）", fontdict={"fontsize": 18})
plt.legend(frameon=False, fontsize=16, loc="best") # 添加图例
plt.tick_params(labelsize=16) # 刻度字体大小 16
plt.grid(ls=":")
# plt.show() # 若独立绘图, 可取消注释;

```

(2) 曲边梯形近似积分计算, 面向对象程序设计可视化代码.

```

def plt_approximate_processing(self):
    """
    可视化随着划分区间次数的增加, 积分近似值的逼近过程
    :return:
    """

    plt.figure(figsize=(14, 5))
    xi = np.linspace(0, 1, 150)
    yi = self.int_fx(xi)
    plt.subplot(121)
    plt.plot(xi, yi, "k-", lw=1)
    plt.fill_between(xi, yi, color="c", alpha=0.5)
    plt.xlabel(r"$x$", fontdict={"fontsize": 18})
    plt.ylabel(r"$f(x)$", fontdict={"fontsize": 18})
    plt.title("被积函数的积分区域", fontdict={"fontsize": 18})
    plt.tick_params(labelsize=16) # 刻度字体大小 16
    plt.subplot(122)
    tol_ = np.abs(self.approximate_values[-1] - self.approximate_values[-2])
    plt.plot(self.approximate_values, "ko--", markerfacecolor="r", markeredgecolor="r",
             label="$a^* = %.15f$" % self.approximate_values[-1])
    plt.xlabel("划分区间数", fontdict={"fontsize": 18})
    plt.ylabel("积分近似值", fontdict={"fontsize": 18})
    plt.title("积分近似值的逼近过程: $tol = %.5e$" % tol_, fontdict={"fontsize": 18})
    plt.legend(frameon=False, fontsize=16, loc="best") # 添加图例
    plt.tick_params(labelsize=16) # 刻度字体大小 16
    ticks, orders = [], np.arange(0, len(self.approximate_values), 2)
    for i in orders:
        ticks.append("$2^{%d}$" % i)
    plt.xticks(orders, ticks)
    plt.grid(ls=":")
    plt.show()

```


3. 利用曲边梯形近似计算积分算法, 计算 $I(f) = \int_0^{0.5} e^x dx$ 的近似值, 精度要求 $\varepsilon = 10^{-8}$, 分析矢量化计算和非矢量计算的效率. 附加: 尝试把面向对象的算法设计改为面向过程的算法设计.

(1) 积分计算代码.

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_1_exp3_1.py
"""

import numpy as np
import time
from fundamentals_python_mathematics_01.trapezoidal_integral import TrapezoidalIntegral

fh = lambda x: np.exp(x) # 被积函数
time1_list, time2_list = [], [] # 矢量化和非矢量化计算时间存储
for i in range(100):
    start = time.time()
    t_int = TrapezoidalIntegral(fh, 0, 0.5, eps=1e-8, max_split_interval_num=200000)
    int_val, split_num = t_int.cal_trapezoid_int_vectorization()
    end = time.time()
    time1 = end - start
    time1_list.append(time1)
    if i == 99:
        print("划分区间数: %d, 积分近似值: %.15f" % (split_num, int_val))
        print("误差: %.10e" % abs(np.exp(0.5) - 1 - int_val))
        t_int.plt_approximate_processing()

# 非矢量化计算
start = time.time()
t_int = TrapezoidalIntegral(fh, 0, 0.5, eps=1e-8, max_split_interval_num=200000)
int_val, split_num = t_int.cal_trapezoid_int_nvectorization()
end = time.time()
time2 = end - start
time2_list.append(time2)
if i == 99:
    print("划分区间数: %d, 积分近似值: %.15f" % (split_num, int_val))
    print("误差: %.10e" % abs(np.exp(0.5) - 1 - int_val))
    t_int.plt_approximate_processing()

print("非矢量计算平均时间与矢量计算平均时间分别为: ", np.mean(time2_list), np.mean(time1_list))
print("非矢量计算时间与矢量计算比值: ", np.mean(time2_list) / np.mean(time1_list))
```

算法执行结果如表 1 - 1, 由于舍入误差的存在, 积分的绝对值误差存在些许差异. 被积函数积分区域以及积分值随划分区间数收敛过程如图 1 - 1 所示.

表 1 - 1 曲边梯形近似计算积分算法计算结果

方法	划分区间数	积分近似值	绝对误差	平均时间消耗 (秒)
矢量化方法	2048	0.6487212739	3.2222333690e-09	0.00037996768951416017
非矢量化方法	2048	0.6487212739	3.2222334800e-09	0.004760208129882812

非矢量计算时间与矢量计算比值: 12.52792871933237

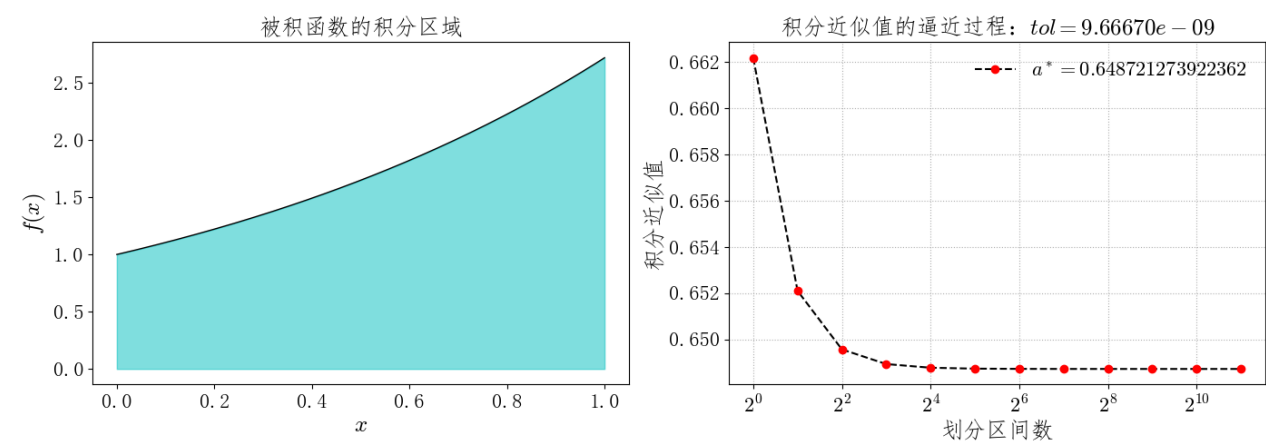


图 1 - 1 被积函数积分区域以及积分值随划分区间数收敛过程

(2) 面向过程的程序设计以及测试代码.

为便于统一可视化图像的字体和公式, 定义 Python 文件如下, 具体求解代码可导入该文件.

```
# -*- coding: UTF-8 -*-
"""
@file_name: util_font.py
"""

import matplotlib.pyplot as plt
import matplotlib as mpl

# 设置数学模式下的字体格式和中文显示
rc = {"font.family": "serif", "mathtext.fontset": "cm"}
plt.rcParams.update(rc)
mpl.rcParams["font.family"] = "FangSong" # 中文显示
plt.rcParams["axes.unicode_minus"] = False # 解决坐标轴负数的负号显示问题
```

面向过程的程序设计

```
# -*- coding: UTF-8 -*-
"""
@file_name: chatper_1_exp3_2.py
"""

import numpy as np
from Experiment.util_font import * # 导入字体文件

def cal_trapezoid_int_vectorization(int_fx, a, b, eps=1e-10, max_split_interval_num=1000):
    """
```

矢量化计算核心算法: 自适应计算划分区间数, 对每个小区间计算梯形面积, 近似曲边梯形面积

:param int_fx: 被积函数, 非符号定义

:param a: 积分下限

:param b: 积分上限

:param eps: 积分精度, 采用划分前后两次精度的绝对值差判断

:param max_split_interval_num: 最大划分区间数

:return: 积分近似值 int_val_n, 划分区间数 split_num, 自适应过程中的积分近似值 approximate_values

"""

approximate_values = [] # 自适应过程中的积分近似值

int_val_n = (b - a) / 2 * (int_fx(a) + int_fx(b)) # 梯形面积

approximate_values.append(int_val_n)

tol, split_num = np.infty, 1 # 初始化, 逼近精度 tol 和区间划分数

while tol > eps and split_num < max_split_interval_num:

int_val_b = int_val_n # 分别模拟当前划分区间数的积分值和下一次划分区间数的积分值

split_num *= 2 # 每次增加一倍的划分数量: 1、2、4、8、16、...

h = (b - a) / split_num # 小区间步长, 等分

x_k = np.linspace(a, b, split_num + 1) # 区间端点, 为 n + 1

f_xk = int_fx(x_k) # 区间端点的函数值

int_val_n = h / 2 * np.sum((f_xk[:-1] + f_xk[1:])) # 积分值, 一维向量相加

approximate_values.append(int_val_n)

tol = np.abs(int_val_n - int_val_b)

return int_val_n, split_num, approximate_values

def cal_trapezoid_int_nvectorization(int_fx, a, b, eps=1e-10, max_split_interval_num=1000):

"""

非矢量化计算核心算法: 自适应计算划分区间数, 对每个小区间计算梯形面积, 近似曲边梯形面积

参数同 cal_trapezoid_int_vectorization 函数

:return:

"""

approximate_values = [] # 自适应过程中的积分近似值

int_val_n = (b - a) / 2 * (int_fx(a) + int_fx(b)) # 梯形面积

approximate_values.append(int_val_n)

tol, split_num = np.infty, 1 # 初始化, 逼近精度 tol 和区间划分数

while tol > eps and split_num < max_split_interval_num:

int_val_b = int_val_n # 分别模拟当前划分区间数的积分值和下一次划分区间数的积分值

split_num *= 2 # 每次增加一倍的划分数量: 1、2、4、8、16、...

h = (b - a) / split_num # 小区间步长, 等分, 标量计算

x_k, f_xk = [], [] # 列表用于存储区间端点 x_k 和对应的函数值 f_xk

for i in range(split_num + 1):

x_k.append(a + i * h) # 端点值

f_xk.append(int_fx(x_k[-1])) # 区间端点的函数值

int_val_n = 0.0 # 积分值

for i in range(split_num):

int_val_n += h / 2 * (f_xk[i] + f_xk[i + 1]) # 积分值, 标量相加

approximate_values.append(int_val_n)

```

        tol = np.abs(int_val_n - int_val_b)
    return int_val_n, split_num, approximate_values

def plt_approximate_processing(int_fx, approximate_values):
    """
    可视化随着划分区间次数的增加, 积分近似值的逼近过程
    :param int_fx: 被积函数, 非符号定义
    :param approximate_values: 自适应过程中的积分近似值
    :return:
    """
    plt.figure(figsize=(14, 5))
    xi = np.linspace(0, 1, 150)
    yi = int_fx(xi)
    plt.subplot(121)
    plt.plot(xi, yi, "k-", lw=1)
    plt.fill_between(xi, yi, color="c", alpha=0.5)
    plt.xlabel(r"$x$", fontdict={"fontsize": 18})
    plt.ylabel(r"$f(x)$", fontdict={"fontsize": 18})
    plt.title("被积函数的积分区域", fontdict={"fontsize": 18})
    plt.tick_params(labelsize=16) # 刻度字体大小 16
    plt.subplot(122)
    tol_ = np.abs(approximate_values[-1] - approximate_values[-2])
    plt.plot(approximate_values, "ko--", markerfacecolor="r", markeredgcolor="r",
             label="$a^* = %.15f$" % approximate_values[-1])
    plt.xlabel("划分区间数", fontdict={"fontsize": 18})
    plt.ylabel("积分近似值", fontdict={"fontsize": 18})
    plt.title("积分近似值的逼近过程: $tol = %.5e$" % tol_, fontdict={"fontsize": 18})
    plt.legend(frameon=False, fontsize=16, loc="best") # 添加图例
    plt.tick_params(labelsize=16) # 刻度字体大小 16
    ticks, orders = [], np.arange(0, len(approximate_values), 2)
    for i in orders:
        ticks.append("$2^{%d}$" % i)
    plt.xticks(orders, ticks)
    plt.grid(ls=":")
    plt.show()

```

对同一个问题, 设计测试代码如下 (注意与面向对象的程序设计的不同):

```

# -*- coding: UTF-8 -*-
"""
@file_name: chapter_1_test_exp3_2.py
"""
import time
from Experiment.chapter_1.exp3_2 import *

fh = lambda x: np.exp(x) # 被积函数

```

```

time1_list, time2_list = [], [] # 矢量化和非矢量化计算时间存储
for i in range(100):
    start = time.time()
    sol = cal_trapezoid_int_vectorization(fh, 0, 0.5, eps=1e-8, max_split_interval_num=200000)
    int_val, split_num, approximate_values = sol[0], sol[1], sol[2]
    end = time.time()
    time1 = end - start
    time1_list.append(time1)
    if i == 99:
        print("划分区间数: %d, 积分近似值: %.15f" % (split_num, int_val))
        print("误差: %.10e" % abs(np.exp(0.5) - 1 - int_val))
        plt_approximate_processing(fh, approximate_values) # 可视化

# 非矢量化计算
start = time.time()
sol = cal_trapezoid_int_nvectorization(fh, 0, 0.5, eps=1e-8, max_split_interval_num=200000)
int_val, split_num, approximate_values = sol[0], sol[1], sol[2]
end = time.time()
time2 = end - start
time2_list.append(time2)
if i == 99:
    print("划分区间数: %d, 积分近似值: %.15f" % (split_num, int_val))
    print("误差: %.10e" % abs(np.exp(0.5) - 1 - int_val))
    plt_approximate_processing(fh, approximate_values)

print("非矢量计算平均时间与矢量计算平均时间分别为: ", np.mean(time2_list), np.mean(time1_list))
print("非矢量计算时间与矢量计算比值: ", np.mean(time2_list) / np.mean(time1_list))

```

算法执行结果如表 1-1 和图 1-1 所示, 由于结果一致, 不再赘述。

4. 采用面向对象的程序设计, 根据迭代公式

$$x_{k+1} = \frac{1}{1 + x_k}, \quad k = 0, 1, \dots,$$

设计算法, 求方程 $x^2 + x - 1 = 0$ 的正根 $x^* = \frac{-1 + \sqrt{5}}{2}$, 取 $x_0 = 1$ 。

```

# -*- coding: UTF-8 -*-
"""
@file_name: chatper_1_exp4.py
"""
import math
from Experiment.util_font import * # 导入字体文件

class IterationExp:
    """
    实验内容 4, 迭代法示例算法, 面向对象的程序设计
    """

```

```

"""
def __init__(self, x0, eps=1e-15, max_iter=100):
    self.x0 = x0  # 迭代初始值
    self.eps = eps  # 精度控制
    self.max_iter = max_iter  # 最大迭代次数
    self.approximate_values = []  # 存储迭代逼近过程中的值

def cal_iter(self):
    """
    迭代过程设计
    :return:
    """
    self.approximate_values.append(self.x0)  # 迭代逼近过程中的值
    x_k, tol, iter_ = self.x0, math.inf, 0  # 初始化, 迭代值 x_k、逼近精度 tol 和迭代次数 iter_
    # 如下为迭代过程, 若满足任何一个条件 (精度要求和最大迭代次数), 则继续迭代
    while tol >= self.eps and iter_ < self.max_iter:
        x_b = x_k  # x_b 为迭代的上一次值
        x_k = 1 / (1 + x_b)  # 迭代公式
        self.approximate_values.append(x_k)  # 存储迭代过程中的近似值
        tol = abs(x_k - x_b)  # 相邻两次迭代的绝对差值为精度, 改变量较小时, 终止
        iter_ += 1  # 迭代次数加一
    return x_k

def plt_approximate_processing(self):
    """
    可视化迭代逼近过程中的近似值曲线
    :return:
    """
    plt.figure(figsize=(7, 5))
    plt.plot(self.approximate_values, "ko--",
             label="$x_k, k=%d$" % (len(self.approximate_values) - 1))  # 可视化迭代过程值
    plt.plot(self.approximate_values[0], "D",
             label="$x_0=%.f, \epsilon=10^{-16}$" % self.approximate_values[0])  # 可视化初值
    plt.plot(len(self.approximate_values) - 1, self.approximate_values[-1], "s",
             label="$x^* = %.15f$" % self.approximate_values[-1])  # 可视化最终近似值
    plt.xlabel("$Iterations(k)$", fontdict={"fontsize": 18})  # x 轴标记
    plt.ylabel(r"$x_k$", fontdict={"fontsize": 18})  # y 轴标记
    tol_ = abs(self.approximate_values[-1] - self.approximate_values[-2])  # 算法终止的精度
    plt.title(r"迭代法示例, 终止精度: $\epsilon=\text{vert } x_{k+1} - x_k \text{ vert} = %.3e$"
             % tol_, fontdict={"fontsize": 18})  # 标题
    plt.legend(frameon=False, fontsize=16, loc="best")  # 添加图例
    plt.tick_params(labels=16)  # 刻度字体大小 16
    plt.grid(ls=":")  # 添加网格线, 且是虚线
    plt.show()

```

测试代码如下:

```

# -*- coding: UTF-8 -*-
"""
@file_name: chatper_1_test_exp4.py
"""

import math
from Experiment.chapter_1.exp4 import IterationExp

IExp = IterationExp(x0=1, eps=1e-16, max_iter=100) # 初始化对象
x = IExp.cal_iter() # 调用迭代方法, 执行迭代
IExp.plt_approximate_processing() # 可视化
print("近似值: %.15f, 绝对值误差: %.15e" % (x, abs((-1 + math.sqrt(5)) / 2 - x)))

```

算法执行结果, 近似值为 0.618033988749895, 共迭代 38 次, 精度控制 $\varepsilon = |x_{k+1} - x_k| = 0.0 < 10^{-16}$, 满足精度要求, 最终与真值的绝对值误差为 1.110223024625157e-16. 迭代逼近过程如图 1-2 所示. 可尝试其他迭代初值. 如初值 $x_0 = 100$ 或 $x_0 = -100$, 满足精度要求下均需迭代 40 次.

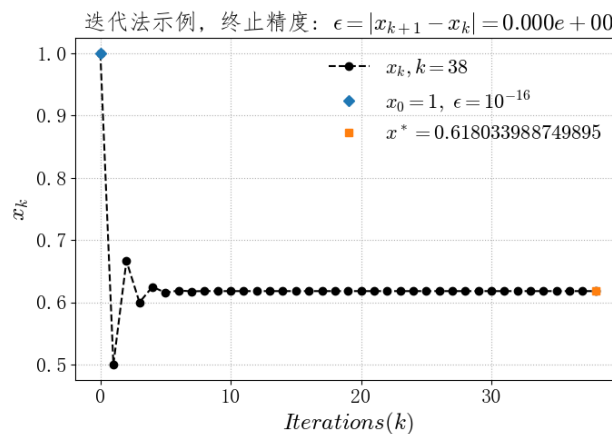


图 1-2 迭代逼近过程曲线

第 2 章 实验内容参考答案

1. 已经测得在某处海洋不同深度处的水温, 如表 2-1 所示, 采用拉格朗日插值、牛顿差商插值和牛顿差分插值, 推测深度为 $500m$ 、 $600m$ 、 $1000m$ 和 $1450m$ 处的水温, 并分别给出插值多项式, 对比分析.

表 2-1 某处海洋不同深度处的水温值

深度 (m)	460	760	1060	1360	1660
水温 ($^{\circ}C$)	7.04	4.28	3.40	2.54	2.13

求解代码如下:

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_2_exp1.py
"""

import numpy as np
import matplotlib.pyplot as plt
from interpolation_02.lagrange_interpolation import LagrangeInterpolation
from interpolation_02.newton_diff_quotient_interp import NewtonDifferenceQuotient
from interpolation_02.newton_difference_interpolation import NewtonDifferenceInterpolation

x = np.array([460, 760, 1060, 1360, 1660]) # 插值节点
y = np.array([7.04, 4.28, 3.40, 2.54, 2.13]) # 插值节点
x0 = np.array([500, 600, 1000, 1450]) # 待求插值点

# 1. 拉格朗日插值
lagI = LagrangeInterpolation(x, y) # 初始化对象
lagI.fit_interp() # 执行插值
y0 = lagI.predict_x0(x0) # 推测未知数据的插值
print("拉格朗日插值多项式: \n", lagI.polynomial) # 打印拉格朗日多项式特征
print("多项式系数: ", lagI.poly_coefficient) # 系数
print("多项式系数对应的各阶次: ", lagI.coefficient_order) # 对应阶次
print("插值点 x0 的多项式插值: ", y0) # 推测的插值
print("=" * 80)

# 2. 牛顿差商插值
NDQ = NewtonDifferenceQuotient(x, y)
NDQ.fit_interp()
y0 = NDQ.predict_x0(x0)
print("牛顿差商插值多项式: \n", NDQ.polynomial) # 打印牛顿差商插值多项式特征
print("多项式系数: ", NDQ.poly_coefficient) # 系数
print("多项式系数对应的各阶次: ", NDQ.coefficient_order) # 对应阶次
print("插值点 x0 的多项式插值: ", y0) # 推测的插值
print("=" * 80)
```



```

# 3. 牛顿差分插值
NDI_f = NewtonDifferenceInterpolation(x, y, diff_method="forward")
NDI_f.fit_interp()
y0 = NDI_f.predict_x0(x0)
print("牛顿前向差分插值多项式: \n", NDI_f.polynomial) # 打印牛顿前向差分插值多项式特征
print("多项式系数: ", NDI_f.poly_coefficient) # 系数
print("多项式系数对应的各阶次: ", NDI_f.coefficient_order) # 对应阶次
print("插值点 x0 的多项式插值: ", y0) # 推测的插值
print("=" * 80)

NDI_b = NewtonDifferenceInterpolation(x, y, diff_method="backward")
NDI_b.fit_interp()
y0 = NDI_b.predict_x0(x0)
print("牛顿后向差分插值多项式: \n", NDI_b.polynomial) # 打印牛顿后向差分插值多项式特征
print("多项式系数: ", NDI_b.poly_coefficient) # 系数
print("多项式系数对应的各阶次: ", NDI_b.coefficient_order) # 对应阶次
print("插值点 x0 的多项式插值: ", y0) # 推测的插值
print("=" * 80)

# 可视化
plt.figure(figsize=(14, 10))
plt.subplot(221)
lagl.plt_interpolation(x0, y0, is_show=False)
plt.subplot(222)
NDQ.plt_interpolation(x0, y0, is_show=False)
plt.subplot(223)
NDI_f.plt_interpolation(x0, y0, is_show=False)
plt.subplot(224)
NDI_b.plt_interpolation(x0, y0, is_show=False)
plt.show()

```

拉格朗日插值、牛顿差商插值、牛顿前向差分插值和牛顿后向差分插值求解结果基本一致。推测深度为:

$$[6.437509, 5.307491, 3.547856, 2.290273],$$

拉格朗日插值、牛顿差商插值的多项式为:

$$g(x) = 1.177984 \times 10^{-11} x^4 - 5.436008 \times 10^{-8} x^3 + 9.250105 \times 10^{-5} x^2 - 0.0714879x + 25.114980,$$

由于牛顿差分插值是关于变量 t 的函数, 如前向差分插值, 其待求值需要通过

$$x = x_0 + th \Rightarrow t = (x - x_0)/h$$

而得到, 其中 $h = 300$, $x_0 = 460$. 具体, 牛顿前向差分插值的多项式为:

$$N_n(t) = 0.0954167t^4 - 0.882450t^3 + 2.919583t^2 - 4.892500t + 7.040000.$$

牛顿后向差分插值的多项式为:

$$N_n(t) = 0.0954167t^4 + 0.644167t^3 + 1.489583t^2 + 0.530833t + 2.130000$$

四种方法的插值多项式曲线如图 2 - 1 所示.

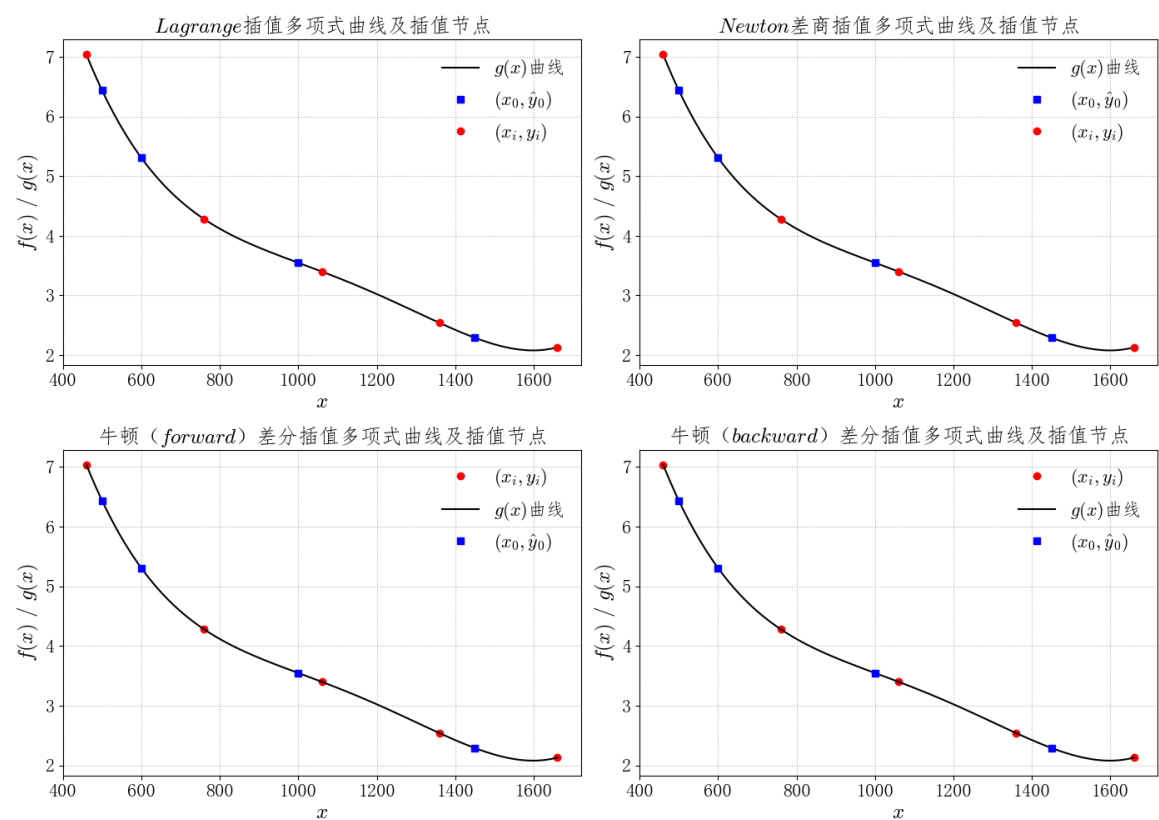


图 2 - 1 不同多项式插值方法的插值图形

2. 给定数据如表 2 - 2, 试求三次样条插值 $S(x)$, 并满足条件:

- (1) $S'(0.25) = 1.0000$, $S'(0.53) = 0.6868$;
- (2) $S''(0.25) = S''(0.53) = 0$.

表 2 - 2 离散插值数据点 ($k = 1, 2, \cdots, 5$)

x_k	0.25	0.30	0.39	0.45	0.53
y_k	0.5000	0.5477	0.6245	0.6708	0.7280

由于条件 (1) 给出了边界条件的一阶导数, 故采用第一种边界条件; 条件 (2) 给出了边界条件的二阶导数且为 0, 故采用自然样条边界条件.

求解代码如下:

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_2_exp2.py
"""
import numpy as np
import matplotlib.pyplot as plt
from interpolation_02.cubic_spline_interpolation import CubicSplineInterpolation

x = np.array([0.25, 0.30, 0.39, 0.45, 0.53])
```

```
y = np.array([0.500, 0.5477, 0.6245, 0.6708, 0.7280])
dy = np.array([1.0000, 0.6868])
d2y = np.array([0, 0])
boundary_cond = ["complete", "natural"]
plt.figure(figsize=(14, 5))
for i, bc in enumerate(boundary_cond):
    csi = CubicSplineInterpolation(x, y, dy=dy, d2y=d2y, boundary_cond=bc)
    csi.fit_interp()
    print(csi.poly_coefficient)
    plt.subplot(121 + i)
    csi.plt_interpolation(is_show=False)
plt.show()
```

表 2 - 3 为三次样条插值分段三次多项式系数，如第一种边界条件的第一段为

$$S_1(x)=1.89x^3-2.43x^2+1.86x^2+0.157, x\in[0.25, 0.3]$$

表 2 - 3 三次样条插值分段三次多项式系数

第一种边界条件				自然样条边界条件			
常数项	x	x^2	x^3	常数项	x	x^2	x^3
0.1571320	1.8608377	-2.429036	1.8862950	0.3554775	-0.2050555	4.6988737	-6.265165
0.1865921	1.5662361	-1.4470306	0.7951779	0.1355217	1.9945019	-2.6329843	1.881344
0.1962726	1.4917711	-1.2560947	0.6319848	0.2744079	0.9261471	0.1063871	-0.4599991
0.2251492	1.2992606	-0.8282936	0.3150951	0.037099	2.5082075	-3.4093028	2.1442156

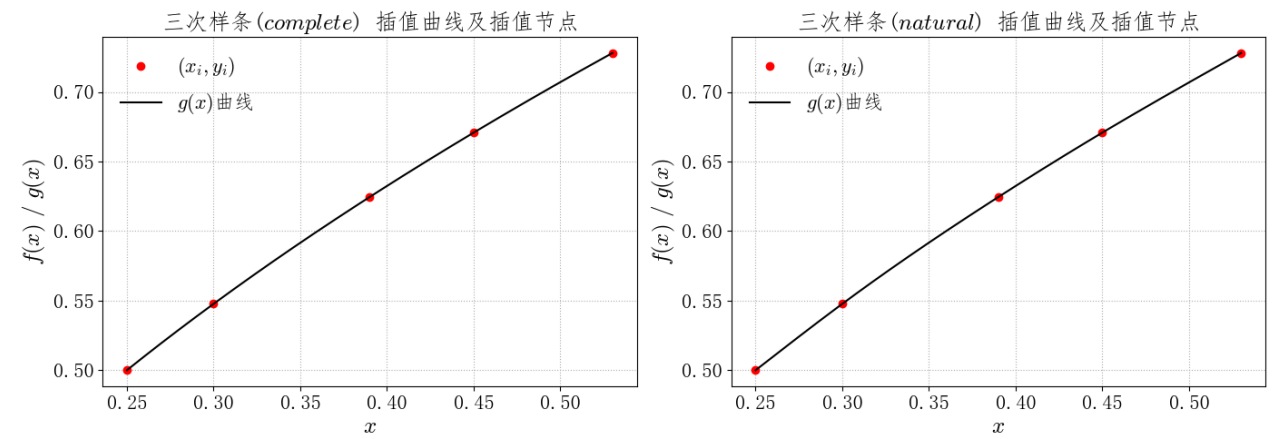


图 2 - 2 三次样条插值插值结果图像

3. 轮船的甲板成近似半椭圆面形，为了得到甲板的面积，首先测得横向最大相间为 8.540 米，然后等间距划分 13 个点，然后测得纵向高度，如表 2 - 4 所示. 采用分段线性插值和三次样条插值，推测每隔0.01m时的高度值 $\hat{y}_k$ ，并进行可视化. 根据推测的高度值 $\hat{y}_k$ ，采用近似曲边梯形面积计算甲板的近似面积.

表 2 - 4 甲板在 [0, 8.540] 内等距 13 个点的高度值

0	0.914	5.060	7.772	8.717	9.083	9.144
9.083	8.722	7.687	5.376	1.073	0	

求解代码如下:

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_2_exp3.py
"""

import numpy as np
import matplotlib.pyplot as plt
from interpolation_02.piecewise_linear_interpolation import PiecewiseLinearInterpolation
from interpolation_02.b_spline_interpolation import BSplineInterpolation

x = np.linspace(0, 8.540, 13)
y = np.array([0, 0.914, 5.060, 7.772, 8.717, 9.083, 9.144, 9.083, 8.722, 7.687, 5.376, 1.073, 0])
xi = np.arange(0, 8.550, 0.01) # 间隔 0.01, 推测插值

plt.figure(figsize=(14, 5))
pli = PiecewiseLinearInterpolation(x, y)
pli.fit_interp()
yi_pli = pli.predict_x0(xi) # 推测
plt.subplot(121)
pli.plt_interpolation(is_show=False)

bsi = BSplineInterpolation(x, y)
bsi.fit_interp()
yi_bsi = bsi.predict_x0(xi) # 推测
plt.subplot(122)
bsi.plt_interpolation(is_show=False)
plt.show()

# 面积计算
S1 = np.sum((yi_pli[:-1] + yi_pli[1:]) / 2) * 0.01
S2 = np.sum((yi_bsi[:-1] + yi_bsi[1:]) / 2) * 0.01
print("分段线性插值得面积: %.5f, 三次均匀 B 样条插值得面积: %.5f" % (S1, S2))
```

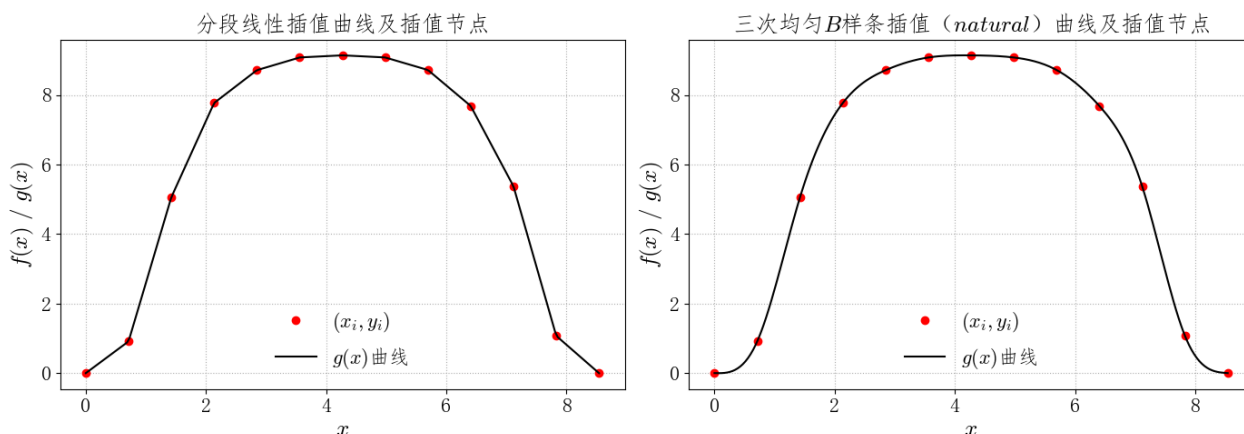


图 2-3 分段线性插值和三次均匀 B 样条插值结果图像

如图 2 - 3 所示, 三次均匀 B 样条插值要比分段线性插值光滑地多. 由于划分较细, 推测预测值较多, 故两种方法所计算的面积差异不大, 其中分段线性插值得面积约为 51.68899, 三次均匀 B 样条插值得面积约为 51.69276.

4. 已知函数 $f(x, y) = xe^{-x^2-y^2}$, $x, y \in [-2, 2]$, 等分 25 组离散数据点并求解 $z_i = f(x_i, y_i)$, 以此离散数据点 (x_i, y_i, z_i) 进行分片双线性插值和二元三点拉格朗日插值, 并求插值节点

$$\{(-1.50, -1.25), (-0.58, -0.69), (0.58, 0.78), (1.65, 1.78)\}$$

处所对应的插值 $\hat{z}_k$, 分析误差误差.

求解代码如下:

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_2_exp4.py
"""

import numpy as np
import matplotlib.pyplot as plt
from interpolation_02.slice_bilinear_interpolation import SliceBiLinearInterpolation
from interpolation_02.bivariate_three_points_lagrange import BivariateThreePointsLagrange

fh = lambda x, y: x * np.exp(-x ** 2 - y ** 2)
x = np.linspace(-2, 2, 25, endpoint=True) # 等距划分
y = np.linspace(-2, 2, 25, endpoint=True)
xi, yi = np.meshgrid(x, y)
Z = xi * np.exp(-xi ** 2 - yi ** 2)
x0 = np.array([-1.50, -0.58, 0.58, 1.65])
y0 = np.array([-1.25, -0.69, 0.78, 1.78])

# 分片双线性插值
sbi = SliceBiLinearInterpolation(x, y, Z.T, x0, y0)
z0 = sbi.fit_2d_interp()
print("插值点值: ", z0)
print("分片双线性插值误差: ", x0 * np.exp(-x0 ** 2 - y0 ** 2) - z0)
fig = plt.figure(figsize=(14, 5))
ax = fig.add_subplot(121, projection='3d')
sbi.plt_3d_surface(ax, "节点数$25$", fh=fh)

# 二元三点拉格朗日插值
btpl = BivariateThreePointsLagrange(x, y, Z.T, x0, y0)
z0 = btpl.fit_interp_2d()
print("插值点值: ", z0)
print("二元三点拉格朗日插值误差: ", x0 * np.exp(-x0 ** 2 - y0 ** 2) - z0)
ax = fig.add_subplot(122, projection='3d')
btpl.plt_3d_surface(ax, "节点数$25$", fh=fh)
plt.show()
```

如图 2 - 4 所示为两种方法的二维插值曲面图, 由于绘制时, 设置了网格点数 200×200 , 直观上差异

性不大, 但是从 MSE 可知, 二元三点拉格朗日插值的精度要高得多. 从表 2 - 5 也可以看出, 二元三点拉格朗日插值在插值点的绝对值误差精度要比分片双线性插值高得多.

分片双线性 (节点数25), $MSE=3.52321e-06$

二元三点拉格朗日 (节点数25), $MSE=1.00421e-07$

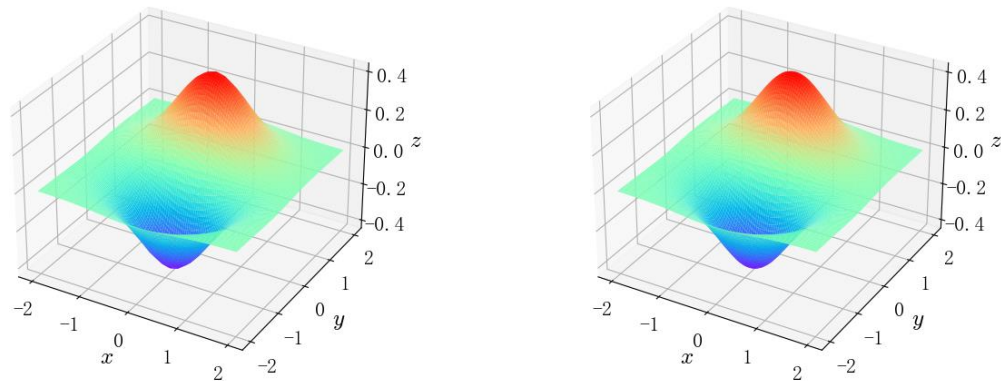


图 2 - 4 两种方法的二维插值曲面图

表 2 - 5 两种方法插值结果及其绝对值误差

k	x_k	y_k	分片双线性插值	绝对值误差	二元三点拉格朗日插值	绝对值误差
1	-1.50	-1.25	-0.03362676	0.00048744	-0.03315744	1.81189315e-05
2	-0.58	-0.69	-0.25329278	0.00408066	-0.25751265	1.39210997e-04
3	0.58	0.78	0.22207069	0.00340855	0.22521158	2.67664883e-04
4	1.65	1.78	0.00474368	0.00018228	0.00458447	2.30629191e-05

第 3 章 实验内容参考答案

1. 已知函数 $f(x) = \sin\left(2x + \frac{\pi}{6}\right)e^{-0.4x}$, $x \in [-\pi, \pi]$, 完成如下内容, 并进行对比分析和可视化.

(1) 利用 $T_6(x)$ 和 $T_{21}(x)$ 的零点作插值点, 构造拉格朗日插值多项式 $L_5(x)$ 和 $L_{20}(x)$.

(2) 以 5 级、15 级和 25 级分别进行切比雪夫级数和勒让德级数逼近.

(3) 以逼近阶次 10 和 20 分别进行最佳一致逼近和最佳平方逼近.

求解代码如下:

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_3_exp1.py
"""

import numpy as np
import sympy
import matplotlib.pyplot as plt
from function_approximation_03.chebyshev_zero_points_interp import ChebyshevZeroPointsInterpolation
from function_approximation_03.chebyshev_series_approximation import ChebyshevSeriesApproximation
from function_approximation_03.legendre_series_approximation import LegendreSeriesApproximation
from function_approximation_03.best_uniform_approximation import BestUniformApproximation
from function_approximation_03.best_square_approximation import BestSquarePolynomialApproximation

fh = lambda x: np.sin(2 * x + np.pi / 6) * np.exp(-0.4 * x)

# 切比雪夫多项式零点插值逼近
orders = np.array([5, 20], dtype=np.int)
plt.figure(figsize=(14, 5))
for i, order in enumerate(orders):
    czpi = ChebyshevZeroPointsInterpolation(fh, orders=order, x_span=[-np.pi, np.pi])
    czpi.fit_approximation()
    print("最大逼近多项式绝对值误差: ", czpi.max_abs_error)
    print("零点值: ", czpi.terms_zeros)
    print("拉格朗日插值逼近多项式系数以及阶次: ")
    print(czpi.poly_coefficient)
    print(czpi.polynomial_orders)
    plt.subplot(121 + i)
    czpi.plot_approximation(is_show=False, is_fh_marker=True)
plt.show()

# 切比雪夫级数逼近
t = sympy.Symbol("t")
fh_sym = sympy.sin(2 * t + np.pi / 6) * sympy.exp(-0.4 * t)
orders = np.array([5, 15, 25], dtype=np.int)
plt.figure(figsize=(18, 4))
```

```

for i, order in enumerate(orders):
    csa = ChebyshevSeriesApproximation(fh_sym, x_span=[-np.pi, np.pi], k=order)
    csa.fit_approximation()
    print("切比雪夫级数逼近最大逼近多项式绝对值误差: ", csa.max_abs_error)
    plt.subplot(131 + i)
    if i == 0:
        csa.plt_approximate(is_show=False, is_fh_marker=False)
    else:
        csa.plt_approximate(is_show=False, is_fh_marker=True)
plt.show()
print("=" * 80)

# 勒让德级数逼近
plt.figure(figsize=(18, 4))
for i, order in enumerate(orders):
    lsa = LegendreSeriesApproximation(fh_sym, x_span=[-np.pi, np.pi], k=order)
    lsa.fit_approximation()
    print("勒让德级数逼近最大逼近多项式绝对值误差: ", lsa.max_abs_error)
    plt.subplot(131 + i)
    if i == 0:
        lsa.plt_approximate(is_show=False, is_fh_marker=False)
    else:
        lsa.plt_approximate(is_show=False, is_fh_marker=True)
plt.show()

# 最佳一致逼近
plt.figure(figsize=(14, 5))
orders = [10, 20]
for i, order in enumerate(orders):
    plt.subplot(121 + i)
    bua = BestUniformApproximation(fh_sym, k=order, interval=[-np.pi, np.pi], eps=1e-10)
    bua.fit_approximation()
    bua.plt_approximate(is_show=False, is_fh_marker=True)
    print("order=%d" % order, ", 最佳一致逼近多项式的逼近误差精度为: ", bua.abs_error)
    print("order=%d" % order, ", 最佳一致逼近多项式的最大绝对误差为: ", bua.max_abs_error)
    print("交错点组: ", bua.cross_point_group)
plt.show()

# 最佳平方逼近
plt.figure(figsize=(14, 5))
orders = [10, 20]
for i, order in enumerate(orders):
    plt.subplot(121 + i)
    bspa = BestSquarePolynomialApproximation(fh_sym, k=order, interval=[-np.pi, np.pi])
    bspa.fit_approximation()
    bspa.plt_approximate(is_show=False, is_fh_marker=True)

```



```
print("order=%d" % order, ", 最佳平方逼近多项式的逼近误差精度为: ", bsqa.mae)
print("order=%d" % order, ", 最佳平方逼近多项式的最大绝对误差为: ", bsqa.max_abs_error)
plt.show()
```

(1) 切比雪夫多项式零点插值逼近求解结果

1) 针对 $T_6(x)$ 零点插值, 最大逼近多项式绝对值误差 1.00521490834651. $T_6(x)$ 的零点值: [3.03454548, 2.22144147, 0.81310401, -0.81310401, -2.22144147, -3.03454548], 拉格朗日插值逼近多项式:

$$L_5(x) = 0.047101x^5 - 0.012215x^4 - 0.63915x^3 + 0.29520x^2 + 1.53345x - 0.50522.$$

2) 针对 $T_{21}(x)$ 零点插值, 最大逼近多项式绝对值误差: 1.3409685806209382e-08. 其 $T_{21}(x)$ 的零点值以及 $L_{20}(x)$ 多项式系数可通过算法执行输出, 然后构造 $L_{20}(x)$, 此处略去.

切比雪夫多项式零点插值逼近示意图如图 3-1 所示, 可见零点数为 21 时的逼近精度非常高.

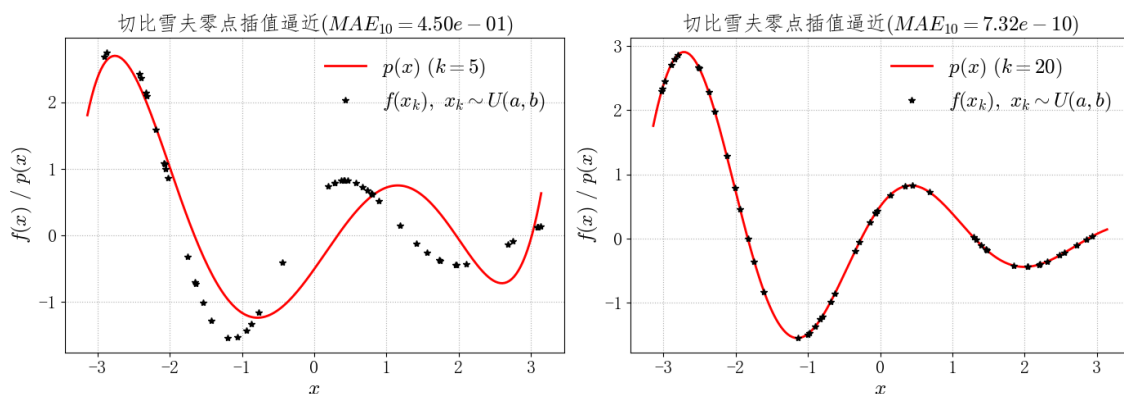


图 3-1 切比雪夫多项式零点插值逼近

(2) 切比雪夫级数和勒让德级数逼近求解结果

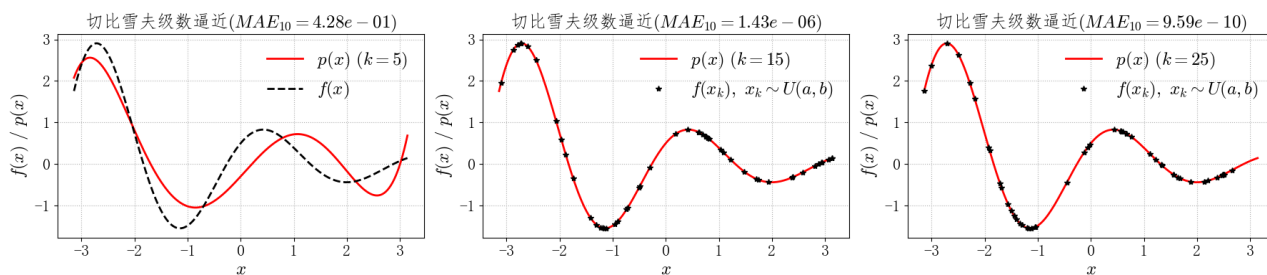


图 3-2 切比雪夫级数逼近

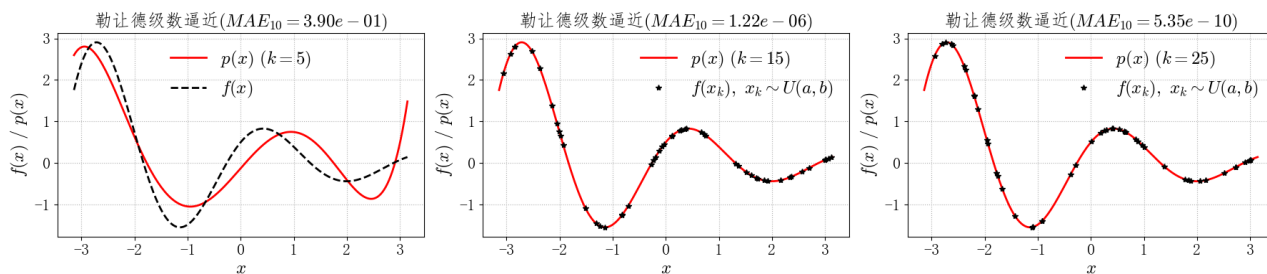


图 3-3 勒让德级数逼近

表 3-1 两种逼近方法在各阶次下的最大绝对值误差

k	5	15	25
切比雪夫级数逼近	0.8011627886693931	2.938834856830441e-06	2.300195034443675e-09
勒让德级数逼近	1.3332554125558462	1.0907087928113679e-05	3.7581859846369525e-09

(3) 最佳一致逼近和最佳平方逼近.

1) 最佳一致逼近最终的误差:

order=10, 最佳一致逼近多项式的逼近误差精度为: {'u': -0.005826362025718471, 'tol': 0.0}

order=10, 最佳一致逼近多项式的最大绝对误差为: 0.006472082073718521

order=20, 最佳一致逼近多项式的逼近误差精度为: {'u': 2.3678749890418126e-10, 'tol': 0.0}

order=20, 最佳一致逼近多项式的最大绝对误差为: 3.966373340880125e-10

2) 最佳平方逼近最终的误差:

order=10, 最佳平方逼近多项式的逼近误差精度 MAE_{10} 为: 0.003280989752225642

order=10, 最佳平方逼近多项式的最大绝对误差为: 0.019499731553764166

order=20, 最佳平方逼近多项式的逼近误差精度 MAE_{10} 为: 1.5235855776074514e-08

order=20, 最佳平方逼近多项式的最大绝对误差为: 9.860001992656464e-08

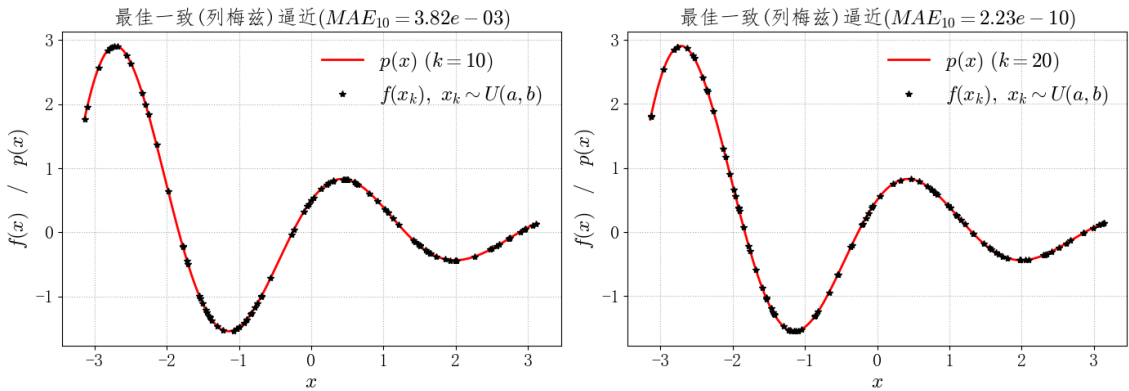


图 3-4 最佳一致逼近示意图

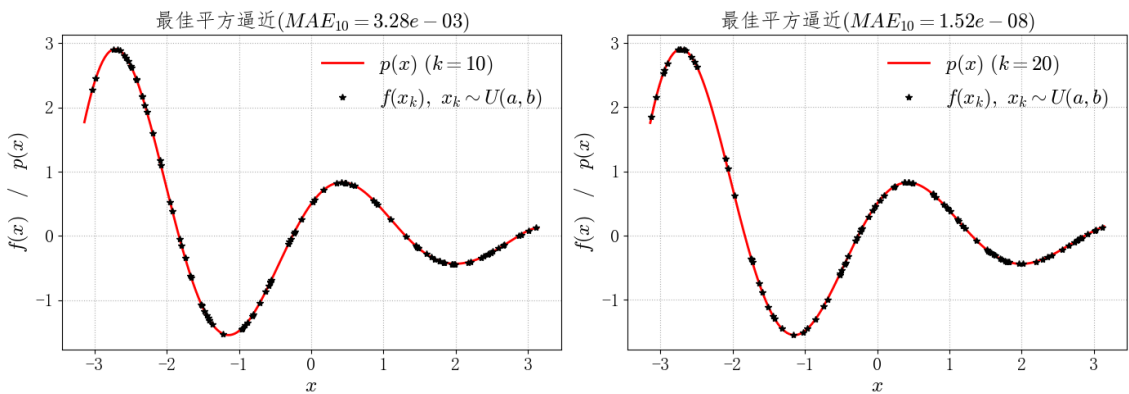


图 3-5 最佳平方逼近示意图

2. 采用 FFT 算法确定函数 $f(x)=0.5x^2\cos 4x+\sin x^2$, $x\in[-\pi,\pi]$ 的 16 次和 64 次三角逼近多项式.

求解代码如下:

```

# -*- coding: UTF-8 -*-
"""
@file_name: chapter_3_exp2.py
"""

import numpy as np
import matplotlib.pyplot as plt
from function_approximation_03.fast_fourier_transform import FastFourierTransformApproximation

fh = lambda x: 0.5 * x ** 2 * np.cos(4 * x) + np.sin(x ** 2)
plt.figure(figsize=(14, 5))
x = np.linspace(-np.pi, np.pi, 16, endpoint=False)
fft = FastFourierTransformApproximation(y=fh(x), interval=[-np.pi, np.pi], fun=fh)
fft.fit_fourier()
print(fft.Ak) # 展开后的余弦项系数
print(fft.Bk) # 展开后的正弦项系数
plt.subplot(121)
fft.plt_approximate(is_show=False, is_fh_marker=False) # 可视化
x = np.linspace(-np.pi, np.pi, 64, endpoint=False)
fft = FastFourierTransformApproximation(y=fh(x), interval=[-np.pi, np.pi], fun=fh)
fft.fit_fourier()
plt.subplot(122)
fft.plt_approximate(is_show=False, is_fh_marker=True) # 可视化
plt.show()

```

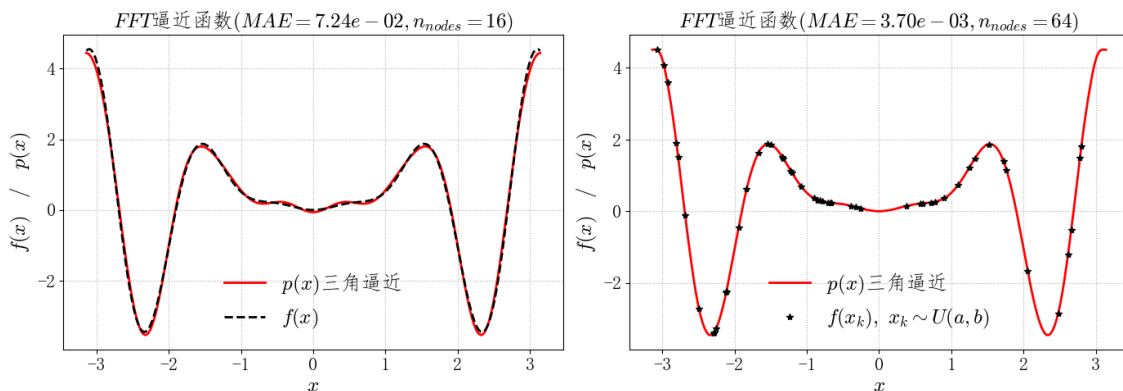


图3-6 FFT三角逼近示意图

对于16次三角逼近多项式, 平均绝对值误差 $7.2430283074e-02$, 最大绝对值误差 $2.5119196376e-01$. 对于64次三角逼近多项式, 平均绝对值误差 $3.7038831560e-03$, 最大绝对值误差: $4.6917548882e-02$.

被逼近函数为偶函数, 正弦项系数几乎为零. 余弦项系数 $[0.59368757, 0.06482945, 0.23663063, -1.67468429, 1.82100266, -0.76449617, -0.03948863, 0.12210052, -0.12547591]$; 正弦项系数 $[0.00000000e+00, -9.63505829e-16, 1.04328072e-15, -1.57983692e-15, 1.41938997e-15, -9.49934800e-16, 8.84018399e-16, -6.86881789e-16, -1.22930936e-16]$.

3. 已知实验数据 $(x_k, f(x_k))$, $k=1, 2, \dots, 16$, 如表3-2所示, 以此实验数据进行多项式最小二乘拟合和正交多项式最小二乘拟合, 拟合阶次分别为2、3、5、8次, 给出拟合的多项式表达式, 分析误差, 并进行可视

化.

表 3-2 实验数据 ($k=1, 2, \dots, 16$)

x_k	-1	-0.6	-0.2	0.2	0.6	1	1.4	1.8
y_k	-2.11802123	-0.70930496	2.32377848	0.85944498	0.98580008	-0.51909279	0.4820484	-2.32350214
x_k	2.2	2.6	3	3.4	3.8	4.2	4.6	5
y_k	-2.60275523	-4.15452315	-3.27630398	-3.65130248	-1.56691817	0.77433117	2.37330911	8.89881447

求解代码如下:

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_3_exp3.py
"""

import numpy as np
import matplotlib.pyplot as plt
from function_approximation_03.least_square_curve_fitting import LeastSquarePolynomialCurveFitting
from function_approximation_03.orthogonal_polynomial_least_squares_fitting import
OrthogonalPolynomialLSFitting

# 实验数据
x = np.linspace(-1, 5, 16)
y = np.array([-2.11802123, -0.70930496, 2.12377848, 0.85944498, 0.98580008, -0.51909279, 0.4820484,
              -2.32350214, -2.60275523, -4.15452315, -3.27630398, -3.65130248, -1.56691817,
              0.77433117, 2.37330911, 8.89881447])

# 多项式最小二乘拟合
orders = [2, 3, 5, 8]
xi = np.linspace(min(x), max(x), 100, endpoint=True)
plt.figure(figsize=(14, 5))
plt.subplot(121)
line_style = ["-", "--", "-.", ":"]
for i, order in enumerate(orders):
    lspcf = LeastSquarePolynomialCurveFitting(x, y, k=order)
    lspcf.fit_ls_curve()
    print("order = %d: " % order, lspcf.poly_coefficient, lspcf.mse)
    yi = lspcf.predict_x0(xi)
    plt.plot(xi, yi, line_style[i], lw=2, label="$k = %d, \ MSE = %.2e$" % (order, lspcf.mse))
plt.plot(x, y, "ko", label="$ (x_k, y_k)$")
plt.legend(frameon=False, fontsize=18)
plt.grid(ls=":")
plt.tick_params(labelsize=16) # 刻度字体大小 16
plt.xlabel("$x$", fontdict={"fontsize": 18})
plt.ylabel("$p(x)$", fontdict={"fontsize": 18})
plt.title("不同阶次的多项式最小二乘拟合", fontdict={"fontsize": 18})
```

```

print("=" * 80)

# 正交多项式最小二乘拟合
plt.subplot(122)
for i, order in enumerate(orders):
    oplsf = OrthogonalPolynomialLSFitting(x, y, k=order)
    oplsf.fit_orthogonal_poly()
    print("order = %d: " % order, oplsf.poly_coefficient, oplsf.mse)
    yi = oplsf.predict_x0(xi)
    plt.plot(xi, yi, line_style[i], lw=2, label="$k = %d, \ MSE = %.2e$" % (order, oplsf.mse))
plt.plot(x, y, "ko", label="$$(x_k, y_k)$")
plt.legend(frameon=False, fontsize=18)
plt.grid(ls=":")
plt.tick_params(labelsize=16) # 刻度字体大小 16
plt.xlabel("$x$", fontdict={"fontsize": 18})
plt.ylabel("$p(x)$", fontdict={"fontsize": 18})
plt.title("不同阶次的正交多项式最小二乘曲线拟合", fontdict={"fontsize": 18})
plt.show()

```

无论是多项式最小二乘拟合, 还是正交多项式最小二乘拟合, 结果一致. 仅有个别系数由于舍入误差的存在, 存在极小的差异. 可通过求解代码打印输出各阶次的拟合系数, 然后构造拟合多项式, 如拟合阶次为 5 阶的多项式为

$$P(x) = 0.0069583x^5 - 0.070621x^4 + 0.68036x^3 - 2.32960x^2 + 0.55284x + 1.33360.$$

如图 3-7 所示, 随着阶次的增加, 均方误差 MSE 在逐渐缩小, 如图例所示.

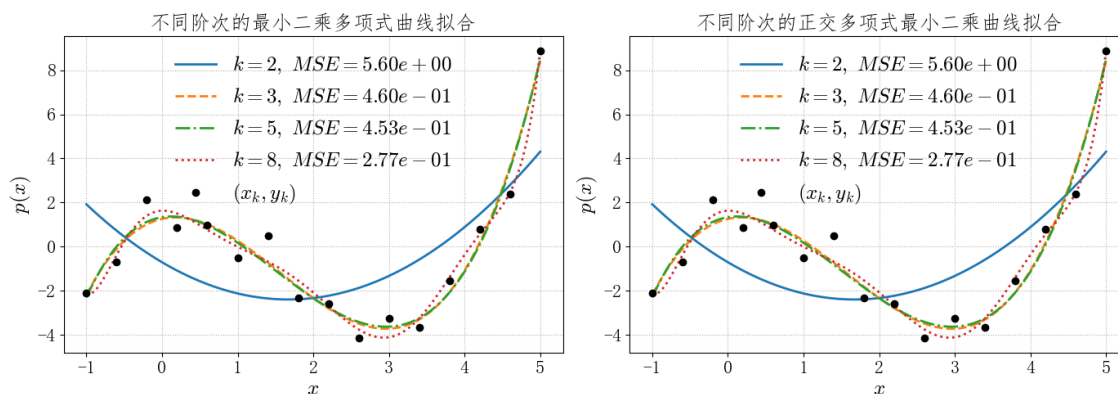


图 3-7 不同阶次下的最小二乘拟合曲线和正交多项式拟合曲线

4. 求 $f(x) = \sin 3x \cdot \ln(1 + 0.5x)$, $x \in [0, 2]$ 的帕德逼近 $R(6, 6)$ 、 $R(10, 10)$ 和 $R(15, 15)$.

求解代码如下:

```

# -*- coding: UTF-8 -*-
"""
@file_name: chapter_3_exp4.py
"""

import sympy
import matplotlib.pyplot as plt

```

```
from function_approximation_03.pade_rational_fraction_approximation import PadeRationalFractionApproximation

t = sympy.Symbol("t")
fun = sympy.sin(3 * t) * sympy.log(1 + 0.5 * t)
fun_expr = sympy.lambdify(t, fun, "numpy")

plt.figure(figsize=(18, 4))
orders = [6, 10, 15]
for i, order in enumerate(orders):
    plt.subplot(131 + i)
    prfa = PadeRationalFractionApproximation(fun, order=order)
    prfa.fit_rational_fraction()
    print(prfa.rational_fraction)
    if i == 0:
        prfa.plt_approximate(0, 2, is_show=False, is_fh_marker=False)
    else:
        prfa.plt_approximate(0, 2, is_show=False, is_fh_marker=True)
plt.show()
```

帕德逼近有理分式项式，可通过求解代码打印输出，然后构造. 如 $R(6, 6)$ 形式如下：

$$R(6, 6) = \frac{0.31275t^6 + 0.10220t^5 - 1.71067t^4 - 0.11158t^3 + 1.5t^2}{0.0048442t^6 + 0.0067308t^5 + 0.052440t^4 + 0.053197t^3 + 0.32012t^2 + 0.17561t + 1}.$$

如图 3 - 8 所示，为帕德逼近，均方误差如标题所示.

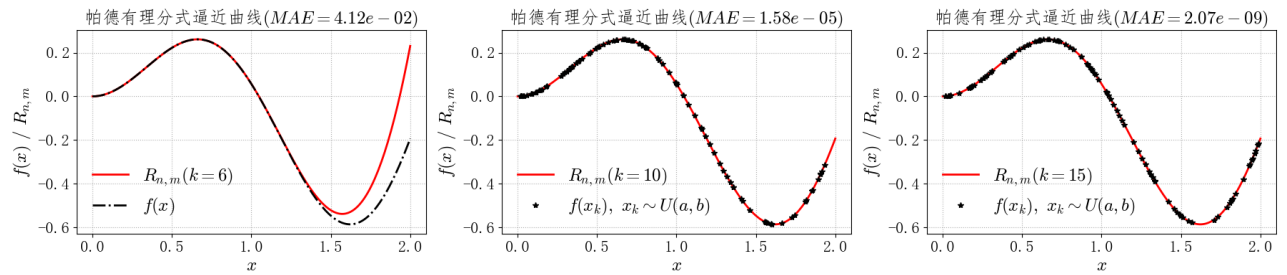


图 3 - 8 帕德逼近示意图

表 3 - 3 帕德逼近误差

误差类型	$R(6, 6)$	$R(10, 10)$	$R(15, 15)$
平均绝对值误差	4.1237747736e-02	1.5841727394e-05	2.0696640514e-09
最大绝对值误差	4.2498471825e-01	2.9160105494e-04	5.7769843353e-08

第4章 实验内容参考答案

1. 求解积分 $I = \int_0^5 x^5 e^{-x} \sin x dx$, 已知精确值 $-e^{-5} \left(3660 \cos 5 + \frac{975 \sin 5}{2} \right) - 15$, 完成如下内容:

- (1) 试用牛顿—科特斯积分公式, 划分区数数为 2 到 8, 分析误差.
- (2) 试用复合梯形公式、复合辛普森公式和复合科特斯公式求解近似积分, 并分析误差, 自设划分区数数.
- (3) 试用龙贝格求积公式求解积分近似值, 外推次数为 6 和 10, 分析误差.

求解代码如下:

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_4_exp1.py
"""

import numpy as np
import sympy
from numerical_integration_04.newton_cotes_integration import NewtonCotesIntegration
from numerical_integration_04.composite_quadrature_formula import CompositeQuadratureFormula
from numerical_integration_04.romberg_acceleration_quad import RombergAccelerationQuadrature
from Experiment.util_font import * # 导入字体文件

int_fun = lambda x: x ** 5 * np.exp(-x) * np.sin(x)
I_f = -np.exp(-5) * (3660 * np.cos(5) + 975 * np.sin(5) / 2) - 15 # 积分精确值

# (1) 牛顿—科特斯积分
num = np.linspace(2, 8, 7) # 划分区数数
int_res = np.zeros(len(num)) # 存储各划分区数数下的积分值
for i, n in enumerate(num):
    nci = NewtonCotesIntegration(int_fun, int_interval=[0, 5], interval_num=n) # 实例化对象
    nci.fit_cotes_int() # 求解积分
    print("n = %d, 科特斯系数: \n" % n, nci.cotes_coefficient) # 打印科特斯系数
    int_res[i] = nci.int_value # 存储积分值
print("=" * 80)
print("积分精确值: %.8f" % I_f)
print("各划分区数数下的积分近似值: \n", int_res) # 误差计算
print("各划分区数数下的积分绝对值误差: \n", abs(I_f - int_res)) # 误差计算
print("=" * 80)

# (2) 复合积分公式
x = sympy.Symbol("x") # 定义符号变量
fun = x ** 5 * sympy.exp(-x) * sympy.sin(x) # 被积函数, 符号函数
for type in ["trapezoid", "simpson", "cotes"]:
    cqf = CompositeQuadratureFormula(fun, [0, 5], interval_num=16, int_type=type, is_remainder=False)
    cqf.fit_int()
    print("积分方法: %s, 积分近似值: %.8f, 绝对值误差: %.8e" % (type, cqf.int_value, abs(I_f - cqf.int_value)))
```



```

print("=" * 80)

interval_n = np.arange(5, 41, 1)
int_type = ["trapezoid", "simpson", "cotes"]
int_error = np.zeros((len(interval_n), 3))
for i in range(3):
    for j, n in enumerate(interval_n):
        cqf = CompositeQuadratureFormula(fun, [0, 5], n, int_type=int_type[i])
        cqf.fit_int()
        int_error[j, i] = np.abs(I_f - cqf.int_value)
plt.figure(figsize=(7, 5))
plt.semilogy(interval_n, int_error[:, 0], "--", lw=1.5, label="$Trapezoid$")
plt.semilogy(interval_n, int_error[:, 1], "o-", lw=1.5, label="$Simpson$")
plt.semilogy(interval_n, int_error[:, 2], "s-", lw=1.5, label="$Cotes$")
plt.xlabel("区间数 $n$", fontdict={"fontsize": 18})
plt.ylabel("绝对误差 $\epsilon$", fontdict={"fontsize": 18})
plt.legend(fontsize=16, frameon=False, ncol=3)
plt.title(r"三种复合求积公式积分误差$\epsilon=\|I - I^n\|$变化曲线", fontdict={"fontsize": 18})
plt.tick_params(labelsize=16) # 刻度字体大小 16
plt.grid(ls=":")
plt.show()

# (3) 龙贝格求积公式
accelerate_num = np.arange(6, 11, 1)
for an in accelerate_num:
    raq = RombergAccelerationQuadrature(int_fun, int_interval=[0, 5], accelerate_num=an)
    raq.fit_int() # 计算龙贝格积分
    if an == 6:
        print(raq.Romberg_acc_table) # 打印外推计算过程表
    print("%d: 积分近似值: %.15f, 绝对值误差: %.15e" % (an, raq.int_value, abs(I_f - raq.int_value)))

```

(1) 牛顿—科特斯积分结果

表 4-1 牛顿—科特斯积分近似值以及绝对值误差

划分区间数	2	3	4	5
积分近似值	-0.83459062	-13.33173287	-20.89992668	-19.8386067
绝对值误差	18.0109445	5.51380224	2.05439156	0.9930716
划分区间数	6	7	8	
积分近似值	-18.45295921	-18.63329789	-18.9289098	
绝对值误差	0.39257591	0.21223723	0.08337468	

(2) 复合积分公式积分结果

设置区间划分数 16, 积分近似值以及绝对误差如表 4-2 所示. 随着划分区间数的增加, 积分近似值绝对值误差的变化曲线如图 4-1 所示.

表 4-2 复合积分公式积分近似值以及绝对值误差

复合积分方法	积分近似值	绝对值误差
复合梯形公式	-18.79677942	4.87556921e-02
复合辛普森公式	-18.84557236	3.72457425e-05
复合科特斯公式	-18.84553510	1.10843885e-08

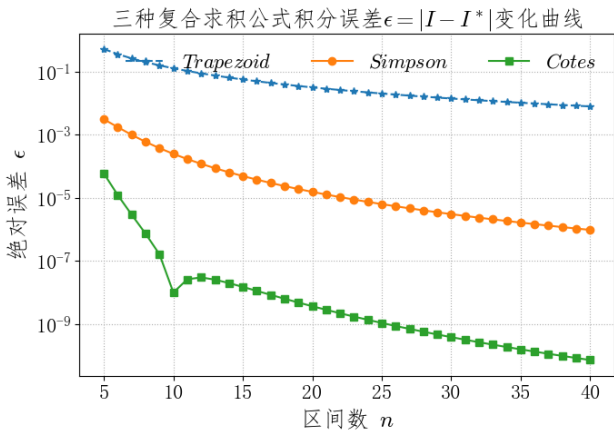


图 4 - 1 三种复合积分公式随划分区间数的增加积分近似解精度变化曲线

(3) 龙贝格求积公式

由于龙贝格求积公式积分精度较高，为便于比较，积分近似值保留小数点后 15 位，结果如表 4 - 3 所示。

表 4 - 3 龙贝格求积公式积分近似值以及绝对值误差

外推次数	6	7	8
积分近似值	-18.845535122500678	-18.845535115271545	-18.845535115270113
绝对值误差	7.230568144223071e-09	1.435296326235402e-12	3.552713678800501e-15
外推次数	9	10	
积分近似值	-18.845535115270106	-18.845535115270106	
绝对值误差	3.552713678800501e-15	3.552713678800501e-15	

2. 含参变量积分，试绘制出 $I(\alpha)$ 与 α 的关系曲线 (零点数为 10)：

- (1) $I(\alpha)=\int_0^{+\infty} e^{-x} \sin (\alpha^2 x) \mathrm{d} x, \alpha \in[0,1.5]$ ，试用高斯—拉盖尔求积公式求解。
- (2) $I(\alpha)=\int_1^2 x^2 \sin (\alpha^2 x) \mathrm{d} x, \alpha \in[0,5]$ ，试用高斯—勒让德求积公式求解。

求解代码如下：

```
# -*- coding: UTF-8 -*-
"""
@file_name:chapter_4_exp2.py
"""
import numpy as np
from numerical_integration_04.gauss_laguerre_int import GaussLaguerreIntegration
from numerical_integration_04.gauss_legendre_int import GaussLegendreIntegration
from Experiment.util_font import * # 导入字体文件
```

```

# (1) 高斯—拉盖尔求积公式
alpha = np.linspace(0, 1.5, 20)
fun = lambda x, alpha: np.exp(-x) * np.sin(alpha ** 2 * x)
int_values = []

for alpha_ in alpha:
    int_fun = lambda x: fun(x, alpha_)
    gli = GaussLaguerreIntegration(int_fun, [0, np.infty], 10)
    gli.fit_int()
    int_values.append(gli.int_value)

# 可视化
plt.figure(figsize=(14, 5))
plt.subplot(121)
plt.plot(alpha, int_values, "-o")
plt.xlabel(r"$\alpha$", fontdict={"fontsize": 18})
plt.ylabel(r"$I(\alpha)$", fontdict={"fontsize": 18})
plt.title(r"高斯—拉盖尔: $I(\alpha)$与$\alpha$关系曲线", fontdict={"fontsize": 18})
plt.tick_params(labelsize=16) # 刻度字体大小 16
plt.grid(ls=":")

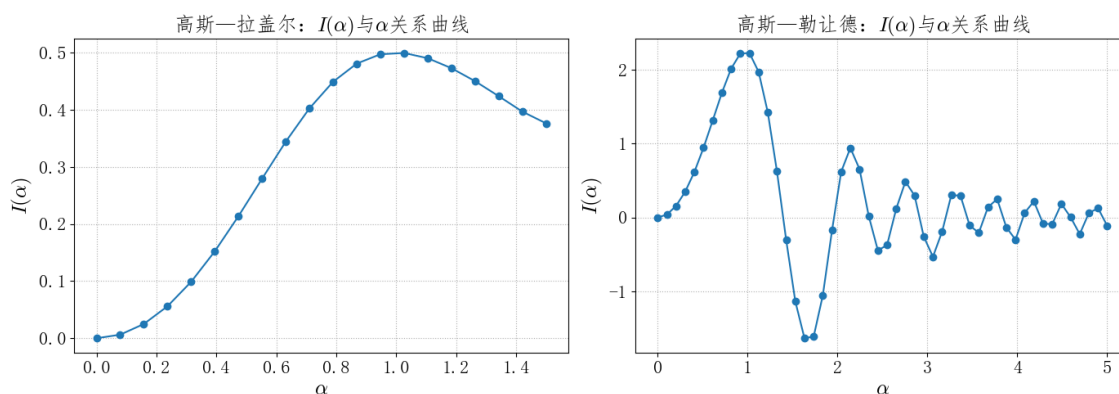
# (2) 高斯—勒让德求积公式
alpha = np.linspace(0, 5, 50)
fun = lambda x, alpha: x ** 2 * np.sin(alpha ** 2 * x)
int_values = []

for alpha_ in alpha:
    int_fun = lambda x: fun(x, alpha_)
    gli = GaussLegendreIntegration(int_fun, [1, 2], 10)
    gli.fit_int()
    int_values.append(gli.int_value)

# 可视化
plt.subplot(122)
plt.plot(alpha, int_values, "-o")
plt.xlabel(r"$\alpha$", fontdict={"fontsize": 18})
plt.ylabel(r"$I(\alpha)$", fontdict={"fontsize": 18})
plt.title(r"高斯—勒让德: $I(\alpha)$与$\alpha$关系曲线", fontdict={"fontsize": 18})
plt.tick_params(labelsize=16) # 刻度字体大小 16
plt.grid(ls=":")
plt.show()

```

结果如图 4-2 所示, 其中高斯—拉盖尔对 $\alpha \in [0, 1.5]$ 等距划分 20 个数值, 高斯—勒让德对 $\alpha \in [0, 5]$ 等距划分 50 个数值, 略去具体积分近似值。

图 4-2 含参变量积分 $I(\alpha)$ 与 α 的关系曲线

3. 采用自适应复合辛普森公式和高斯-勒让德公式, 计算二重积分 $I = \iint_D e^{-x^2-y^2} dx dy$, 其中平面区域 $D: x^2 + y^2 \leq 4$, 已知积分精确值为 $\pi(1 - e^{-4})$.

首先讲积分转化为累次积分形式

$$I = \iint_D e^{-x^2-y^2} dx dy = \int_0^{2\pi} \int_0^2 r e^{-r^2} dr d\theta.$$

求解代码如下:

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_4_exp3.py
"""
import numpy as np
from numerical_integration_04.composite_simpson_2d_integration import CompositeSimpsonDoubleIntegration
from numerical_integration_04.gauss_legendre_2d_integration import GaussLegendreDoubleIntegration

# 自适应复合辛普森公式
int_fun = lambda theta, r: r * np.exp(-r ** 2)
I_f = np.pi * (1 - np.exp(-4)) # 精确值

csdi = CompositeSimpsonDoubleIntegration(int_fun, [0, 2 * np.pi], [0, 2], eps=1e-15, increment=20)
csdi.fit_2d_int()
print("划分区间数: %d, 积分近似值: %.15f" % (csdi.sub_interval_num, csdi.int_value))
print("绝对值误差: %.15e" % abs(I_f - csdi.int_value))
csdi.plt_precision(is_show=True, exact_int=I_f)

# 高斯-勒让德二重积分
gl2d = GaussLegendreDoubleIntegration(int_fun, [0, 2 * np.pi], [0, 2], zeros_num=15)
gl2d.cal_2d_int()
print("积分近似值: %.15f" % gl2d.int_value)
print("绝对值误差: %.15e" % abs(I_f - gl2d.int_value))
```

(1) 自适应复合辛普森公式

每次递增区间数为 20, 精度要求 $\varepsilon = 10^{-15}$. 如图 4-3 所示, 满足精度要求下共划分区间数 1480, 积分

近似值为 3.084052377011181, 绝对值误差 3.863576125695545e-14, 最终收敛精度 4.440892098500626e-16.

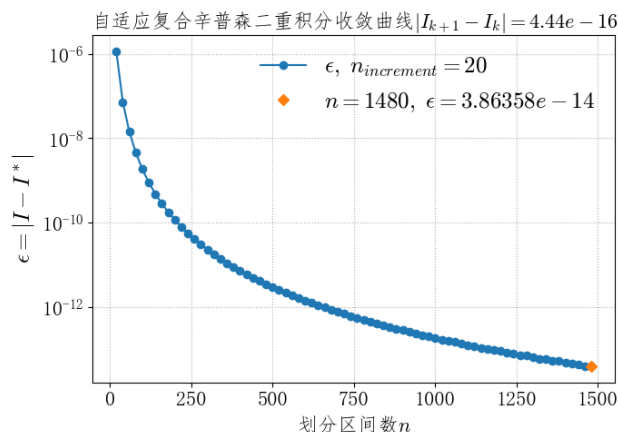


图 4-3 自适应复合辛普森二重积分收敛曲线 (区间数递增 20)

(2) 高斯—勒让德二重积分

零点数设置为 15, 积分近似值 3.084052377011149, 绝对值误差 6.661338147750939e-15.

4. 采用高斯—勒让德积分法计算如下一般区域积分, 分析误差.

$$(1) I_1 = \int_0^{\pi/4} \int_{\sin x}^{\cos x} (2y \sin x + \cos^2 x) dy dx = \frac{5\sqrt{2} - 4}{6},$$

$$(2) I_2 = \int_0^{\pi} \int_x^{\pi} \int_0^{xy} \frac{1}{y} \sin \frac{z}{y} dz dy dx = 0.5\pi^2 + 2.$$

求解代码如下:

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_4_exp4.py
"""
import numpy as np
from numerical_integration_04.gauss_2d_general_region import Gauss2DGeneralIntegration
from numerical_integration_04.gauss_3d_general_region import Gauss3DGeneralIntegration

# 一般区域的二重积分
fh = lambda x, y: 2 * y * np.sin(x) + np.cos(x) ** 2
a, b, c_x, d_x = 0, np.pi / 4, lambda x: np.sin(x), lambda x: np.cos(x)
I_f = (5 * np.sqrt(2) - 4) / 6
g2dgi = Gauss2DGeneralIntegration(fh, a, b, c_x, d_x, np.array([10, 10]))
int_val = g2dgi.cal_2d_int()
print("积分近似值: %.15f, 绝对值误差: %.15e" % (int_val, abs(I_f - int_val)))

# 一般区域的三重积分
fh = lambda x, y, z: np.sin(z / y) / y
I_f = 0.5 * np.pi ** 2 + 2
a, b, c_x, d_x = 0, np.pi, lambda x: 0 * x, lambda x: x
alpha_xy, beta_xy = lambda x, y: 0 * x * y, lambda x, y: x * y
g3dgi = Gauss3DGeneralIntegration(fh, a, b, c_x, d_x, alpha_xy, beta_xy, np.array([10, 10, 15]))
```

```
int_val = g3dgi.cal_3d_int()
print("积分近似值: %.15f, 绝对值误差: %.15e" % (int_val, abs(I_f - int_val)))
```

(1) 设置零点数 [10, 10]

积分近似值: 0.511844635310912, 绝对值误差: 6.661338147750939e-16.

(2) 设置零点数 [10, 10, 15]

积分近似值: 6.934802200544678, 绝对值误差: 8.881784197001252e-16.

5. 已知离散数据由表 4 - 4 提供, 试采用平均抛物插值和样条函数插值计算离散数据积分 (x_k 可由 [0, 2] 等距产生).

表 4 - 4 采样离散数据 ($k = 1, 2, \cdots, 16$)

x_k	0.0	0.13333333	0.26666667	0.4	0.53333333	0.66666667
y_k	5.36205899	8.28004495	10.8856753	10.94805195	9.74022299	9.74022299
x_k	0.8	0.93333333	1.06666667	1.2	1.33333333	1.46666667
y_k	10.94805195	10.8856753	8.28004495	5.36205899	3.33123078	2.05908083
x_k	1.6	1.73333333	1.86666667	2.0		
y_k	1.25357447	0.72289804	0.35817702	0.09780694		

求解代码如下:

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_4_exp5.py
"""
import numpy as np
import matplotlib.pyplot as plt
from numerical_integration_04.average_parabolic_interpolation_integral import
AverageParabolicInterpolationIntegral
from interpolation_02.b_spline_interpolation import BSplineInterpolation
from numerical_integration_04.cubic_bspline_interpolation_integration import
CubicBSplineInterpolationIntegration

x = np.linspace(0, 2, 16)
y = np.array([5.36205899, 8.28004495, 10.8856753, 10.94805195, 9.74022299, 9.74022299, 10.94805195,
              10.8856753, 8.28004495, 5.36205899, 3.33123078, 2.05908083, 1.25357447, 0.72289804,
              0.35817702, 0.09780694])

# 平均抛物插值
apii = AverageParabolicInterpolationIntegral(x, y)
apii.fit_int()
print("积分值: %.15f" % apii.int_value)

# 样条函数插值
cbisi = CubicBSplineInterpolationIntegration(x, y)
cbisi.fit_int()
```

```

print("积分值: %.15f" % cbsi.int_value)

# 可视化
bsi = BSplineInterpolation(x, y, boundary_cond="natural")
bsi.fit_interp()
plt.figure(figsize=(14, 5))
plt.subplot(121)
plt.plot(x, y, "o-", lw=1.5)
plt.xlabel("$x$", fontdict={"fontsize": 18})
plt.ylabel("$y$", fontdict={"fontsize": 18})
plt.title("原试验数据折线图", fontsize=18)
plt.tick_params(labelsize=16) # 刻度字体大小 16
plt.grid(ls=":")
plt.subplot(122)
bsi.plt_interpolation(is_show=False)
plt.legend(frameon=False, fontsize=16)
plt.xlabel("$x$", fontdict={"fontsize": 18})
plt.ylabel("$y$", fontdict={"fontsize": 18})
plt.title("三次均匀B样条插值曲线", fontsize=18)
plt.show()

```

平均抛物插值计算积分近似值 12.773129778611111, 三次均匀 B 样条插值计算积分值近似值 12.770976283069766. 三次均匀 B 样条插值曲线如图 4 - 4 所示.

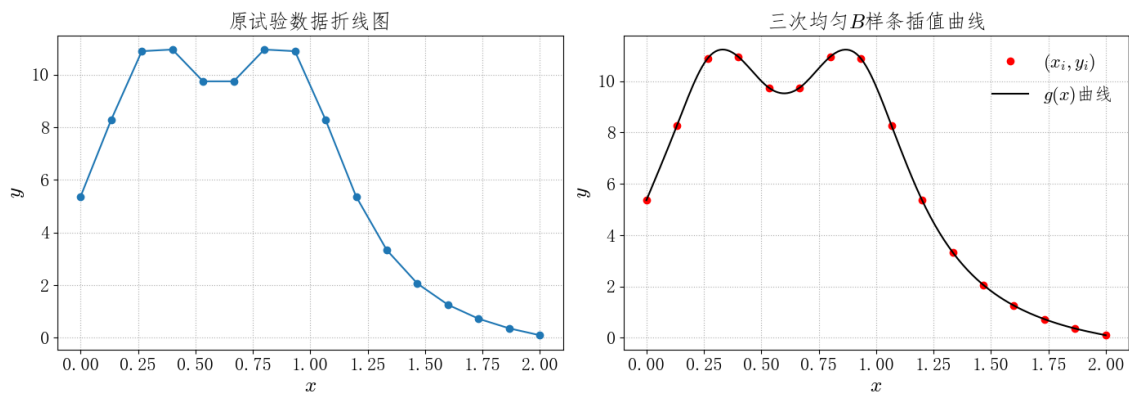


图 4 - 4 原采用数据折线与三次均匀 B 样条插值曲线

第 5 章 实验内容参考答案

1. 试用中点公式、三点公式、五点公式计算 $f(x) = \frac{\sin x}{(1+x)^2}$, $x \in [0, 5]$ 在微分点

[0.21, 1.23, 1.75, 2.89, 3.14, 3.78, 4.11, 4.56]

处的一阶数值微分近似值, 并分析微分误差, 采取适当的方法进行可视化.

求解代码如下:

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_5_exp1.py
"""

import numpy as np
import matplotlib.pyplot as plt
import sympy
from numerical_differentiation_05.middle_point_formula_differentiation \
    import MiddlePointFormulaDifferentiation
from numerical_differentiation_05.three_five_points_formula_differentiation \
    import ThreeFivePointsFormulaDifferentiation

def plt_differentiation(classname, fun, x0, points_type, diff_type):
    """
    可视化函数, param classname: 类名字符串
    """
    h_vector = [0.2, 0.001] # 不同微分步长
    plt.figure(figsize=(14, 5))
    for i, h in enumerate(h_vector):
        plt.subplot(121 + i)
        if classname == "MiddlePointFormulaDifferentiation":
            obj = eval(classname)(fun, h=h, is_error=False)
        else:
            obj = eval(classname)(fun, h=h, points_type=points_type, diff_type=diff_type)
        y0 = mpfd.fit_diff(x0)
        if h == 0.001:
            obj.plt_differentiation([0, 5], x0, y0, is_show=False, is_fh_marker=True)
        else:
            obj.plt_differentiation([0, 5], x0, y0, is_show=False)
    plt.show()

# 中点公式法
x = sympy.Symbol("x")
fun = sympy.sin(x) / (1 + x) ** 2 # 被微分函数
x0 = np.array([0.21, 1.23, 1.75, 2.89, 3.14, 3.78, 4.11, 4.56]) # 求解指定点的微分
mpfd = MiddlePointFormulaDifferentiation(fun, h=0.01, is_error=False) # 实例化
```

```

y0 = mpfd.fit_diff(x0) # 中点公式求解微分
diff_val = sympy.lambdify(x, fun.diff(x, 1))(x0) # 函数一阶导在x0的函数值, 用于误差分析
print("微分值: ", y0, "\n 精确值: ", diff_val, "\n 绝对值误差: ", abs(diff_val - y0))
plt_differentiation("MiddlePointFormulaDifferentiation", fun, x0, None, None)

# 三点公式法
tfpfd = ThreeFivePointsFormulaDifferentiation(fun, h=0.01, points_type="three", diff_type="stirling")
y0 = tfpfd.predict_diff_x0(x0)
print("微分值: ", y0, "\n 精确值: ", diff_val, "\n 绝对值误差: ", abs(diff_val - y0))
plt_differentiation("ThreeFivePointsFormulaDifferentiation", fun, x0, "three", "stirling")

# 五点公式法
tfpfd = ThreeFivePointsFormulaDifferentiation(fun, h=0.01, points_type="five", diff_type="middle")
y0 = tfpfd.predict_diff_x0(x0)
print("微分值: ", y0, "\n 精确值: ", diff_val, "\n 绝对值误差: ", abs(diff_val - y0))
plt_differentiation("ThreeFivePointsFormulaDifferentiation", fun, x0, "five", "middle")

```

(1) 中点公式法

设置微分步长 $h = 0.01$. 一阶数值微分近似值与绝对值误差如表 5 - 1 所示.

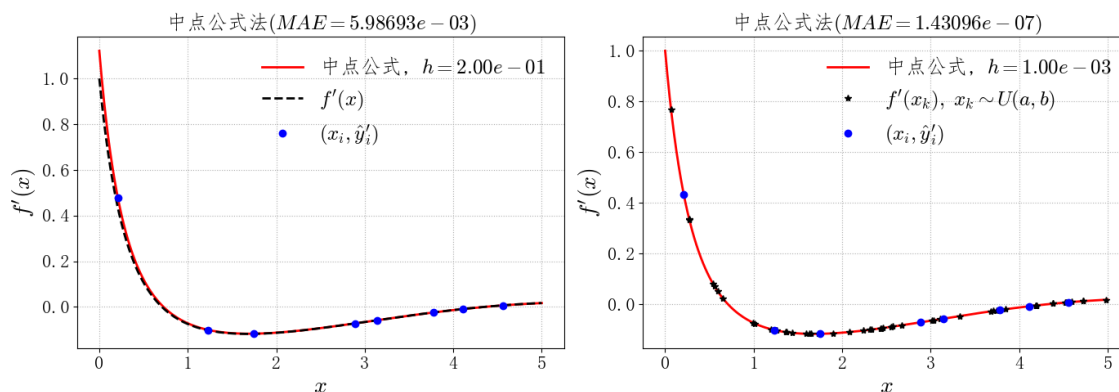


图 5 - 1 中点公式法在 $h = 0.2$ 和 $h = 0.001$ 时的一阶数值微分值图像

(2) 三点公式法

设置微分步长 $h = 0.01$. 由于三点公式与中点公式法计算的一阶数值微分近似值差异性不大, 故略去..

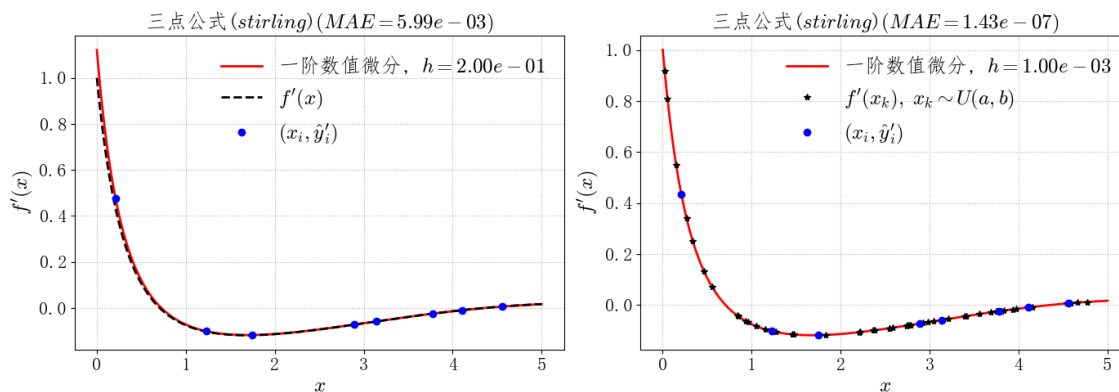


图 5 - 2 三点公式法在 $h = 0.2$ 和 $h = 0.001$ 时的一阶数值微分值图像

(3) 五点公式法

采用 middle 形式的五点公式求解，微分步长 $h = 0.01$ 。一阶数值微分近似值与绝对值误差如表 5 - 1 所示，在当前微分步长下，求解精度非常高，且高于中点公式法。

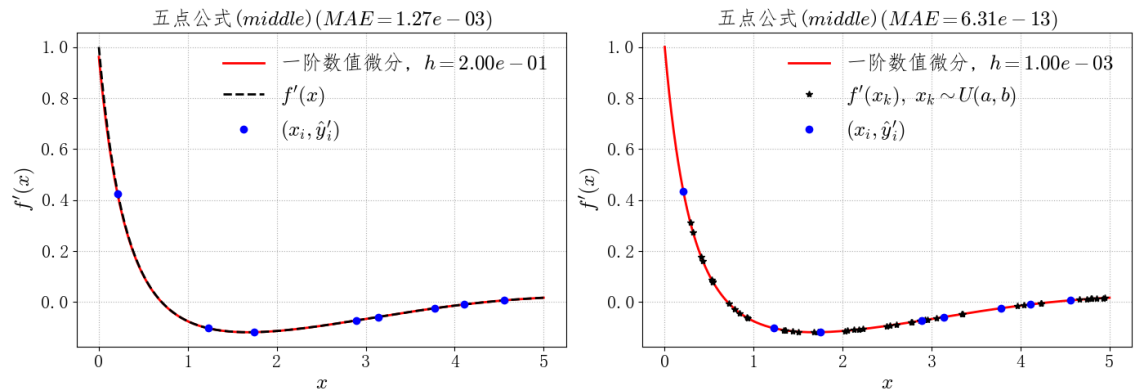


图 5 - 3 五点公式 (middle) 法在 $h = 0.2$ 和 $h = 0.001$ 时的一阶数值微分值图像

表 5 - 1 中点公式法与五点公式法求解一阶数值微分近似值与绝对值误差 ($k = 1, 2, \cdots, 8$)

x_k	精确微分值 $f'(x_k)$	中点公式法 $h = 0.01$		五点公式法 (middle) $h = 0.01$	
		一阶微分值	绝对值误差	一阶微分值	绝对值误差
0.21	0.43266792	0.43277329	1.05374585e-04	0.43266787	4.63083532e-08
1.23	-0.10276573	-0.10276113	4.59735158e-06	-0.10276573	5.54874646e-10
1.75	-0.11819798	-0.11819629	1.68670766e-06	-0.11819798	1.16308588e-10
2.89	-0.07246254	-0.07246243	1.08955996e-07	-0.07246254	1.11825965e-11
3.14	-0.05838923	-0.05838928	4.70914013e-08	-0.05838923	8.92171059e-12
3.78	-0.024234	-0.02423433	3.25809190e-07	-0.024234	7.17592305e-12
4.11	-0.00934869	-0.0093491	4.10576363e-07	-0.00934869	6.86094757e-12
4.56	0.00659075	0.00659028	4.66462359e-07	0.00659075	6.24351439e-12

2. 实验测得一组数据如表 5 - 2 所示，试采用离散数据形式的五点公式、三次样条插值和隐式格式求解各点的一阶微分值。

表 5 - 2 实验数据 ($k = 1, 2, \cdots, 11$)

x_k	0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0
$f(x_k)$	0.48	0.38	0.31	0.33	0.36	0.41	0.51	0.43	0.35	0.29	0.28

求解代码如下：

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_5_exp2.py
"""
import numpy as np
from numerical_differentiation_05.discrete_data_3_5_points_differentiation \
    import DiscreteData_3_5_PointsDifferentiation
from numerical_differentiation_05.discrete_data_cubic_spline_differetiation \
```

```
import DiscreteDataCubicSplineDifferential
from numerical_differentiation_05.implicit_numerical_diff import ImplicitNumericalDifferentiation

# 离散数据形式的五点公式
x = np.linspace(0, 1, 11)
y = np.array([0.48, 0.38, 0.31, 0.33, 0.36, 0.41, 0.51, 0.43, 0.35, 0.29, 0.28])
ddpd5 = DiscreteData_3_5_PointsDifferentiation(x, y, points_type="five")
diff_val = ddpd5.cal_diff()
print("微分值: ", diff_val)

# 三次样条插值, 修改边界条件, complete, natural
ddcsd = DiscreteDataCubicSplineDifferential(x, y, boundary_cond="complete")
diff_val = ddcsd.predict_diff_x0(x)
print("微分值: ", diff_val)

# 隐式格式
imp_nd = ImplicitNumericalDifferentiation(x, y)
imp_nd.fit_diff()
diff_val = imp_nd.predict_x0(x) # 近似值
print("微分值: ", diff_val)
```

求解结果如表 5 - 3 所示, 其中三次样条插值在第一边界条件下的微分结果与隐式格式一致.

表 5 - 3 不同的方法求解离散数据形式的一阶数值微分结果 ($k = 1, 2, \cdots, 11$)

x_k	$f(x_k)$	五点公式法	三次样条插值 (complete)	隐式格式
0	0.48	-0.6	-0.6	-0.6
0.1	0.38	-1.06666667	-1.07597792	-1.07597792
0.2	0.31	-0.23333333	-0.19608832	-0.19608832
0.3	0.33	0.30833333	0.36033119	0.36033119
0.4	0.36	0.36666667	0.25476354	0.25476354
0.5	0.41	0.91666667	1.02061464	1.02061464
0.6	0.51	0.14166667	0.1627779	0.1627779
0.7	0.43	-0.96666667	-1.07172622	-1.07172622
0.8	0.35	-0.74166667	-0.67587301	-0.67587301
0.9	0.29	-0.40833333	-0.42478175	-0.42478175
1.0	0.28	0.275	0.275	0.275

3. 已知函数 $f(x) = x^2 + \sqrt[3]{x} + \sin(x + \cos^2 x)$, $x \in [1, 5]$, 待求微分点 x_k , $k = 1, 2, \cdots, 10$ 随机生成, 其中 $x_k \sim U(1, 5)$, 随机种子 100.

- (1) 采用理查森外推算法求解一阶数值微分 x_k , 外推次数分别为 3 和 9, 分析误差, 并适当可视化.
- (2) 采用多点公式法和三次均匀 B 样条函数法求解在微分点 x_k 的二阶微分值, 分析误差, 并适当可视化.

求解代码如下:

```
# -*- coding: UTF-8 -*-
```

```

"""
@file_name: chapter_5_exp3.py
"""

import numpy as np
import sympy
import matplotlib.pyplot as plt
from numerical_differentiation_05.richardson_extrapolation_differentiation \
    import RichardsonExtrapolationDifferentiation
from numerical_differentiation_05.five_points_second_order_differentiation \
    import FivePointsSecondOrderDifferentiation
from numerical_differentiation_05.cubic_bspline_2_order_differentiation import
CubicBSpline2OrderDifferentiation

fun = lambda x: x ** 2 + x ** (1/3) + np.sin(x + np.cos(x) ** 2) # 微分函数
x = sympy.symbols("x")
fun_sym = x ** 2 + x ** (1/3) + sympy.sin(x + sympy.cos(x) ** 2) # 微分函数, 符号定义
dfun = sympy.lambdify(x, fun_sym.diff()) # 一阶导函数

np.random.seed(100)
x0 = 1 + 4 * np.random.rand(10) # 随机生成, 均匀分布
x0 = np.asarray(sorted(x0)) # 排序
y_true = dfun(x0) # 一阶微分真值
print("真值 x: ", x0)
print("真值 dy: ", y_true)

# (1) 理查森外推算法
plt.figure(figsize=(14, 5))
for i, step in enumerate([3, 9]):
    plt.subplot(121 + i)
    red = RichardsonExtrapolationDifferentiation(fun, step=step)
    diff_value = red.predict_diff_x0(x0)
    print("近似微分值: ", diff_value)
    print("绝对值误差: ", np.abs(y_true - diff_value))
    red.plt_differentiation([1, 5], dfun, x0, diff_value, is_show=False, is_fh_marker=True)
plt.show()
print("=" * 80)

# (2) 多点公式法二阶数值微分
d2fun = sympy.lambdify(x, sympy.diff(fun_sym, x, 2)) # 二阶导函数
fpsod = FivePointsSecondOrderDifferentiation(diff_fun=fun, h=0.01)
diff_val = fpsod.fit_2d_diff(x0)
print("二阶近似微分: ", diff_val)
y_true = d2fun(x0) # 二阶导数真值
print("精确值: ", y_true, "\n 绝对值误差: ", np.abs(y_true - diff_val))
print("=" * 80)

```

```
# (2) 三次均匀 B 样条函数法二阶数值微分
cbsod = CubicBSpline2OrderDifferentiation(fun, n=20, h=0.01)
diff_val = cbsod.predict_diff_x0(x0)
print("二阶近似微分: ", diff_val)
print("绝对值误差: ", np.abs(y_true - diff_val))
print("=" * 80)
cbsod.plt_2_order_different([1, 5], d2fun, is_show=True, is_fh_marker=True)
plt.show()
```

(1) 理查森外推算法

当外推次数为 3 时，最大绝对值误差 2.4111616249e-04，平均绝对值误差 7.1131146394e-05；当外推次数为 9 时，最大绝对值误差 3.9194869572e-11，平均绝对值误差 1.4307013352e-11。

表 5-4 理查森外推算法计算一阶数值微分近似值以及绝对值误差 ($k=1, 2, \dots, 10$)

x_k	一阶微分真值 $f'(x_k)$	外推次数 3		外推次数 9	
		一阶微分值	绝对值误差	一阶微分值	绝对值误差
1.01887542	2.39623199	2.39626607	3.40786973e-05	2.39623199	5.72075720e-12
1.48627648	3.29280607	3.2928387	3.26307279e-05	3.29280607	1.37241329e-11
1.54682636	3.3651459	3.36515834	1.24334491e-05	3.3651459	1.96052063e-11
2.11347754	3.06525642	3.06522021	3.62046939e-05	3.06525642	8.62643290e-12
2.69807036	3.91455888	3.9145207	3.81730156e-05	3.91455888	1.34643408e-11
3.17361977	6.02052929	6.02049825	3.10427932e-05	6.02052929	1.98046024e-11
3.30037332	6.46003834	6.46005992	2.15807456e-05	6.46003834	1.24922295e-11
3.68299634	7.47182956	7.47186678	3.72221159e-05	7.47182956	3.14415161e-12
4.30341102	8.66573265	8.66569204	4.06108152e-05	8.66573265	7.81064102e-12
4.37910453	8.79711383	8.79706797	4.58604990e-05	8.79711383	5.24913446e-12

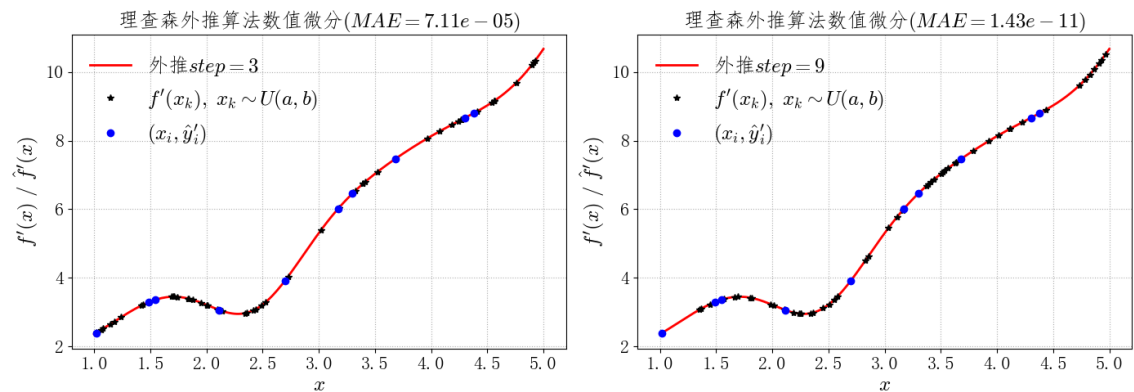


图 5-4 理查森外推算法求解一阶数值微分示意图

(2) 二阶数值微分

微分步长统一设置为 $h=0.01$ ，三次均匀 B 样条插值法左右延拓结点数 $n=20$ 。如所示，多点公式法求解的精度非常高。想要提高三次均匀 B 样条函数法的求解精度，需要延拓的节点数更多，但也意味着求解复杂性的提高，以及舍入误差的影响。

表 5 - 5 不同方法求解二阶数值微分近似值以及绝对值误差 ($k = 1, 2, \cdots, 10$)

x_k	二阶微分真值 $f''(x_k)$	多点公式法 (五点), $h = 0.01$		三次均匀 B 样条函数法	
		二阶微分值	绝对值误差	二阶微分值	绝对值误差
1.01887542	2.01979367	2.01979368	9.58966151e-09	2.0198411	4.74268812e-05
1.48627648	1.34786379	1.34786375	4.03828968e-08	1.34798708	1.23281061e-04
1.54682636	1.03311606	1.03311602	4.53376057e-08	1.03321986	1.03794854e-04
2.11347754	-1.18949573	-1.18949562	1.07352707e-07	-1.18984925	3.53526256e-04
2.69807036	4.28077401	4.28077391	9.78287993e-08	4.28108027	3.06261235e-04
3.17361977	3.74489877	3.74489881	3.78951945e-08	3.74488995	8.81626961e-06
3.30037332	3.20232224	3.20232228	3.59586250e-08	3.20226079	6.14536563e-05
3.68299634	2.26033364	2.26033363	5.95270766e-09	2.26030713	2.65019936e-05
4.30341102	1.7118181	1.71181811	3.62889097e-09	1.71173263	8.54764534e-05
4.37910453	1.77040023	1.77040024	1.13648824e-08	1.77029292	1.07304723e-04

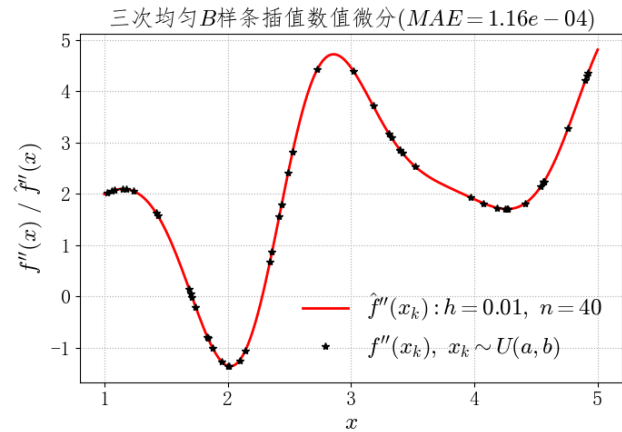


图 5 - 5 三次均匀 B 样条插值求解二阶数值微分示意图

第 6 章 实验内容参考答案

1. 使用各种高斯消元法求解线性方程组:

$$\begin{pmatrix} 12 & -3 & 3 & 4 \\ -18 & 3 & -1 & -1 \\ 1 & 1 & 1 & 1 \\ 3 & 1 & -1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 15 \\ -15 \\ 6 \\ 2 \end{pmatrix}.$$

给出中间计算过程, 对解进行验证.

求解代码如下:

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_6_exp1.py
"""
import numpy as np
from direct_solution_linear_equations_06.gaussian_elimination_algorithm import GaussianEliminationAlgorithm

A = np.array([[12, -3, 3, 4], [-18, 3, -1, -1], [1, 1, 1, 1], [3, 1, -1, 1]])
b = np.array([15, -15, 6, 2])
sol_method = ["sequential", "column", "complete", "jordan"]
for method in sol_method:
    gea = GaussianEliminationAlgorithm(A, b, sol_method=method)
    gea.fit_solve()
    print(method + ", 消元后的矩阵: \n", gea.augmented_matrix)
    print(method + ", 线性方程组的解: \n", gea.x)
    print(method + ", 线性方程组的解验证误差: \n", gea.eps)
    if method == "jordan":
        print("Jordan 消元法, 系数矩阵的逆矩阵是: \n", gea.jordan_inverse_matrix)
        print("Jordan 消元法, 逆矩阵验证: \n", np.dot(A, gea.jordan_inverse_matrix))
    print("=" * 80)
```

(1) 高斯顺序消去法, 消元后的增广矩阵 B (保留小数点后 2 位, 其中 $-8.88178420\text{e-}16$ 表示为 -10^{-16}):

$$B = \left(\begin{array}{cccc|c} 12.00 & -3.00 & 3.00 & 4.00 & 15.0 \\ & -1.50 & 3.50 & 5.00 & 7.50 \\ & & 3.67 & 4.83 & 11.00 \\ & & & 2.76 & -10^{-16} \end{array} \right).$$

线性方程组的解及解的验证误差: $\mathbf{x}^* = [1.0, 2.0, 3.0, -10^{-16}]^T$, $\boldsymbol{\varepsilon} = [0.0, 0.0, 0.0, 10^{-16}]^T$.

(2) 列主元高斯消去法, 依次选择的列主元分别为 (Python 输出格式):

$$[-18.0, 1.5, 1.8518518518518519].$$

消元后的增广矩阵 B (保留小数点后 2 位, 其中 $-8.88178420\text{e-}16$ 表示为 -10^{-16}):

$$B = \left(\begin{array}{cccc|c} -18.00 & 3.00 & -1.00 & -1.00 & -15.00 \\ & 1.50 & -1.67 & -0.83 & -0.50 \\ & & 1.85 & 0.30 & 5.56 \\ & & & 3.64 & -10^{-16} \end{array} \right).$$

线性方程组的解及解的验证误差: $\mathbf{x}^* = [1.0, 2.0, 2.0, -10^{-16}]^T$, $\boldsymbol{\varepsilon} = [10^{-15}, 10^{-15}, 0.0, 10^{-16}]^T$.

(3) 全主元高斯消去法, 依次选取的全主元以及行列索引分别为:

$$[(-18.00, [1, 0]), (3.33, [1, 3]), (-1.75, [3, 2])].$$

消元后的增广矩阵 $\mathbf{B}$ (注意是交换行列后的):

$$\mathbf{B} = \left(\begin{array}{cccc|c} -18.00 & -1.00 & -1.00 & 3.00 & -15.00 \\ & 3.33 & 2.33 & -1.00 & 5.00 \\ & & -1.75 & 1.75 & -1.75 \\ & & & 1.73 & 3.47 \end{array} \right).$$

由增广矩阵得到的解为 $\hat{\mathbf{x}}^* = [1.0, 10^{-15}, 3.0, 2.0]^T$, 需要按照列交换的索引变换为原方程组的解, 故线性方程组的解及解的验证误差: $\mathbf{x}^* = [1.0, 2.0, 2.0, -10^{-16}]^T$, $\boldsymbol{\varepsilon} = [10^{-15}, 10^{-15}, 0.0, 10^{-16}]^T$.

(4) 高斯—约当消去法结合列主元高斯消去法, 消元后的增广矩阵 $\mathbf{B}$:

$$\mathbf{B} = \left(\begin{array}{cccc|c} 1.0 & 0.0 & 0.0 & 0.0 & 1.0 \\ 0.0 & 1.0 & 0.0 & 0.0 & 2.0 \\ 0.0 & 0.0 & 1.0 & 0.0 & 3.0 \\ 0.0 & 0.0 & 0.0 & 1.0 & -10^{-16} \end{array} \right).$$

线性方程组的解及解的验证误差: $\mathbf{x}^* = [1.0, 2.0, 2.0, -10^{-16}]^T$, $\boldsymbol{\varepsilon} = [10^{-15}, 10^{-15}, 0.0, 10^{-16}]^T$.

高斯—约当消元法求解得到系数矩阵 $\mathbf{A}$ 的逆矩阵 (保留小数点后 4 位):

$$\mathbf{A}^{-1} = \left(\begin{array}{cccc} -0.0440 & -0.0769 & 0.0769 & 0.0220 \\ -0.1868 & -0.0769 & 0.5769 & 0.0934 \\ -0.0440 & -0.0770 & 0.5769 & -4.7802 \\ 0.2747 & 0.2308 & -0.2308 & 0.3626 \end{array} \right).$$

对逆矩阵进行验证 $\mathbf{A}\mathbf{A}^{-1} = \mathbf{I}$, 可知精度较高 (舍入误差的影响, 非对角线元素接近于 0, 保留原 Python 输出格式):

$$\left(\begin{array}{cccc} 1.00000000e+00 & -1.11022302e-16 & 0.00000000e+00 & 0.00000000e+00 \\ -5.55111512e-17 & 1.00000000e+00 & 1.66533454e-16 & -2.22044605e-16 \\ 5.55111512e-17 & 2.77555756e-17 & 1.00000000e+00 & 0.00000000e+00 \\ -5.55111512e-17 & 0.00000000e+00 & -5.55111512e-17 & 1.00000000e+00 \end{array} \right).$$

2. 使用矩阵三角分解法和平方根法求解线性方程组:

$$\begin{pmatrix} 4 & 1 & -1 & 0 \\ 1 & 3 & -1 & 0 \\ -1 & -1 & 5 & 2 \\ 0 & 0 & 2 & 4 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 7 \\ 8 \\ -4 \\ 6 \end{pmatrix}.$$

对解进行验证.

求解代码如下:

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_6_exp2.py
"""
import numpy as np
from direct_solution_linear_equations_06.square_root_decomposition import SquareRootDecompositionAlgorithm
from direct_solution_linear_equations_06.doolittle_decomposition_lu import DoolittleTriangularDecompositionLU
```

```

A = np.array([[4, 1, -1, 0], [1, 3, -1, 0], [-1, -1, 5, 2], [0, 0, 2, 4]])
b = np.array([7, 8, -4, 6])

# 矩阵三角分解法
dtd_lu = DoolittleTriangularDecompositionLU(A, b, sol_method="pivot")
dtd_lu.fit_solve()
print("下三角矩阵 L: \n", dtd_lu.L)
print("上三角矩阵 U: \n", dtd_lu.U)
print("置换矩阵 P: \n", dtd_lu.P)
print("系数矩阵的逆矩阵: \n", dtd_lu.inverse_matrix)
print("逆矩阵验证: ", np.dot(A, dtd_lu.inverse_matrix))
print("线性方程组的解: ", dtd_lu.x)
print("解的精度验证: ", dtd_lu.eps)
print("=" * 80)

# 平方根法
srda = SquareRootDecompositionAlgorithm(A, b, sol_method="cholesky")
srda.fit_solve()
print("下三角矩阵 L 为: \n", srda.L)
print("线性方程组的解 x 为: \n", srda.x)
print("解的精度验证: \n", srda.eps)
print("=" * 60)
srda = SquareRootDecompositionAlgorithm(A, b, sol_method="improved")
srda.fit_solve()
print("下三角矩阵 L 为: \n", srda.L)
print("对角矩阵 D 为: \n", srda.D)
print("线性方程组的解 x 为: \n", srda.x)
print("解的精度验证: \n", srda.eps)

```

(1) 矩阵三角分解法

采用选主元三角分解法, 得

$$L = \begin{pmatrix} 1.00 & & & \\ 0.25 & 1.00 & & \\ -0.25 & -0.27 & 1.00 & \\ 0.00 & 0.00 & 0.44 & 1.00 \end{pmatrix}, U = \begin{pmatrix} 4.00 & 1.00 & -1.00 & 0.00 \\ & 2.75 & -0.75 & 0.00 \\ & & 4.55 & 2.00 \\ & & & 3.12 \end{pmatrix}, P = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

线性方程组的解及解的验证精度: $\mathbf{x}^* = [1.0, 2.0, -1.0, 2.0]^T$, $\boldsymbol{\varepsilon} = [0.0, 0.0, 0.0, 0.0]^T$. 矩阵 $\mathbf{A}$ 的逆矩阵 $\mathbf{A}^{-1}$ 以及对逆矩阵的验证, 略去, 可通过计算代码打印输出, 然后构造矩阵.

(2) 平方根法

1) 平方根法求解 $\mathbf{A} = \mathbf{L}\mathbf{L}^T$, 结果为:

$$L = \begin{pmatrix} 2.000 & & & \\ 0.500 & 1.658 & & \\ -0.500 & -0.452 & 2.132 & \\ 0.000 & 0.000 & 0.938 & 1.766 \end{pmatrix}.$$

线性方程组的解及解的验证精度: $\mathbf{x}^* = [1.0, 2.0, -1.0, 2.0]^T$, $\boldsymbol{\varepsilon} = [0.0, 0.0, 10^{-16}, 0.0]^T$.

2) 改进的平方根法求解 $\mathbf{A} = \mathbf{LDL}^T$, 结果为:

$$\mathbf{L} = \begin{pmatrix} 1.000 & & & \\ 0.250 & 1.000 & & \\ -0.250 & -0.273 & 1.000 & \\ 0.000 & 0.000 & 0.440 & 1.000 \end{pmatrix}, \mathbf{D} = \begin{pmatrix} 4.000 & & & \\ & 2.750 & & \\ & & 4.545 & \\ & & & 3.120 \end{pmatrix}.$$

线性方程组的解及解的验证精度: $\mathbf{x}^* = [1.0, 2.0, -1.0, 2.0]^T$, $\boldsymbol{\epsilon} = [0.0, 0.0, 0.0, 0.0]^T$.

3. 用追赶法求解线性方程组:

$$\begin{pmatrix} 2 & 1 & & \\ 1 & 2 & -3 & \\ & 3 & -7 & 4 \\ & & 2 & 5 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 3 \\ -3 \\ -10 \\ 2 \end{pmatrix}.$$

给出消元后或分解的矩阵, 对解进行验证.

求解代码如下:

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_6_exp3.py
"""

import numpy as np
from direct_solution_linear_equations_06.chasing_method_tridiagonal_matrix \
    import ChasingMethodTridiagonalMatrix

d = np.array([3, -3, -10, 2]) # 右端向量
a = np.array([1, 3, 2]) # 次对角线元素, 对角线以下
b = np.array([2, 2, -7, 5]) # 主对角线元素
c = np.array([1, -3, 4]) # 次对角线元素, 对角线以上

sol_method = ["gauss", "doolittle", "crout"]
for method in sol_method:
    cmtm = ChasingMethodTridiagonalMatrix(diag_a=a, diag_b=b, diag_c=c, d_vector=d, sol_method=method)
    cmtm.fit_solve()
    print("y: ", cmtm.y)
    print("x: ", cmtm.x)
    print("eps: ", cmtm.eps)
    print("=" * 60)
```

(1) 三对角高斯消元法

消元后的增广矩阵为

$$\mathbf{B} = \left(\begin{array}{cccc|c} 2.0 & 1.0 & & & 3.0 \\ & 1.5 & -3.0 & & -4.5 \\ & & -1.0 & 4.0 & -1.0 \\ & & & 13.0 & 0.0 \end{array} \right).$$

线性方程组的解及解的验证精度: $\mathbf{x}^* = [2.0, -1.0, 1.0, 0.0]^T$, $\boldsymbol{\epsilon} = [0.0, 0.0, 0.0, 0.0]^T$.

(2) 三对角矩阵的 Doolittle 分解

$$\mathbf{L} = \begin{pmatrix} 1.0 & & & \\ 0.5 & 1.0 & & \\ & 2.0 & 1.0 & \\ & & -2.0 & 1.0 \end{pmatrix}, \mathbf{U} = \begin{pmatrix} 2.0 & 1.0 & & \\ & 1.5 & -3.0 & \\ & & -1.0 & 4.0 \\ & & & 13.0 \end{pmatrix}.$$

线性方程组的解及解的验证精度：回代过程，令 $\mathbf{Ly} = \mathbf{b}$ 得 $\mathbf{y}^* = [3.0, -4.5, -1.0, 0.0]^T$ 令 $\mathbf{Ux} = \mathbf{y}$ 得 $\mathbf{x}^* = [2.0, -1.0, 1.0, 0.0]^T$, $\boldsymbol{\varepsilon} = [0.0, 0.0, 0.0, 0.0]^T$.

(3) 三对角矩阵的 Crout 分解

$$\mathbf{L} = \begin{pmatrix} 2.0 & & & \\ 1.0 & 1.5 & & \\ & 3.0 & -1.0 & \\ & & 2.0 & 13.0 \end{pmatrix}, \mathbf{U} = \begin{pmatrix} 1.0 & 0.5 & & \\ & 1.0 & -2.0 & \\ & & 1.0 & -4.0 \\ & & & 1.0 \end{pmatrix}.$$

线性方程组的解及解的验证精度：回代过程，令 $\mathbf{Ly} = \mathbf{b}$ 得 $\mathbf{y}^* = [1.5, -3.0, 1.0, 0.0]^T$ 令 $\mathbf{Ux} = \mathbf{y}$ 得 $\mathbf{x}^* = [2.0, -1.0, 1.0, 0.0]^T$, $\boldsymbol{\varepsilon} = [0.0, 0.0, 0.0, 0.0]^T$.

4. 用 QR 分解法求解线性方程组：

$$\begin{pmatrix} 1 & 3 & 5 & -4 & 2 \\ 1 & 3 & 2 & -2 & 1 \\ 1 & -2 & 1 & -1 & -1 \\ 1 & -4 & 1 & 1 & -1 \\ 1 & 2 & 1 & -1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{pmatrix} = \begin{pmatrix} 3 \\ -1 \\ 3 \\ 3 \\ -1 \end{pmatrix}.$$

给出正交矩阵 $\mathbf{Q}$ 和上三角矩阵 $\mathbf{R}$ ，对解进行验证。

求解代码如下：

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_6_exp4.py
"""
import numpy as np
from direct_solution_linear_equations_06.qr_orthogonal_decomposition import QROrthogonalDecomposition

A = np.array([[1, 3, 5, -4, 2], [1, 3, 2, -2, 1], [1, -2, 1, -1, -1], [1, -4, 1, 1, -1], [1, 2, 1, -1, 1]])
print(np.linalg.matrix_rank(A))
b = np.array([3, -1, 3, 3, -1])

sol_method = ["schmidt", "householder", "givens"]
for method in sol_method:
    qr = QROrthogonalDecomposition(A, b, sol_method=method)
    qr.fit_solve()
    print("正交化矩阵 Q: \n", qr.Q)
    print("上三角矩阵 R: \n", qr.R)
    print("线性方程组的解: \n", qr.x)
    print("验证解的精度: \n", qr.eps)
    print("=" * 60)
```

结果采用 Python 输出格式 (结果保留小数点后 8 位)。三种方法的正交化矩阵基本一致，由于计算舍入

误差的存在, 个别元素存在极小的差异, 如第 3 行第 5 列的元素, 近似为 0, 如 Householder 变换方法的计算结果为 $1.85152643 \times 10^{-16}$. 如下为 Schmidt 正交化法的正交矩阵结果:

$$\mathbf{Q} = \begin{pmatrix} 0.447213595 & 0.405065019 & 0.775566344 & -0.022822605 & 0.184114924 \\ 0.447213595 & 0.405065019 & -0.291916953 & 0.091290421 & -0.736459694 \\ 0.447213595 & -0.373906172 & -0.086365963 & -0.807920228 & 3.01502725 \times 10^{-16} \\ 0.447213595 & -0.685494648 & 0.138185540 & 0.550024788 & -0.092057462 \\ 0.447213595 & 0.249270781 & -0.535468968 & 0.189427624 & 0.644402233 \end{pmatrix}.$$

略去三种正交化方法的上三角矩阵 $\mathbf{R}$, 可通过计算代码打印输出, 然后构造矩阵 $\mathbf{R}$.

(1) Schmidt 正交化法

线性方程组的解及解的验证精度:

$$\mathbf{x}^* = [-0.50, -1.50, -10^{-15}, -1.50, 1.00]^T, \quad \boldsymbol{\varepsilon} = [10^{-15}, 10^{-15}, 0.0, 10^{-16}, 10^{-15}]^T.$$

(2) Householder 变换方法

线性方程组的解及解的验证精度:

$$\mathbf{x}^* = [-0.50, -1.50, 10^{-15}, -1.50, 1.00]^T, \quad \boldsymbol{\varepsilon} = [10^{-15}, 10^{-15}, 0.0, 0.0, 10^{-16}]^T.$$

(3) Givens 变换方法

线性方程组的解及解的验证精度:

$$\mathbf{x}^* = [-0.50, -1.50, 10^{-16}, -1.50, 1.00]^T, \quad \boldsymbol{\varepsilon} = [0.0, 10^{-16}, 10^{-15}, 10^{-16}, 0.0]^T.$$

第 7 章 实验内容参考答案

1. 利用雅可比迭代法、高斯—赛德尔迭代法和超松弛迭代法求解如下线性方程组的解，初始解向量为 $\mathbf{x}^{(0)} = (0, 0, 0, 0, 0)^T$ ，精度要求 $\varepsilon = 10^{-15}$ 。并分析不同的松弛因子 ω 对超松弛迭代法收敛速度的影响。

$$\begin{pmatrix} 28 & -3 & 0 & 0 & 0 \\ -3 & 38 & -10 & 0 & -5 \\ -10 & 0 & 25 & -15 & 0 \\ 0 & 0 & -15 & 45 & 0 \\ 0 & -5 & 0 & 0 & 30 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{pmatrix} = \begin{pmatrix} 6 \\ 5 \\ 1 \\ 4 \\ 2 \end{pmatrix}.$$

求解代码如下：

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_7_exp1.py
"""

import numpy as np
from iterative_solution_linear_equation_07.jacobi_gauss_seidel_iterative import JacobiGSIterativeMethod
from iterative_solution_linear_equation_07.SOR_iterative import SORIteration
from Experiment.util_font import * # 导入字体文件

A = np.array([[28, -3, 0, 0, 0], [-3, 38, -10, 0, -5], [-10, 0, 25, -15, 0],
              [0, 0, -15, 45, 0], [0, -5, 0, 0, 30]]) # 系数矩阵
b = np.array([6, 5, 1, 4, 2]) # 右端向量
x0 = np.array([0, 0, 0, 0, 0]) # 初始解向量

jacobi = JacobiGSIterativeMethod(A, b, x0, eps=1e-15, method="Jacobi", is_out_info=True)
jacobi.fit_solve() # 雅可比迭代求解
print("=" * 80)
guass_seidel = JacobiGSIterativeMethod(A, b, x0, eps=1e-15, method="G-S", is_out_info=True)
guass_seidel.fit_solve() # G-S 迭代求解
plt.figure(figsize=(14, 5))
plt.subplot(121)
jacobi.plt_convergence(is_show=False) # 可视化雅可比迭代收敛性
plt.subplot(122)
guass_seidel.plt_convergence(is_show=False) # 可视化 G-S 迭代收敛性
plt.show()

plt.figure(figsize=(7, 5))
iter_num_jacobi = jacobi.iterative_info["Iteration_number"] # 获取雅可比迭代次数
iter_num_jacobi = np.linspace(1, iter_num_jacobi, iter_num_jacobi)
iter_num_gs = guass_seidel.iterative_info["Iteration_number"] # 获取 G-S 迭代次数
iter_num_gs = np.linspace(1, iter_num_gs, iter_num_gs)
plt.semilogy(iter_num_jacobi, jacobi.precision, "o--", lw=1.5,
              label=r"$Jacobi: \backslash \epsilon = \%3e, \backslash n_{\{iter\}} = \%d$" % (jacobi.precision[-1], iter_num_jacobi[-1]))
plt.semilogy(iter_num_gs, guass_seidel.precision, "-*", lw=1.5,
```

```

        label="$GS: \epsilon=%.3e, n_{iter}=%d$" % (guass_seidel.precision[-1], iter_num_gs[-1])) #
# 绘制 G-S
plt.xlabel("$Iterations$", fontdict={"fontsize": 18})
plt.ylabel("$Precision(\epsilon)$", fontdict={"fontsize": 18})
plt.title("$Jacobi$和$Gauss-Seidel$的$\epsilon=\text{Vert } b - Ax^* \text{Vert }_2$收敛曲线", fontdict={"fontsize": 18})
plt.legend(frameon=False, fontsize=16)
plt.tick_params(labelsize=16) # 刻度字体大小 16
plt.grid(ls=":")
plt.show()

# 超松弛迭代法
omega_vector = np.linspace(0.1, 2, 19, endpoint=False)
object_, iter_num = [], [] # 存储对象, 迭代次数
for omega in omega_vector:
    sor = SORIteration(A, b, x0, eps=1e-15, omega=omega, is_out_info=False)
    sor.fit_solve() # SOR 迭代求解
    object_.append(sor)
    iter_num.append(sor.iterative_info["Iteration_number"])

plt.figure(figsize=(14, 5))
plt.subplot(121)
idx = int(np.argmin(iter_num))
plt.plot(omega_vector, iter_num, "o--", lw=2)
plt.plot(omega_vector[idx], iter_num[idx], "D",
        label="$\omega=%.2f, k=%d$" % (omega_vector[idx], iter_num[idx]))
plt.xlabel(r"$\omega$ in [0.1, 1.9], \step=0.1$", fontdict={"fontsize": 18})
plt.ylabel("$Iterations(k)$", fontdict={"fontsize": 18})
plt.title("不同松弛因子的$SOR$迭代法所需迭代次数", fontdict={"fontsize": 18})
plt.tick_params(labelsize=16) # 刻度字体大小 16
plt.legend(frameon=False, fontsize=18)
plt.grid(ls=":")
plt.subplot(122)
idx = int(np.argmin(iter_num))
object_[idx].plt_convergence(is_show=False)
plt.title("$SOR$的$\epsilon=\text{Vert } b - Ax^* \text{Vert }_2$收敛曲线: $\omega=%.2f$"
        % omega_vector[idx], fontdict={"fontsize": 18})
plt.show()
print("=" * 80)
# 最佳超松弛因子下的迭代信息
for key in object_[idx].iterative_info.keys():
    print(key, ": ", object_[idx].iterative_info[key])

```

计算代码迭代输出信息如下 (按 Python 输出格式):

(1) 雅可比迭代法

Success_Info: Jacobi 迭代, 优化终止, 收敛到近似解

Convergence: Spectral radius 0.47677

Iteration_number: 49
Solution_X: [0.23852459 0.22622951 0.23592896 0.16753188 0.10437158]
Precision: 8.005932084973442e-16

(2) 高斯—塞德尔迭代法

Success_Info: G-S 迭代, 优化终止, 收敛到近似解
Convergence: Spectral radius 0.25110
Iteration_number: 27
Solution_X: [0.23852459 0.22622951 0.23592896 0.16753188 0.10437158]
Precision: 4.440892098500626e-16

(3) 超松弛迭代法

Success_Info: SOR 迭代, 优化终止, 收敛到近似解
Omega: Omega 1.10000
Convergence: Spectral radius 0.15240
Iteration_number: 19
Solution_X: [0.23852459 0.22622951 0.23592896 0.16753188 0.10437158]
Precision: 4.440892098500626e-16

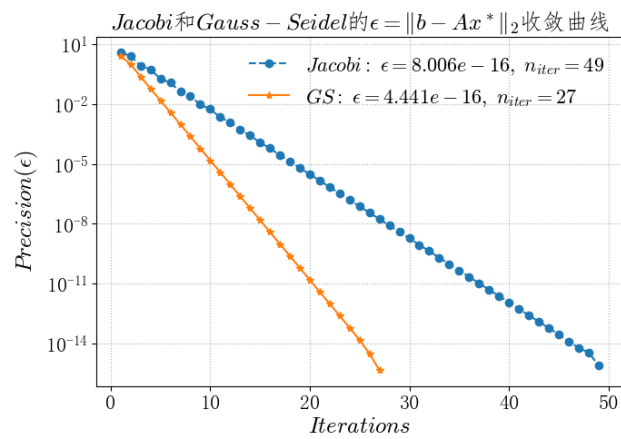


图 7 - 1 雅可比与高斯—赛德尔迭代法精度收敛曲线

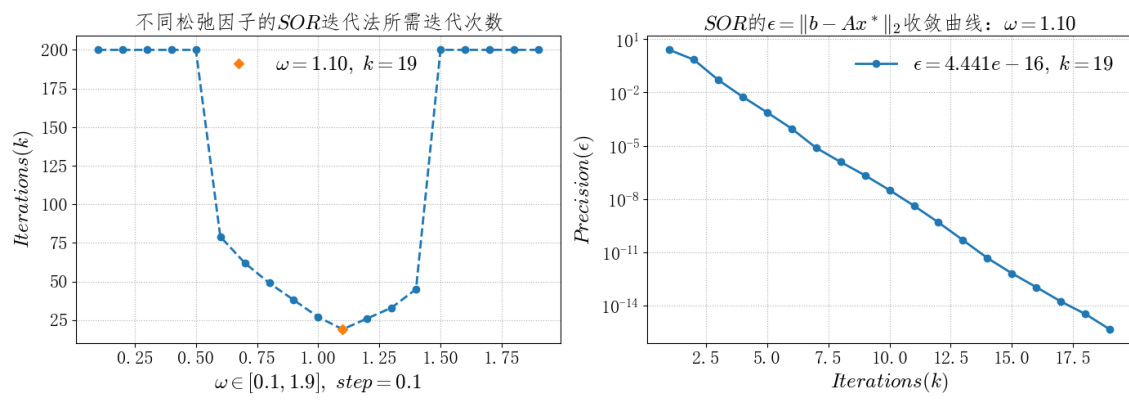


图 7 - 2 SOR 迭代法松弛因子的选择以及在最佳松弛因子下的解精度收敛曲线

可见三种方法在保留小数点后 8 位的数值解一致. 但谱半径、最终解的验证精度和迭代效率不同. 其中超松弛迭代法在选择的最佳松弛因子 $\omega = 1.1$ 下迭代 19 次, 如图 7-2 所示. 高斯-塞德尔迭代法比雅可比迭代法收敛速度更快, 且谱半径 0.25110 小于雅可比迭代法的谱半径 0.47677.

2. 采用三种不同的块迭代法求解如下线性方程组的解, 块向量分别设置为 $[2, 2, 1]$ 和 $[4, 1]$, 初始解向量为 $\mathbf{x}^{(0)} = (0, 0, 0, 0, 0)^T$, 精度要求 $\varepsilon = 10^{-15}$, 松弛因子 $\omega = 1.03$. 并分析不同块迭代法在不同块向量下的迭代效率和解向量的精度.

$$\begin{pmatrix} 2.6934 & 0.6901 & 0.3997 & 0.6010 & 0.4390 \\ 0.6901 & 2.8784 & 0.8799 & 0.5978 & 0.4514 \\ 0.3997 & 0.8799 & 3.3216 & 0.4673 & 0.8282 \\ 0.6010 & 0.5978 & 0.4673 & 2.8412 & 0.5511 \\ 0.4390 & 0.4514 & 0.8282 & 0.5511 & 2.8704 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}.$$

求解代码如下:

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_7_exp2.py
"""
import numpy as np
from iterative_solution_linear_equation_07.block_iterative_method import BlockIterative
from Experiment.util_font import * # 导入字体文件

A = np.array([[2.6934, 0.6901, 0.3997, 0.6010, 0.4390], [0.6901, 2.8784, 0.8799, 0.5978, 0.4514],
              [0.3997, 0.8799, 3.3216, 0.4673, 0.8282], [0.6010, 0.5978, 0.4673, 2.8412, 0.5511],
              [0.4390, 0.4514, 0.8282, 0.5511, 2.8704]]) # 系数矩阵
b, x0 = np.array([1, 1, 1, 1, 1]), np.array([0, 0, 0, 0, 0]) # 右端向量, 初始解向量

block_vectors = [np.array([4, 1]), np.array([2, 2, 1])] # 块迭代向量
plt.figure(figsize=(14, 5))
for i, block_vector in enumerate(block_vectors):
    # 块 Jacobi 迭代法
    b_jacobi = BlockIterative(A, b, x0, block_vector, eps=1e-15, is_out_info=True)
    b_jacobi.fit_solve()
    # 块 G-S 迭代法
    gs_block = BlockIterative(A, b, x0, block_vector, eps=1e-15, is_out_info=True, method="G-S")
    gs_block.fit_solve()
    # 块 SOR 迭代法
    sor_block = BlockIterative(A, b, x0, block_vector, eps=1e-15, is_out_info=True, omega=1.03, method="SOR")
    sor_block.fit_solve()
    print("=" * 80)
    # 可视化
    plt.subplot(121 + i)
    iter_n_jacobi = b_jacobi.iterative_info["Iteration_number"] # 获取雅可比迭代次数
    iter_n_jacobi = np.linspace(1, iter_n_jacobi, iter_n_jacobi)
    iter_n_gs = gs_block.iterative_info["Iteration_number"] # 获取 G-S 迭代次数
    iter_n_gs = np.linspace(1, iter_n_gs, iter_n_gs)
```

```

iter_n_sor = sor_block.iterative_info["Iteration_number"] # 获取 G-S 迭代次数
iter_n_sor = np.linspace(1, iter_n_sor, iter_n_sor)
label_text = "$BSOR: \epsilon = %.2e, k = %d, \omega = 1.03$" % (sor_block.precision[-1], iter_n_sor[-1])
plt.semilogy(iter_n_sor, sor_block.precision, "s-", lw=1.5, label=label_text)
plt.semilogy(iter_n_jacobi, b_jacobi.precision, "o--", lw=1.5,
             label="$BJ: \epsilon = %.2e, k = %d$" % (b_jacobi.precision[-1], iter_n_jacobi[-1]))
plt.semilogy(iter_n_gs, gs_block.precision, "p-", lw=1.5,
             label="$BGS: \epsilon = %.2e, k = %d$" % (gs_block.precision[-1], iter_n_gs[-1]))
plt.xlabel("$Iterations(k)$", fontdict={"fontsize": 18})
plt.ylabel("$Precision(\epsilon)$", fontdict={"fontsize": 18})
if i == 0:
    plt.title("块迭代法的 $\epsilon = \|b - Ax^*\|_2$ 收敛曲线, 块向量 $[4,1]$",
             fontdict={"fontsize": 18})
else:
    plt.title("块迭代法的 $\epsilon = \|b - Ax^*\|_2$ 收敛曲线, 块向量 $[2,2,1]$",
             fontdict={"fontsize": 18})
plt.legend(frameon=False, fontsize=16)
plt.tick_params(labelsize=16) # 刻度字体大小 16
plt.grid(ls=":")
plt.show()

```

(1) 块向量 $[2, 2, 1]$

三种块迭代法的解向量一致, 为

$$\mathbf{x}^* = [0.2260363, 0.17354005, 0.14821791, 0.20355884, 0.20467486]^T,$$

解向量的收敛精度和迭代次数如图 7-3 (右) 图例所示, 可见块 SOR 迭代法在当前松弛因子 $\omega = 1.03$ 下最优, 而块高斯—塞德尔迭代法几乎与块 SOR 一致。

(2) 块向量 $[4, 1]$

三种块迭代法的解向量一致, 也与块向量情况下一致, 略去。解向量的收敛精度和迭代次数如图 7-3 (左) 图例所示, 可见块 SOR 迭代法在当前松弛因子 $\omega = 1.03$ 下最优。

(3) 对于此例, 若不采用块迭代法 (可设置块向量 $[1, 1, 1, 1, 1]$), 则满足精度要求下, 雅可比迭代法需迭代 166 次, 高斯—塞德尔迭代法需迭代 22 次, 超松弛迭代法需迭代 23 次。

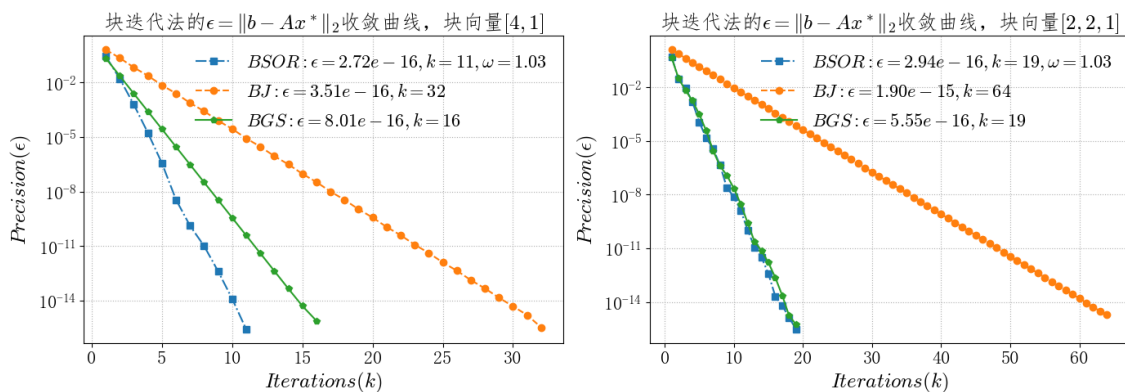


图 7-3 不同的块迭代法在不同的块向量下的解向量的精度收敛曲线

3. 用最速下降法和共轭梯度法求解如下方程组的解, $\mathbf{x}^{(0)} = (0, 0, \dots, 0)^T$, $\varepsilon = 10^{-14}$, 并对比分析.

$$\begin{pmatrix} 3 & -1 & & & & -0.5 \\ -1 & 3 & -1 & & & \\ & -1 & 3 & -1 & & \\ & & -1 & 3 & -1 & \\ & & & -1 & 3 & -1 \\ -0.5 & & & & -1 & 3 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{pmatrix} = \begin{pmatrix} 2.5 \\ 1.5 \\ 1 \\ 1 \\ 1.5 \\ 2.5 \end{pmatrix}.$$

求解代码如下:

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_7_exp3.py
"""
import numpy as np
import matplotlib.pyplot as plt
from iterative_solution_linear_equation_07.conjugate_gradient_method import ConjugateGradientMethod
from iterative_solution_linear_equation_07.steepest_descent_method import SteepestDescentMethod
from iterative_solution_linear_equation_07.pre_conjugate_gradient import PreConjugateGradient

A = -1 * np.array([[ -3, 1, 0, 0, 0, 0.5], [1, -3, 1, 0, 0, 0], [0, 1, -3, 1, 0, 0],
                  [0, 0, 1, -3, 1, 0], [0, 0, 0, 1, -3, 1], [0.5, 0, 0, 0, 1, -3]])
b = np.array([2.5, 1.5, 1, 1, 1.5, 2.5])
x0 = np.zeros(6)

sdm = SteepestDescentMethod(A, b, x0, eps=1e-14, is_out_info=True) # 最速下降法
sdm.fit_solve()
print("=" * 80)

cgm = ConjugateGradientMethod(A, b, x0, eps=1e-14, is_out_info=True) # 共轭梯度法
cgm.fit_solve()
print("=" * 80)

pcg = PreConjugateGradient(A, b, x0, eps=1e-14, is_out_info=True) # 预处理共轭梯度法
pcg.fit_solve()

# 可视化
plt.figure(figsize=(14, 5))
plt.subplot(121)
sdm.plot_convergence_x(is_show=False, style="go-")
plt.subplot(122)
cgm.plot_convergence_x(is_show=False)
plt.show()
```

该线性方程组的系数矩阵的条件数条件数: 4.325211195769102, 良态的且是对称正定矩阵, 共轭梯度法迭代收敛的速度远高于最速下降法, 如图 7 - 4 所示. 如采用预处理共轭梯度法, 需迭代 6 次. 最终三种方法的解向量一致, 为

$$\mathbf{x}^* = [1.57142857, 1.42857143, 1.21428571, 1.21428571, 1.42857143, 1.57142857]^T.$$

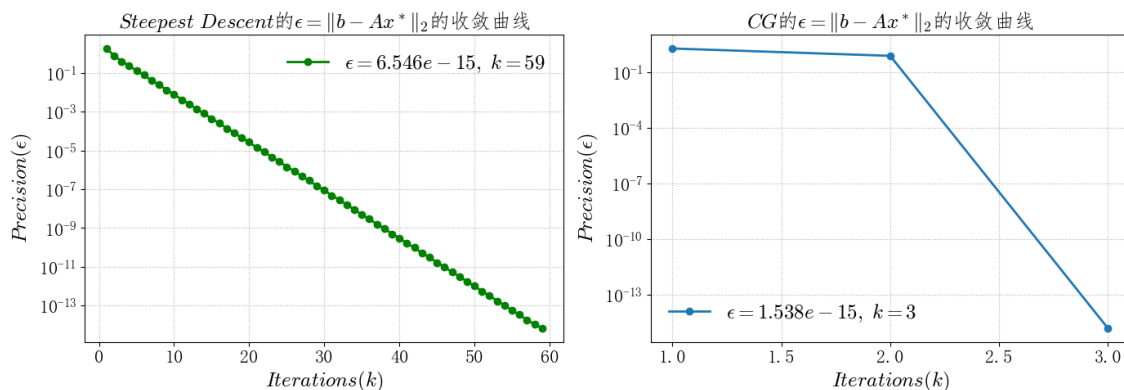


图 7-4 最速下降法与共轭梯度法解向量的精度收敛曲线

4. *如下二维泊松方程的边值问题, 解析解为 $u(x, y) = x^3 + y^3$. 分别采用高斯-塞德尔迭代法、最速下降法、共轭梯度法和预处理共轭梯度法求解, 并对比迭代效率和解向量的精度. 网格划分分别为 10×10 和 20×20 , 其他参数自行设计. 对数值解进行可视化, 并分析误差.

$$\begin{cases} \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 6(x + y), & 0 < x < 1, 0 < y < 1, \\ u(0, y) = y^3, u(1, y) = 1 + y^3, & 0 \leq y \leq 1, \\ u(x, 0) = x^3, u(x, 1) = 1 + x^3, & 0 \leq x \leq 1. \end{cases}$$

求解代码如下:

(1) 定义二维泊松方程模型

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_7_poisson_model.py
"""
class PoissonModel:
    """
    泊松方程模型, 按照二维坐标系命名, u(0, y)为左边界, u(x, 0)为下边界, 依次类推
    """
    fun_xy = lambda x, y: -6 * (x + y) # 泊松方程右端方程, x 和 y 为方程变量, 注意负号
    exact_sol = lambda x, y: x ** 3 + y ** 3 # 泊松方程的精确解
    left_boundary = lambda y: y ** 3 # u(0, y)
    right_boundary = lambda y: 1 + y ** 3 # u(1, y)
    upper_boundary = lambda x: 1 + x ** 3 # u(x, 0)
    lower_boundary = lambda x: x ** 3 # u(x, 1)
```

(2) 求解大型稀疏矩阵的迭代法代码, 在类 `PDESolvePoisson2dModel` 中修改即可, 此外导入已定义的泊松方程模型.

```
from Experiment.chapter_7.poisson_model import PoissonModel # 方程模型
# 以下为求解大型稀疏矩阵的方法
from iterative_solution_linear_equation_07.pre_conjugate_gradient import PreConjugateGradient
from iterative_solution_linear_equation_07.conjugate_gradient_method import ConjugateGradientMethod
```

```

from iterative_solution_linear_equation_07.steepest_descent_method import SteepestDescentMethod
from iterative_solution_linear_equation_07.jacobi_gauss_seidel_iterative import JacobiGSIterativeMethod

@staticmethod
def _solve_sparse_matrix_method_(sp_mat, b):
    """
    求解大型稀疏矩阵的迭代法
    :return:
    """

    x0 = np.zeros(len(b)) # 初始解向量
    pcg = PreConjugateGradient(sp_mat, b, x0, 1e-5, omega=1.5, max_iter=200, is_out_info=True)
    sol_x = pcg.fit_solve() # 求解
    pcg_iter = pcg.iterative_info["Iteration_number"]
    cg = ConjugateGradientMethod(sp_mat, b, x0, 1e-5, max_iter=200, is_out_info=True) # 共轭梯度法
    cg.fit_solve()
    cg_iter = cg.iterative_info["Iteration_number"]
    sdm = SteepestDescentMethod(sp_mat, b, x0, 1e-5, max_iter=400, is_out_info=True) # 最速下降法
    sdm.fit_solve()
    sdm_iter = sdm.iterative_info["Iteration_number"]
    gs = JacobiGSIterativeMethod(sp_mat, b, x0, eps=1e-5, max_iter=400, method="g-s", is_out_info=True) #
    G-S 迭代法
    gs.fit_solve()
    gs_iter = gs.iterative_info["Iteration_number"]

    # 如下代码可实现三种方法的可视化
    plt.figure(figsize=(7, 5))
    plt.semilogy(range(1, pcg_iter + 1), pcg.precision, "o-", lw=2, label="$PCG, \ k=%d$" % pcg_iter)
    plt.semilogy(range(1, cg_iter + 1), cg.precision, "s--", lw=2, label="$CG, \ k=%d$" % cg_iter)
    plt.semilogy(range(1, sdm_iter + 1), sdm.precision, "+-", lw=2, label="$SD, \ k=%d$" % sdm_iter)
    plt.semilogy(range(1, gs_iter + 1), gs.precision, ":", lw=2, label="$GS, \ k=%d$" % gs_iter)
    plt.xlabel("$Iterations(k)$", fontdict={"fontsize": 18})
    plt.ylabel("$Precision(\epsilon)$", fontdict={"fontsize": 18})
    plt.title("各迭代法求解泊松方程的解向量 $x^*$ 收敛曲线", fontdict={"fontsize": 18})
    plt.legend(frameon=False, fontsize=16)
    plt.tick_params(labels=16) # 刻度字体大小 16
    plt.grid(ls=":")
    plt.show()
    return sol_x # 预处理共轭梯度法的解

```

(3) 求解代码如下:

```

# -*- coding: UTF-8 -*-
"""
@file_name: chapter_7_exp4.py
"""

from Experiment.chapter_7.pde_solve_2d_poisson_test import PDESolvePoisson2dModel

```

```

poisson = PDESolvePoisson2dModel([0, 1], [0, 1], 10, 10, is_show=True, is_exact_fun=True)
poisson.fit_pde()

poisson = PDESolvePoisson2dModel([0, 1], [0, 1], 20, 20, is_show=True, is_exact_fun=True)
poisson.fit_pde()

```

求解结果分析. 当网格划分为 10×10 时, 如图 7-5 所示, 预处理共轭梯度法与共轭梯度法具有较快的收敛速度. 如图 7-6 所示, 为采用预处理共轭梯度法求解的解向量, 所得到的泊松方程的数值解曲面与误差曲面, 精度较高.

当网格划分为 20×20 时, 如图 7-5 所示, 预处理共轭梯度法仅需 18 次迭代即可, 共轭梯度法需要迭代 57 次. 最速下降法在最大迭代次数 400 内, 解向量的精度 $\|Ax^* - b\|_2$ 仅为 1.1972075053467717, 收敛速度过慢, 甚至不收敛. 高斯-塞德尔迭代法已达最大迭代次数 400, 解向量的精度 $\|Ax^* - b\|_2$ 仅为 0.007330872507227993, 且在求解过程中出现了溢出 Overflow. 如图 7-6 所示, 为采用预处理共轭梯度法求解的解向量, 所得到的泊松方程的数值解曲面与误差曲面, 从误差曲面可见精度非常高.

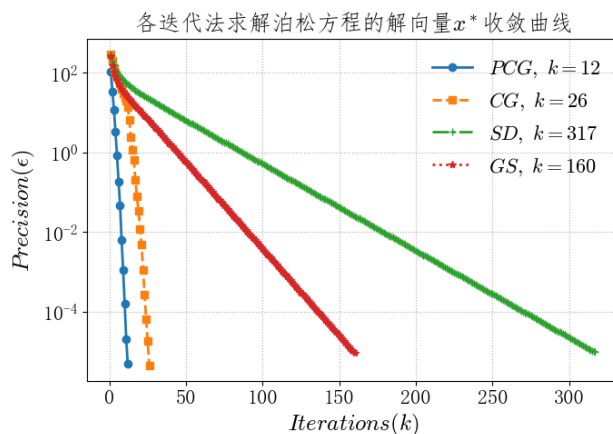


图 7-5 各迭代法的解向量收敛曲线 (网格划分 10×10)

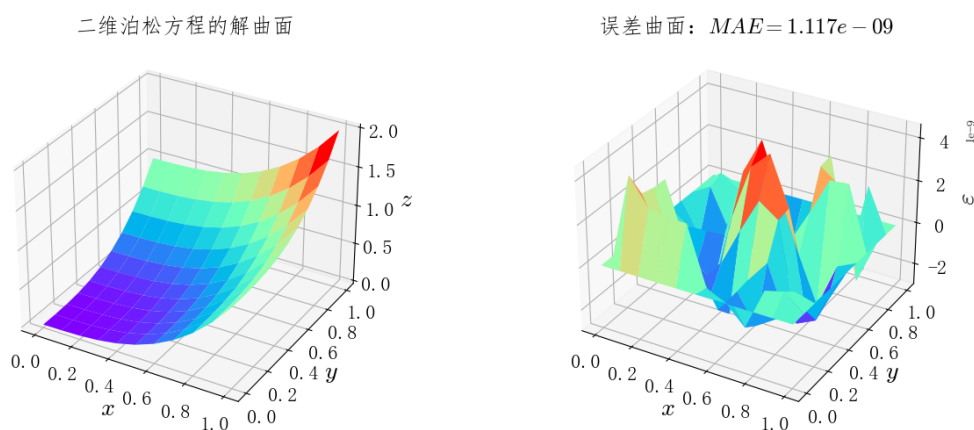


图 7-6 预处理共轭梯度法求解的二维泊松方程数值解曲面与误差曲面 (网格划分 10×10)

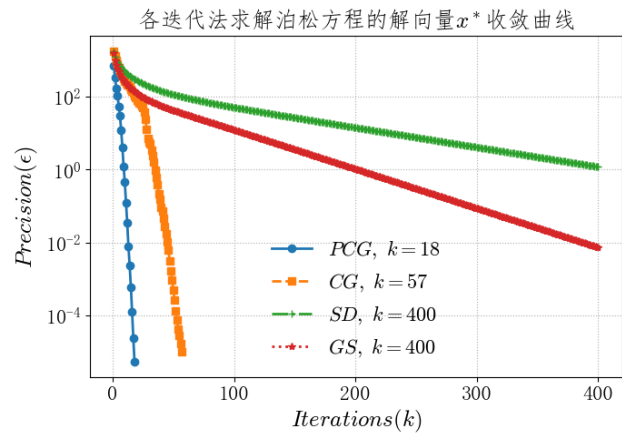


图 7 - 7 各迭代法的解向量收敛曲线 (网格划分 20×20)

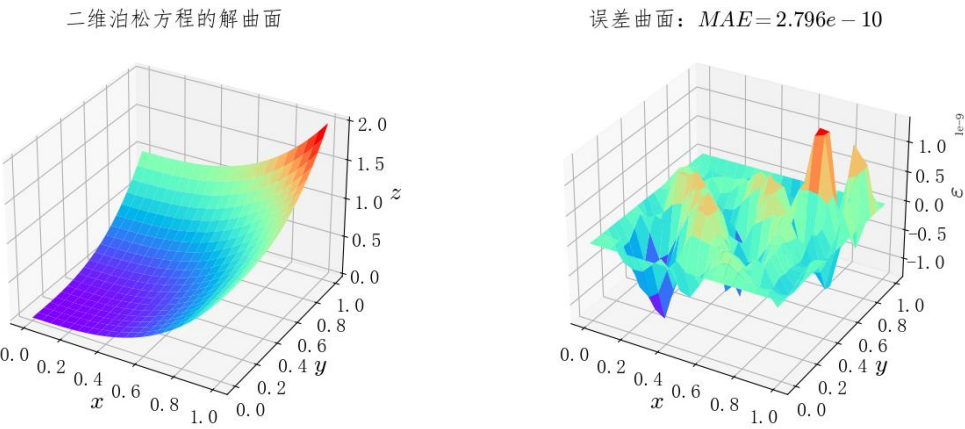


图 7 - 8 预处理共轭梯度法求解的二维泊松方程数值解曲面与误差曲面 (网格划分 20×20)

第 8 章 实验内容参考答案

1. 使用区间分割法求 $e^x + 10x - 2 = 0$ 的近似根, 精度要求 $\varepsilon = 10^{-16}$, 试通过可视化方法, 确定有根区间. 求解代码如下:

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_8_exp1.py
"""

import matplotlib.pyplot as plt
from Experiment.util_font import *
from solving_equation_08.interval_segmentation_method import IntervalSegmentation_Root

equation = lambda x: np.exp(x) + 10 * x - 2 # 待求方程

# 可视化, 确定有根区间
# xi = np.linspace(-1, 1, 200)
# yi = equation(xi)
# plt.figure(figsize=(7, 5))
# plt.plot(xi, yi, "-")
# plt.show()

span = [-1, 1] # 有根区间
options = {"eps": 1e-16, "display": "display", "pltFuns": False} # 参数设置, 某些参数可不设置
funEvals = ["regula", "dichotomy"]
linestyle = ["-", "--", "-."]
roots, eps = [], []
plt.figure(figsize=(14, 5))
for i, funEval in enumerate(funEvals):
    ism = IntervalSegmentation_Root(equation, span, eps=1e-16, display="display", funEval=funEval)
    ism.fit_root() # 求近似根
    print("%.25e, %.25f" % (equation(ism.root_precision_info[-1, -2]), ism.root_precision_info[-1, -2]))
    eps.append(ism.root_precision_info[:, -1])
    roots.append(ism.root_precision_info[:, -2])

plt.subplot(121)
plt.plot(np.arange(1, len(roots[0]) + 1), roots[0], "-s", lw=2,
         label="$\%s: \ x^{\%s} = \%.15f$" % (funEvals[0], roots[0][-1]))
plt.plot(np.arange(1, 16), roots[1][:15], "--o", lw=2, label="$\%s: \ x^{\%s} = \%.15f$" % (funEvals[1], roots[1][-1]))
plt.xlabel("$Iterations(k)$", fontdict={"fontsize": 18})
plt.ylabel("$x^k$", fontdict={"fontsize": 18})
plt.title("区间分割法近似根$x^k$的收敛曲线（前 15 次迭代）", fontdict={"fontsize": 18})
plt.legend(frameon=False, fontsize=16)
plt.tick_params(labelsize=16) # 刻度字体大小 16
plt.grid(ls=":")
```

```
plt.subplot(122)
plt.plot(np.arange(1, 11), eps[0][:10], "-s", lw=2, label="$%s: \ k=%d$" % (funEvals[0], len(eps[0])))
plt.plot(np.arange(1, 11), eps[1][:10], "--o", lw=2, label="$%s: \ k=%d$" % (funEvals[1], len(eps[1])))
plt.xlabel("$Iterations(k)$", fontdict={"fontsize": 18})
plt.ylabel("$f(x^*)$", fontdict={"fontsize": 18})
plt.title("近似根$x^*$的精度$f(x^*)$收敛曲线（前 10 次迭代）", fontdict={"fontsize": 18})
plt.legend(frameon=False, fontsize=16)
plt.tick_params(labelsize=16) # 刻度字体大小 16
plt.grid(ls=":")
plt.show()
```

两种方法所求近似根为 $x^* = 0.0905251013072549959304069$ ，精度为 $f(x^*) = 0.0$ 。其中试值法迭代 13 次收敛，而二分法需迭代 52 次。

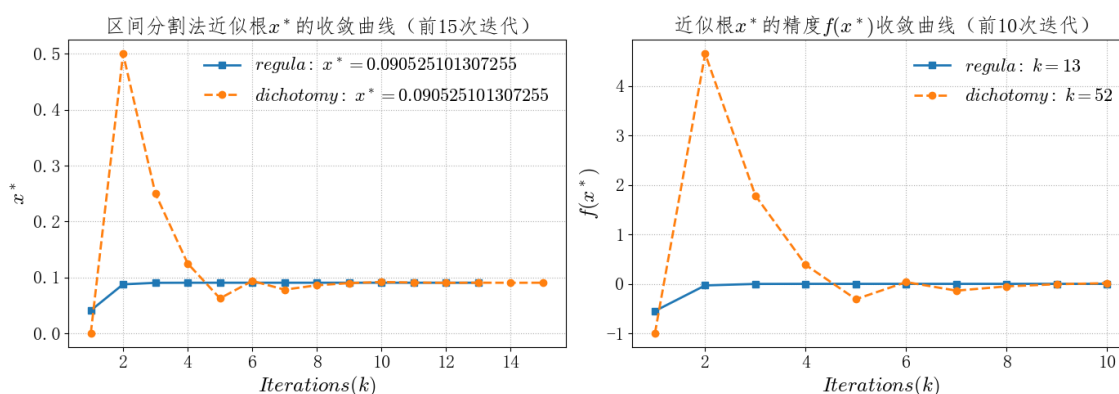


图 8-1 近似根以及近似根的精度收敛曲线

2. 用不动点迭代法、埃特金加速法和斯特芬森加速法求解 $3x^2 - e^x = 0$ 在 $[3, 4]$ 中的近似根，精度要求 $\varepsilon = 10^{-16}$ ，迭代公式为 $x_{k+1} = \varphi(x_k) = 2\ln x_k + \ln 3$ ，初值 $x_0 = 3.5$ 。

求解代码如下：

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_8_exp2.py
"""
import numpy as np
from solving_equation_08.iterative_solution_method import IterativeSolutionMethod_Root
from Experiment.util_font import *

fai_x = lambda x: 2 * np.log(x) + np.log(3) # 迭代公式
x0 = 3.5 # 迭代初值

method_eval = ["stable", "aitken", "steffensen"]
plt.figure(figsize=(7, 5))
line_style = ["*-.", "o--", "s-"]
for method, style in zip(method_eval, line_style):
    ism = IterativeSolutionMethod_Root(fai_x, x0=x0, eps=1e-16, max_iter=200, method=method)
```

```

ism.fit_root()
info = ism.root_precision_info
plt.semilogy(info[:, 0], info[:, 2], style,
              label="$%s: \ k=%d, \ \epsilon=%.2e$" % (method, len(info), info[-1, 2]))
plt.xlabel("$Iterations(k)$", fontdict={"fontsize": 18})
plt.ylabel("$Precision(\epsilon)$", fontdict={"fontsize": 18})
plt.title(r"各迭代法近似根$x^*$的$\epsilon=\text{vert } x_{k+1} - x_k \text{vert}$收敛曲线",
          fontdict={"fontsize": 18})
plt.legend(frameon=False, fontsize=16)
plt.tick_params(labelsize=16) # 刻度字体大小 16
plt.grid(ls=":")
plt.show()

```

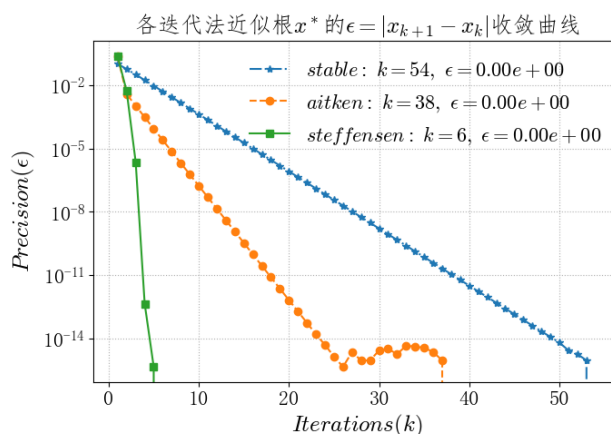


图 8-2 不同迭代法求解近似根的精度收敛曲线

- (1) 不动点迭代法需迭代 54 次, 近似根为 $x^* = 3.7330790286328134897075870$.
- (2) 埃特金加速法需迭代 38 次, 近似根为 $x^* = 3.7330790286328130456183771$.
- (3) 斯特芬森加速法需迭代 6 次, 近似根为 $x^* = 3.7330790286328143778860067$.

三种方法的精度在 Python 容差内, 均为 $f(x^*) = 0.0$, 精度收敛图像如图 8-2 所示.

3. 用各种牛顿法求解方程 $2e^{-x} \sin x = 0$ 的近似根, 精度要求 $\epsilon = 10^{-16}$, 初值为 $x_0 = 0.5$ 和 $x_0 = 3$.

求解代码如下:

```

# -*- coding: UTF-8 -*-
"""
@file_name: chapter_8_exp3.py
"""
import numpy as np
import sympy
from Experiment.util_font import *
from solving_equation_08.newton_root_method import NewtonRootMethod

t = sympy.Symbol("t")
equ = 2 * sympy.exp(-t) * sympy.sin(t)
x_init = [0.5, 3.0] # 迭代初值

```



```
plt.figure(figsize=(14, 5))
line_style = ["*:", "o--", "+-", "p-"]
method_vector = ["newton", "halley", "downhill", "multiroot"]
for i, x0 in enumerate(x_init):
    plt.subplot(121 + i)
    for style, method in zip(line_style, method_vector):
        newton = NewtonRootMethod(equ, x0=x0, eps=1e-16, method=method)
        newton.fit_root()
        iter_root_precision = np.asarray(newton.root_precision_info, np.float)
        plt.semilogy(iter_root_precision[:, 0], iter_root_precision[:, 2], style, lw=2,
                      label="$%s(k=%d, \epsilon=%.2e)$" %
                          (method, len(iter_root_precision), iter_root_precision[-1, 2]))
    plt.xlabel("$Iterations(k)$", fontdict={"fontsize": 18})
    plt.ylabel("$Precision(\epsilon)$", fontdict={"fontsize": 18})
    plt.title(r"各牛顿迭代法$(x_0=0.1f)$: $x_k$的$\epsilon=\text{vert } f(x_k) \text{vert}$收敛曲线" % x0,
              fontdict={"fontsize": 18})
    plt.legend(frameon=False, fontsize=16)
    plt.tick_params(labelsize=16) # 刻度字体大小 16
    plt.grid(ls=":")
plt.show()
```

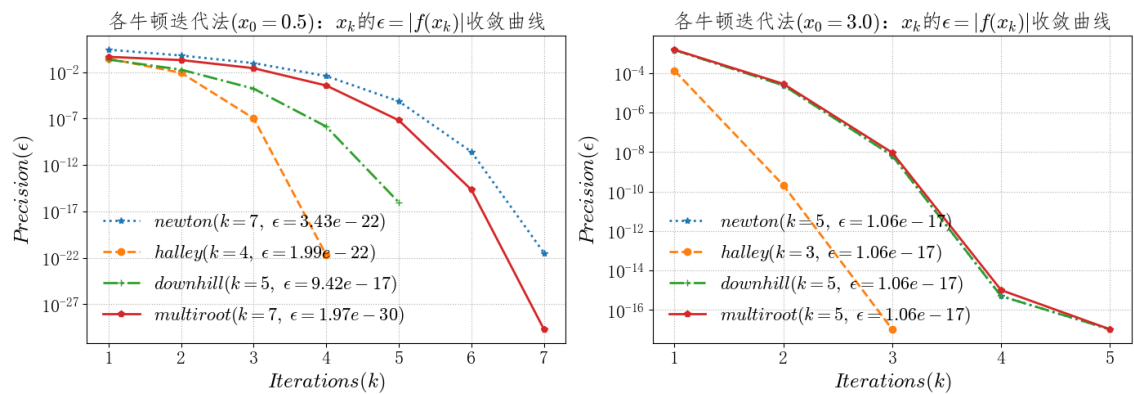


图 8 - 3 各牛顿迭代法在不同初值下近似根的精度收敛曲线

各牛顿迭代法在不同初值下近似根的精度收敛曲线如图 8 - 3 所示, 结果如表 8 - 1 所示. 其中当 $x_0 = 0.5$ 时, 牛顿下山法第 1 次迭代需要下山因子 0.5. 由于方程不存在重根, 故牛顿法与重根情形下的迭代结果一致. 哈利法始终表现较为优越.

表 8 - 1 各牛顿迭代法在不同初值下近似根以及精度

方法	初值 x_0	迭代次数	近似根 x^*	精度 $ f(x^*) $
牛顿法	0.5	7	-0.00000000000000000000	3.429083691687112e-22
哈利法		4	0.00000000000000000000	1.985233470127266e-22
牛顿下山法		5	-0.000000000000000004712	9.424270716322230e-17
重根情形		7	0.00000000000000000000	1.972152263052530e-30
牛顿法	3.0	5	3.14159265358979311600	1.058435733606881e-17

哈利法		3	3.14159265358979311600	1.058435733606881e-17
牛顿下山法		5	3.14159265358979311600	1.058435733606881e-17
重根情形		5	3.14159265358979311600	1.058435733606881e-17

4. 用弦截法和抛物线法求解 $4x^4 - 10x^3 + 1.25x^2 + 5x - 0.5 = 0$ 的全部实根, 精度要求 $\varepsilon = 10^{-16}$, 试通过可视化方法, 确定有根区间.

求解代码如下:

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_8_exp4.py
"""

import numpy as np
from solving_equation_08.double_points_secant_method import DoublePointsSecantMethod
from solving_equation_08.parabola_method import ThreePointParabolaMethod
from Experiment.util_font import *

fx = lambda x: 4 * x ** 4 - 10 * x ** 3 + 1.25 * x ** 2 + 5 * x - 0.5
# 可视化, 确定有根区间
xi = np.linspace(-1, 2.5, 200)
yi = fx(xi)
plt.figure(figsize=(7, 5))
plt.plot(xi, yi, "-", lw=2)
plt.plot(xi, np.zeros(len(xi)), "-", lw=0.5)
plt.xlabel("$x$", fontdict={"fontsize": 18})
plt.ylabel("$y$", fontdict={"fontsize": 18})
plt.title("非线性方程在区间$[-1, 2.5]$内的有根情况", fontdict={"fontsize": 18})
plt.grid(ls=":")
plt.tick_params(labels=16) # 刻度字体大小 16
plt.show()

x_span = np.array([[ -0.7, -0.5], [ -0.2, 0.2], [ 0.8, 1.2], [ 2.0, 2.1]])
for i, span in enumerate(x_span):
    sm = DoublePointsSecantMethod(fx, span, eps=1e-16, display="display") # 弦截法
    sm.fit_root()
    pm = ThreePointParabolaMethod(fx, span, eps=1e-16, display="display") # 抛物线法
    pm.fit_root()
print("=" * 80)
```

如图 8-4 所示, 可确定四个有根区间为 $\{[-0.7, -0.5], [-0.2, 0.2], [0.8, 1.2], [2.0, 2.1]\}$. 求解结果如表 8-2 所示. 由于算法在迭代过程中, 可能满足其他终止的条件, 故最终的精度未必满足要求, 但精度也足够高.

表 8-2 弦截法与抛物线法在不同区间下的近似根以及精度

方法	有根区间	迭代次数	近似根 x^*	精度 $ f(x^*) $
----	------	------	-----------	---------------

弦截法	[-0.7, -0.5]	7	-0.63185728794208895209	-8.881784197001252e-16
抛物线法		5	-0.63185728794208895209	-8.881784197001252e-16
弦截法	[-0.2, 0.2]	4	0.09941613231752083335	0.000000000000000e+00
抛物线法		3	0.09941613231752084723	0.000000000000000e+00
弦截法	[0.8, 1.2]	5	0.96033249593488856366	0.000000000000000e+00
抛物线法		3	0.96033249593488856366	0.000000000000000e+00
弦截法	[2.0, 2.1]	9	2.07210865968967938855	-8.881784197001252e-15
抛物线法		4	2.07210865968967938855	-8.881784197001252e-15

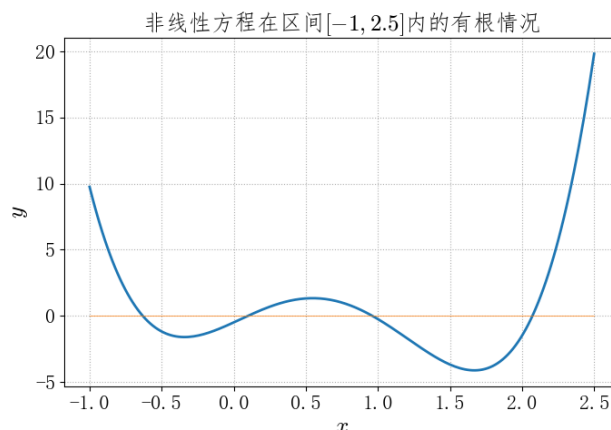


图 8-4 非线性方程的 4 各个实根区间

5. 用劈因子法求解 $16x^4 - 40x^3 + 5x^2 + 20x + 6 = 0$ 的一对共轭复根, 二次因子初值 $(p_0, q_0) = (1, 3)$.

求解代码如下:

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_8_exp5.py
"""
import numpy as np
from solving_equation_08.cut_factor_method import CutFactorMethod

P = np.array([16, -40, 5, 20, 6])
cfm = CutFactorMethod(P, p0=1, q0=3)
cfm.fit_cut_factor()
print("二次因子系数: ", cfm.omega_x)
print("根: ", cfm.root)
print("精度: ", cfm.precision)
```

所求二次因子为 $\omega^*(x) = x^2 + 0.71212352x + 0.15327027$. 方程的一对共轭复根为

$$-0.35606176 + 0.16275838j, -0.35606176 - 0.16275838j.$$

精度均为 $|f(x^*)| = 3.85818233 \times 10^{-9}$.

6. 用逐次压缩牛顿法求解多项式 $x^4 + 4x^3 + 2x^2 - 4x - 3 = 0$ 的全部零点, 并验证精度.

求解代码如下:

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_8_exp6.py
"""

import numpy as np
from solving_equation_08.successive_compression_newton_method import SuccessiveCompressionNewton

p_coeff = np.array([1, 4, 2, -4, -3])
scn = SuccessiveCompressionNewton(p_coeff, 1e-16, 10000)
scn.fit_root()
scn.plt_polynomial_root()
print("多项式的全部零点和精度: ", )
for i in range(len(scn.root)):
    print("%.15f, %.15e" % (scn.root[i], scn.precision[i]))
```

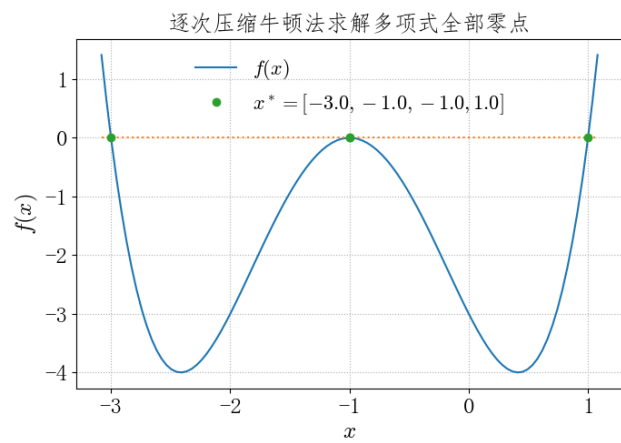


图 8 - 5 逐次压缩牛顿法求解多项式的全部零点示意图

表 8 - 3 逐次压缩牛顿法求解多项式的全部零点以及精度 ($k = 1, 2, 3, 4$)

k	1	2	3	4
零点 x_k^*	1.0000000000000000	-0.999999999631897	-1.000000000368105	-3.0000000000000000
精度 $ f(x_k^*) $	0.0000000000000000e+00	0.0000000000000000e+00	0.0000000000000000e+00	0.0000000000000000e+00

第 9 章 实验内容参考答案

1. 试采用不动点迭代法求解如下线性方程组, 初值可通过隐函数可视化的方法确定, 精度要求 $\varepsilon = 10^{-16}$.

$$\begin{cases} x^3 + y^3 - 6x + 3 = 0, \\ x^3 - y^3 - 6y + 2 = 0. \end{cases}$$

令 $\mathbf{x} = (x_1, x_2)^T$, 构造迭代公式如下:

$$\Phi(\mathbf{x}) = \begin{pmatrix} \varphi_1(\mathbf{x}) \\ \varphi_2(\mathbf{x}) \end{pmatrix} = \begin{pmatrix} \frac{1}{6}x_1^3 + \frac{1}{6}x_2^3 + \frac{1}{2} \\ \frac{1}{6}x_1^3 - \frac{1}{6}x_2^3 + \frac{1}{3} \end{pmatrix}.$$

求解代码:

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_9_expl.py
"""
import sympy
from Experiment.util_font import *
import numpy as np
from nonlinear_equations_09.nlinequs_fixed_point import NLinearFxFixedPoint

# 可视化方法
plt.figure(figsize=(7, 5))
x, y = sympy.symbols("x, y")
nlin_equs = [x ** 3 + y ** 3 - 6 * x + 3, x ** 3 - y ** 3 - 6 * y + 2] # 定义的非线性方程组
p0 = sympy.plot_implicit(nlin_equs[0], show=False, line_color="r", points=500) # 绘制第一个方程
p1 = sympy.plot_implicit(nlin_equs[1], show=False, line_color="b", points=500) # 绘制第二个方程
p0.extend(p1) # 在方程 1 的基础上添加方程 2 的图像
p0.show()

def nlin_equs(x):
    """
    非线性方程组的定义
    :param x: 向量, x[0] 表示 x, x[1] 表示 y
    :return:
    """
    y = [x[0] ** 3 / 6 + x[1] ** 3 / 6 + 0.5, x[0] ** 3 / 6 - x[1] ** 3 / 6 + 1 / 3]
    return np.asarray(y, dtype=np.float)

x_init = np.array([[-2.5, -1.5], [0.5, 0.5], [1.8, 1.2]])
nlf = NLinearFxFixedPoint(nlin_equs, x_init[1], max_iter=1000, eps=1e-16, is_plt=True)
nlf.fit_roots()
print("x1: %.10f, x2: %.10f, 精度: %.10e" % (x[1][0], x[1][1], x[2]))
```

从中可以看出, 方程组有 3 个解, 设置初值为 $\{(-2.5, -1.5)^T, (0.5, 0.5)^T, (1.8, 1.2)^T\}$, 但只有

$(0.5, 0.5)^T$ 满足不动点迭代法收敛性条件. 其解向量与验证精度为

$$\mathbf{x}^* = (0.5323703723, 0.3512574476)^T, \|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\| = 0.0.$$

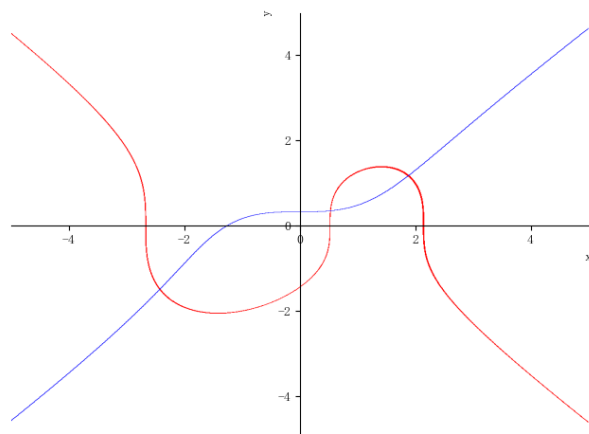


图 9-1 非线性方程组的隐式图像

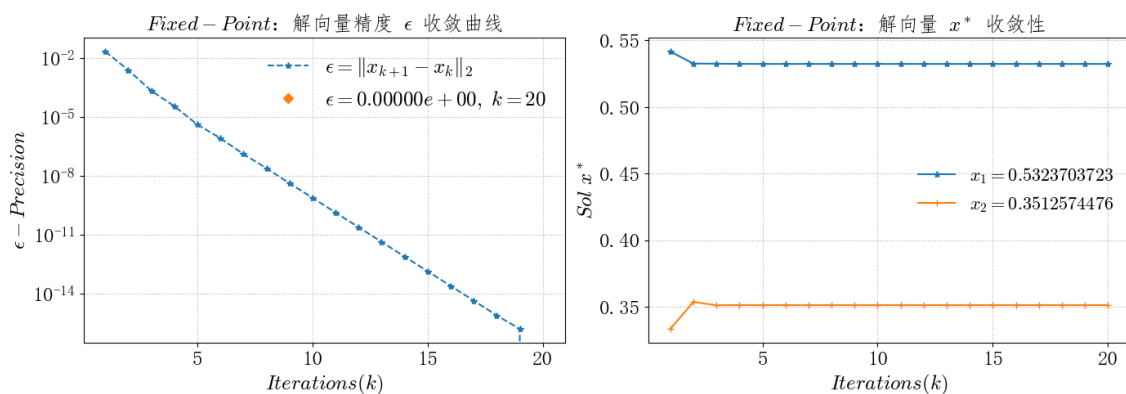


图 9-2 不动点迭代法在初值 $(0.5, 0.5)^T$ 时的收敛性

2. 试采用各牛顿迭代法求解如下非线性方程组, 初值可通过隐函数可视化的方法确定, 精度要求 $\varepsilon = 10^{-15}$, 其他参数自定.

$$\begin{cases} 3x - \cos x - \sin y = 0, \\ 4y - \sin x - \cos y = 0. \end{cases}$$

求解代码如下:

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_9_exp2.py
"""
import sympy
from Experiment.util_font import *
import numpy as np
from nonlinear_equations_09.nlinequs_newton import NLinearFxNewton

x, y = sympy.symbols("x, y")
nlin_equs = [3 * x - sympy.cos(x) - sympy.sin(y), 4 * y - sympy.sin(x) - sympy.cos(y)] # 定义的非线性方程组
```

```
# 可视化方法
plt.figure(figsize=(7, 5))
p0 = sympy.plot_implicit(nlin_equs[0], show=False, line_color="r", points=500) # 绘制第一个方程
p1 = sympy.plot_implicit(nlin_equs[1], show=False, line_color="b", points=500) # 绘制第二个方程
p0.extend(p1) # 在方程 1 的基础上添加方程 2 的图像
p0.show()

# 牛顿法求解
x0 = np.array([0.5, 0.5])
mni = NLinearFxNewton(nlin_equs, [x, y], x0, max_iter=200, eps=1e-15, method="downhill", is_plt=True)
mni.fit_roots()
print(mni.iter_roots_precision[-1])
print(mni.fxs_precision)
print("下山因子: ", mni.downhill_lambda)
print("=" * 80)
```

牛顿下山法在不同初值下的计算结果如表 9 - 1 所示, 由于不需要下山因子, 故牛顿法与牛顿下山法结果一致. 略去方程组的隐式绘制图像.

表 9 - 1 牛顿下山法在不同初值下的计算结果

初值	迭代次数	x^*	y^*	$ f_1(x^*, y^*) $	$ f_2(x^*, y^*) $	$\ F(x^*, y^*)\ _2$
$(0, 0)^T$	6	0.41516943	0.33679122	1.66533454e-16	2.22044605e-16	2.77555756e-16
$(0.2, 0.2)^T$	6	0.41516943	0.33679122	1.11022302e-16	0.00000000e+00	1.11022302e-16
$(0.5, 0.5)^T$	5	0.41516943	0.33679122	1.66533454e-16	2.22044605e-16	2.77555756e-16
$(2, 2)^T$	8	0.41516943	0.33679122	1.11022302e-16	0.00000000e+00	1.11022302e-16

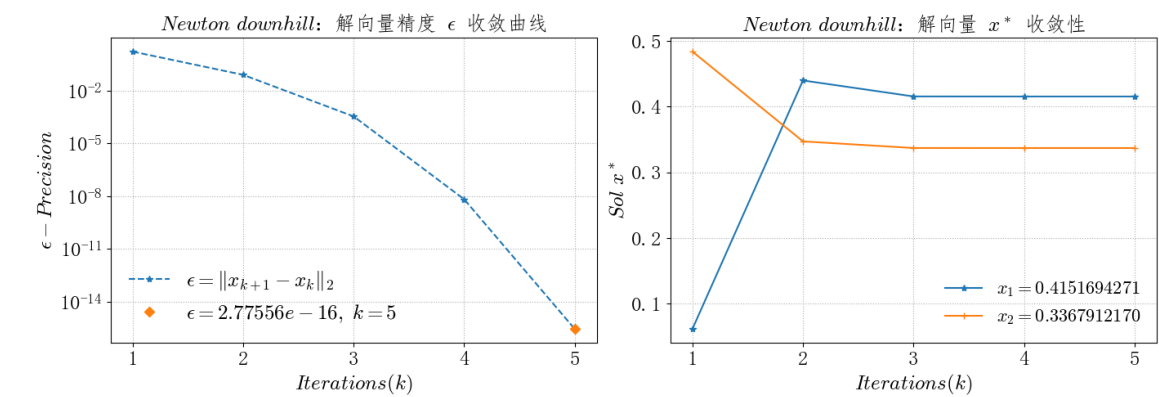


图 9 - 3 牛顿下山法在初值 $(0.5, 0.5)^T$ 时的收敛性

3. 试采用秩 1 和秩 2 算法求解如下非线性方程组的近似解, 精度要求 $\epsilon = 10^{-15}$, 若采用离散差商矩阵作为初始修正矩阵, 则步长 $h = 0.01$, 初值分别为 $(-0.8, 1.5)^T$ 、 $(1, -1)^T$ 和 $(-0.5, 2.2)^T$, 也可通过绘制隐函数图象确定初始值.

$$\begin{cases} x^2 e^{-\frac{xy^2}{2}} + e^{-\frac{x}{2}} \sin(xy) = 0, \\ y^2 \cos(x + y^2) + x^2 e^{x+y} = 0. \end{cases}$$

求解代码如下:

```

# -*- coding: UTF-8 -*-
"""
@file_name: chapter_9_exp3.py
"""

import sympy
import numpy as np
import time
from Experiment.util_font import *
from nonlinear_equations_09.rank2_quasi_newton_dd import Rank2QuasiNewton
from nonlinear_equations_09.rank1_quasi_newton_dd import Rank1QuasiNewton

plt.figure(figsize=(7, 5))
x, y = sympy.symbols("x, y")
nlin_equs = [x ** 2 * sympy.exp(-x * y ** 2 / 2) + sympy.exp(-x / 2) * sympy.sin(x * y),
             y ** 2 * sympy.cos(x + y ** 2) + x ** 2 * sympy.exp(x + y)]
p0 = sympy.plot_implicit(nlin_equs[0], show=False, line_color="r")
p1 = sympy.plot_implicit(nlin_equs[1], show=False, line_color="c")
p0.extend(p1)
p0.show()

def nlin_funs(x):
    nlinequs = [x[0] ** 2 * np.exp(-x[0] * x[1] ** 2 / 2) + np.exp(-x[0] / 2) * np.sin(x[0] * x[1]),
               x[1] ** 2 * np.cos(x[0] + x[1] ** 2) + x[0] ** 2 * np.exp(x[0] + x[1])]
    return np.asarray(nlinequs, dtype=np.float64)

x_init = np.array([[ -0.8, 1.5], [1, -1], [ -0.5, 2.2]]) # 迭代初值
for x0 in x_init:
    plt.figure(figsize=(14, 5))
    line_style = ["-o", ":", "-.", "-p"] # 线型与点型
    class_ = ["Rank1QuasiNewton", "Rank2QuasiNewton"] # 类名, 秩1 和秩2 算法
    for k in range(2):
        plt.subplot(121 + k) # 绘制子图
        if k == 0:
            method_list = ["Broyden", "Broyden2Th", "InvBroyden", "InvBroyden2Th"] # 秩1 算法
        else:
            method_list = ["DFP", "BFGS", "BFGS", "InvBFGS"] # 秩2 算法
        for i, method in enumerate(method_list):
            start = time.time() # 标记开始时间
            msr = eval(class_[k])(nlin_funs, x0, max_iter=1000, eps=1e-15, method=method,
                                is_plt=False, diff_mat_h=0.01)
            msr.fit_nlin_roots() # 求解近似解
            end = time.time() # 标记算法终止时间
            print("时间: ", end - start)
            print(method, ":", x0, msr.iter_roots_precision[-1])

```



```

print("=" * 80)
error_2 = np.linalg.norm(nlin_funs(msr.roots)) # 解的精度
irp = [rt[-1] for rt in msr.iter_roots_precision] # 获取其迭代精度
label_txt = method + ": k=" + str(len(irp)) + ", \epsilon_{F_2}=%.3e" % error_2
plt.semilogy(range(1, len(irp) + 1), irp, line_style[i], label="$%s$" % label_txt)
plt.xlabel(r"$Iterations(k)$", fontdict={"fontsize": 18})
plt.ylabel(r"$\epsilon$-Precision", fontdict={"fontsize": 18})
if k == 0:
    plt.title("不同秩 1 拟牛顿法的解向量 $x^*$ 精度收敛性", fontdict={"fontsize": 18})
else:
    plt.title("不同秩 2 拟牛顿法的解向量 $x^*$ 精度收敛性", fontdict={"fontsize": 18})
plt.legend(frameon=False, fontsize=16)
plt.tick_params(labelsize=18) # 刻度字体大小 18
plt.grid(ls=":")
plt.show()

```

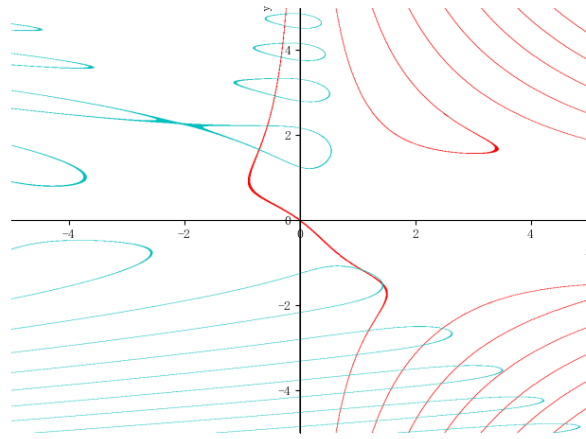
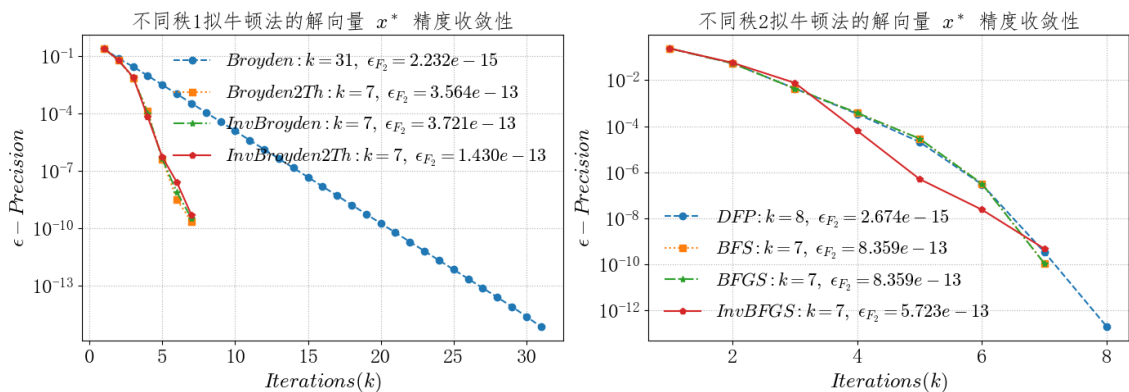


图 9-4 非线性方程组的隐式绘制图像

图 9-5 不同的拟牛顿法在初值 $(-0.8, 1.5)^T$ 时的收敛性

(1) 初值 $(-0.8, 1.5)^T$ 时, 八种方法均收敛, 保留小数点后 8 位的解向量一致, 如下:

$$\mathbf{x}^* = (-0.70380126, 1.66165329)^T.$$

其各种方法的收敛速度以及最终的收敛精度如图 9-5 所示. 具体信息可通过求解代码打印输出.

(2) 初值 $(1, -1)^T$ 时, 八种方法均收敛, 保留小数点后 8 位的解向量一致, 如下:

$$\mathbf{x}^* = (1.0824701, -1.17365078)^T.$$

其各种方法的收敛速度以及最终的收敛精度如图 9-6 所示. 具体信息可通过求解代码打印输出.

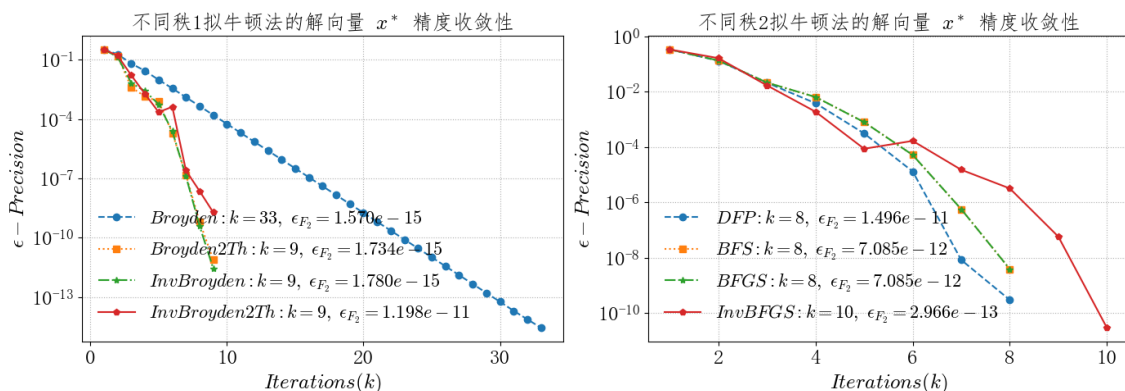


图 9-6 不同的拟牛顿法在初值 $(1, -1)^T$ 时的收敛性

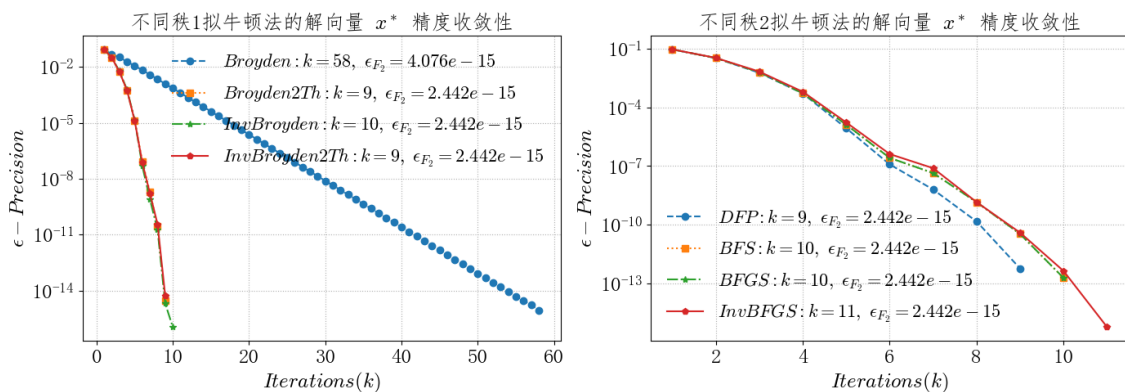


图 9-7 不同的拟牛顿法在初值 $(-0.5, 2.2)^T$ 时的收敛性

(3) 初值 $(-0.5, 2.2)^T$ 时, 八种方法均收敛, 保留小数点后 8 位的解向量一致, 如下:

$$\mathbf{x}^* = (-0.56091405, 2.22063176)^T.$$

其各种方法的收敛速度以及最终的收敛精度如图 9-7 所示. 具体信息可通过求解代码打印输出.

4. 试采用高斯—牛顿法、阻尼最小二乘法 and 全局化 LM 法求解如下非线性方程组, 精度要求 $\varepsilon = 10^{-16}$, 其他参数自定.

$$\begin{cases} 4x - y + 0.1e^x - 1 = 0, \\ -x + 4y + 0.125x^2 = 0. \end{cases}$$

求解代码如下:

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_9_exp4.py
"""
import sympy
import numpy as np
from Experiment.util_font import *
from nonlinear_equations_09.gauss_newton import GaussNewtonIteration
from nonlinear_equations_09.damped_least_square_method import DampedLeastSquare_LM
```

```

from nonlinear_equations_09.global_LM_method import GlobalLevenbergMarquardt

plt.figure(figsize=(7, 5))
x, y = sympy.symbols("x, y")
nlin_equs = [4 * x - y + 0.1 * sympy.exp(x) - 1, -x + 4 * y + 0.125 * x ** 2]
p0 = sympy.plot_implicit(nlin_equs[0], show=False, line_color="r")
p1 = sympy.plot_implicit(nlin_equs[1], show=False, line_color="c")
p0.extend(p1)
p0.show()

def nlin_funs(x):
    nlinequs = [4 * x[0] - x[1] + 0.1 * np.exp(x[0]) - 1, -x[0] + 4 * x[1] + 0.125 * x[0] ** 2]
    return np.asarray(nlinequs, dtype=np.float)

x0 = np.array([0, 0]) # 初值
h = [0.01, 0.01]
# 高斯—牛顿法
gni = GaussNewtonIteration(nlin_funs, x0, h, max_iter=1000, eps=1e-16, is_plt=True)
gni.fit_roots()
print(gni.iter_roots_precision[-1])

# 阻尼最小二乘
dls = DampedLeastSquare_LM(nlin_funs, x0, h, u=0.01, v=5, max_iter=1000, eps=1e-16, is_plt=True)
dls.fit_nlinequs_roots()
print(dls.iter_roots_precision[-1])

# 全局化LM法
glm = GlobalLevenbergMarquardt(nlin_funs, x0, h, delta=1, max_iter=1000, eps=1e-16, is_plt=True)
glm.fit_nlinequs_roots()
print(glm.iter_roots_precision[-1])

```

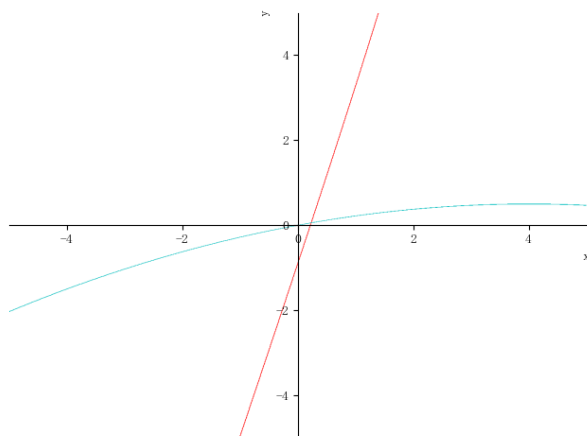


图 9-8 非线性方程组隐式绘制图像

从图 9-8 可以看出, 方程组在 $(0, 0)^T$ 附近存在一个解. 设置离散差商矩阵的微分步长 $h = 0.01$, 则保留小数点后 8 位, 三种方法求解解向量一致, 如下:

$$x^* = (0.23256701, 0.05645152)^T.$$

三种方法的收敛性以及收敛精度如图 9-9、图 9-10 和图 9-11 所示。

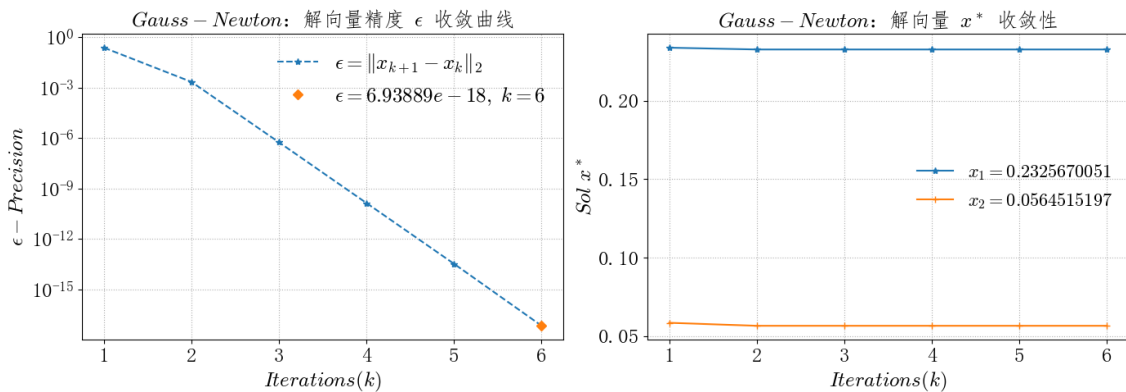


图 9-9 高斯—牛顿法收敛性示意图

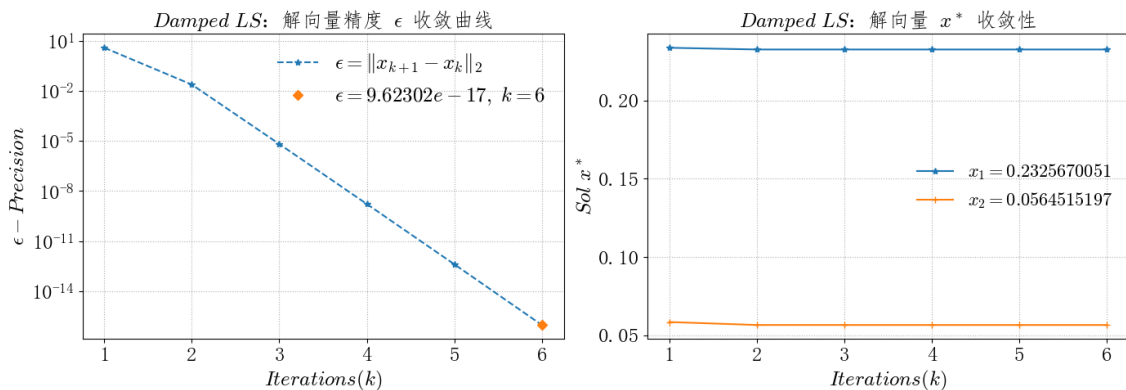


图 9-10 阻尼最小二乘法收敛性示意图

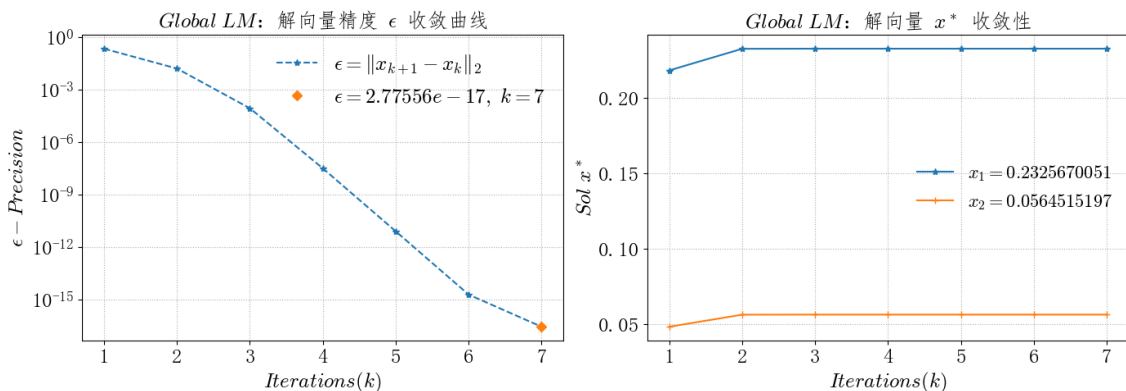


图 9-11 全局 LM 法收敛性示意图

5. 采用同伦延拓法求解如下非线性方程组的解，分别取 $N=4$ 、 $N=8$ 和 $N=10$ ，初值可取 $(0, 0, 0)^T$ 和 $(1, 0, -1)^T$ 。

$$\begin{cases} 3x - \cos yz - 0.5 = 0, \\ 4x^2 - 625y^2 + 2y - 1 = 0, \\ e^{-xy} + 20z + 10\pi/3 - 1 = 0. \end{cases}$$

求解代码如下：

```

# -*- coding: UTF-8 -*-
"""
@file_name: chapter_9_exp5.py
"""

import sympy
import numpy as np
from nonlinear_equations_09.homotopy_continuation_method import HomotopyContinuationMethod

x_init = np.array([[0, 0, 0], [1, 0, -1]])
N_num = [4, 8, 10]
for x0 in x_init:
    for n in N_num:
        x, y, z = sympy.symbols("x, y, z")
        nlin_funs = [3 * x - sympy.cos(y * z) - 0.5,
                     4 * x ** 2 - 625 * y ** 2 + 2 * y - 1,
                     sympy.exp(-x * y) + 20 * z + 10 * sympy.pi / 3 - 1]
        hcm = HomotopyContinuationMethod(nlin_funs, [x, y, z], x0, N=n, method="newton")
        x, eps = hcm.fit_roots()
        print(x0, n)
        for i in range(3):
            print("x = %.16f, %.10e" % (x[i], eps[i]))
        print("=" * 60)

```

表 9-2 同伦延拓法在不同条件下的求解结果

初值	N	近似解向量 $\mathbf{x}^* = (x^*, y^*, z^*)^T$, 以及精度 $\mathbf{F}(\mathbf{x}^*)$			
$(0, 0, 0)^T$	4	$\mathbf{x}^*$	0.4999995226806401	0.0032431288993785	-0.5235177627293582
		$\mathbf{F}(\mathbf{x}^*)$	9.3644120236e-09	-8.9329639011e-05	8.4999010434e-09
	8	$\mathbf{x}^*$	0.5000000000000004	-0.000000000027652	-5.5285318201e-12
		$\mathbf{F}(\mathbf{x}^*)$	1.3322676296e-15	-0.5235987755983680	-1.8390334675e-16
	10	$\mathbf{x}^*$	0.5000000000000000	0.0000000000000005	-0.5235987755982989
		$\mathbf{F}(\mathbf{x}^*)$	0.0000000000e+00	9.0796849748e-16	-1.9602601862e-15
$(1, 0, -1)^T$	4	$\mathbf{x}^*$	0.4999995290985030	0.0032445575434213	-0.5235177260834855
		$\mathbf{F}(\mathbf{x}^*)$	2.9887923803e-08	-9.2239551116e-05	2.8232861198e-08
	8	$\mathbf{x}^*$	0.5000000000000012	-0.000000000080384	-0.5235987755984997
		$\mathbf{F}(\mathbf{x}^*)$	3.5527136788e-15	-1.6071820808e-11	3.3688103320e-15
	10	$\mathbf{x}^*$	0.4999995325239298	0.0031990647749698	-0.5235188629820862
		$\mathbf{F}(\mathbf{x}^*)$	1.1102230246e-16	-5.2909066017e-17	-1.8390334675e-16

第 10 章 实验内容参考答案

1. 已知矩阵 $\mathbf{A}$ ，试根据要求求解特征值和对应的特征向量，并可视化观察收敛过程，其中精度要求 $\varepsilon = 10^{-16}$ 。对所求特征值 λ 与特征向量 $\mathbf{x}$ 进行验证，可与库函数 `np.linalg.eig(A)` 所求结果对比，或采用 $\|(\mathbf{A} - \lambda \mathbf{I})\mathbf{x}\|_2$ 度量。

$$\mathbf{A} = \begin{pmatrix} 17 & 0 & 1 & 0 & 15 \\ 23 & 5 & 7 & 14 & 16 \\ 4 & 0 & 13 & 0 & 22 \\ 10 & 12 & 19 & 21 & 3 \\ 11 & 18 & 25 & 2 & 19 \end{pmatrix}.$$

(1) 初值向量 $\mathbf{v}^{(0)} = (0.1, 0.1, 0.1, 0.1, 0.1)^T$ ，用幂法和反幂法分别求解矩阵 $\mathbf{A}$ 按模最大和按模最小特征值以及对应的特征向量。

(2) 用原点平移反幂法求解矩阵 $\mathbf{A}$ 在初始 μ_0 分别为 50、18、12、2.2 和 -14 时的特征值和对应的特征向量。
求解代码如下：

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_10_exp1.py
"""

import numpy as np
from matrix_eigenvalue_calculation_10.utils.mat_eig_utils import MatrixEigenvalueUtils
from matrix_eigenvalue_calculation_10.power_method_eig import PowerMethodMatrixEig
from matrix_eigenvalue_calculation_10.origin_translation_inverse_power import OriginTranslationInversePower

A = np.array([[17, 0, 1, 0, 15], [23, 5, 7, 14, 16], [4, 0, 13, 0, 22], [10, 12, 19, 21, 3], [11, 18, 25, 2, 19]])
v0 = 0.1 * np.ones(5)

# 乘幂法
pmme = PowerMethodMatrixEig(A, v0=v0, eps=1e-16)
eig, vector = pmme.fit_eig()
utils = MatrixEigenvalueUtils(pmme.iter_eigenvalue, pmme.iter_eig_vector)
utils.show_iteration()
utils.plt_matrix_eig("乘幂法")
err = np.dot((A - eig * np.eye(5)), vector)
print(np.linalg.norm(err))

eig_vector = np.linalg.eig(A) # 库函数求解
idx = np.argmax(abs(eig_vector[0])) # 最大特征值索引
max_eig = eig_vector[0][idx]
max_vector = eig_vector[1][:, idx]
print(max_eig)
print(max_vector / max_vector[np.argmax(max_vector)])
print("=" * 80)
```

```

# 反幂法
pmme = PowerMethodMatrixEig(A, v0=v0, eps=1e-16, eig_type="inverse")
eig, vector = pmme.fit_eig()
utils = MatrixEigenvalueUtils(pmme.iter_eigenvalue, pmme.iter_eig_vector)
utils.show_iteration()
utils.plt_matrix_eig("反幂法")
err = np.dot((A - eig * np.eye(5)), vector)
print(np.linalg.norm(err))

eig_vector = np.linalg.eig(A) # 库函数求解
idx = np.argmin(abs(eig_vector[0])) # 最小特征值索引
min_eig = eig_vector[0][idx]
min_vector = eig_vector[1][:, idx]
print(min_eig)
print(min_vector / min_vector[np.argmax(min_vector)])
print("=" * 80)

# 原点平移反幂法
mius = [50, 18, 12, 2.2, -14]
for miu in mius:
    otip = OriginTranslationInversePower(A, u0=v0, miu=miu, eps=1e-16)
    otip.fit_eig()
    print("特征值: ", otip.iter_eigenvalue[-1])
    print("特征向量", otip.eig_vector / np.max(np.abs(otip.eig_vector)))
eig_vector = np.linalg.eig(A)
for i in range(5):
    print("库函数所求特征值 %d: %.14f" % (i + 1, eig_vector[0][i]))
    print("特征向量: \n", eig_vector[1][:, i] / np.max(np.abs(eig_vector[1][:, i])))
print("=" * 80)

```

(1) 幂法和反幂法

矩阵的条件数为 48.31169519705382. 乘幂法验证结果为 $\|(\mathbf{A} - \lambda^* \mathbf{I})\mathbf{x}^*\|_2 = 5.95808 \times 10^{-14}$, 反幂法验证结果为 $\|(\mathbf{A} - \lambda^* \mathbf{I})\mathbf{x}^*\|_2 = 3.97205 \times 10^{-15}$. 求解结果如表 10 - 1 所示, 与库函数求解相比, 可见乘幂法与反幂法求解精度较高. 乘幂法与反幂法的收敛次数与收敛过程如图 10 - 1 和图 10 - 2 所示.

表 10 - 1 乘幂法与反幂法所求特征值与特征向量

方法	特征值	特征向量 (从左到右构成向量)				
乘幂法	54.493668784322011	0.41527636	0.83325757	0.57023411	0.83556993	1.0
np.linalg.eig(A)	54.493668784322075	0.41527636	0.83325757	0.57023411	0.83556993	1.0
反幂法	2.3352328658399304	0.52535582	-1.11859308	1.0	-0.48699226	-0.58028138
np.linalg.eig(A)	2.3352328658399424	0.52535582	-1.11859308	1.0	-0.48699226	-0.58028138

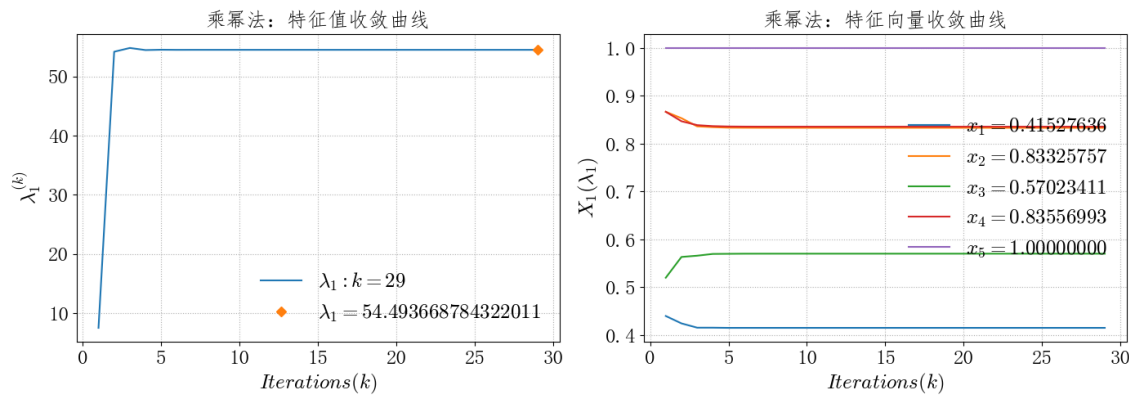


图 10 - 1 乘幂法求按模最大特征值与其对应的特征向量收敛曲线

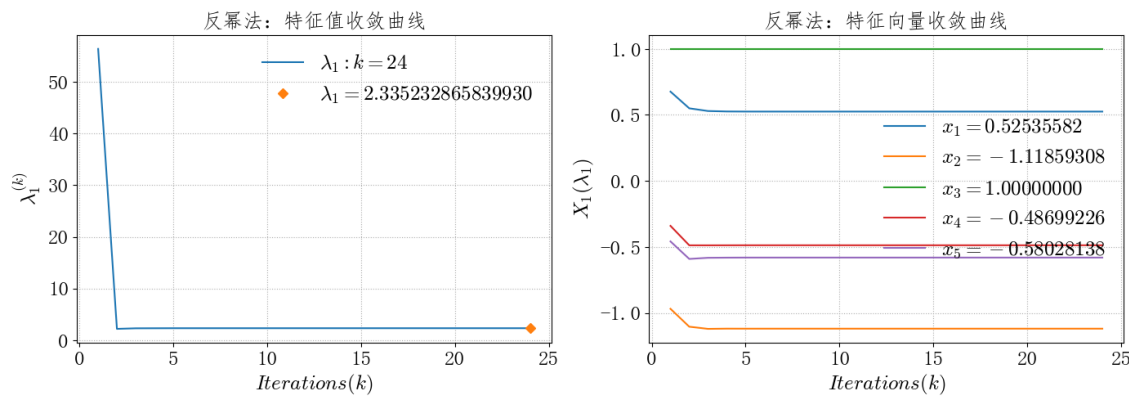


图 10 - 2 反幂法求按模最小特征值与其对应的特征向量收敛曲线

(2) 原点平移反幂法

原点平移反幂法所求矩阵特征值与迭代次数如表 10 - 2 所示，与库函数求解比较，精度非常高. 对应特征向量如表 10 - 3 所示，与库函数相比，在保留小数点后 8 位情况下一致，差别在于特征向量整体的负号，可在特征向量归一化时，消除正负符号，使得最大值均为 1.0.

表 10 - 2 原点平移反幂法求解矩阵特征值

初值	迭代次数	原点平移反幂法	np.linalg.eig(A)
50	17	54.49366878432201	54.49366878432208
18	14	18.293549850778476	18.29354985077848
12	20	12.925792509295027	12.92579250929503
2.2	9	2.335232865839933	2.33523286583994
-14	15	-13.048244010235445	-13.04824401023546

表 10 - 3 原点平移反幂法求解矩阵特征值所对应的特征向量

特征值	特征向量 (从左到右构成向量)				
54.49366878432201	0.41527636	0.83325757	0.57023411	0.83556993	1.0
18.293549850778476	-0.28767559	0.41355058	-0.25095089	1.0	-0.00807812
12.925792509295027	0.70486263	0.9949287	-1.0	0.04787845	-0.12478377
2.335232865839933	0.46965767	-1.0	0.89398014	-0.43536141	-0.51876003
-13.048244010235445	0.4735094	0.49182496	0.771874	-0.65502936	-1.0

2. 已知对称矩阵 $\mathbf{A}$, 试根据要求求解矩阵的特征值, 并可视化观察收敛过程, 其中精度要求 $\varepsilon = 10^{-16}$.

$$\mathbf{A} = \begin{pmatrix} 1 & 1/2 & 1/3 & 1/4 & 1/5 & 1/6 \\ 1/2 & 1 & 2/3 & 1/2 & 2/5 & 1/3 \\ 1/3 & 2/3 & 1 & 3/4 & 3/5 & 1/2 \\ 1/4 & 1/2 & 3/4 & 1 & 4/5 & 2/3 \\ 1/5 & 2/5 & 3/5 & 4/5 & 1 & 5/6 \\ 1/6 & 1/3 & 1/2 & 2/3 & 5/6 & 1 \end{pmatrix}.$$

- (1) 用瑞利商加速幂法求解矩阵 $\mathbf{A}$ 的特征值和特征向量.
- (2) 对矩阵 $\mathbf{A}$ 进行施密特正交化分解 QR 法, 求解矩阵的全部特征值.
- (3) 用正交相似变换约化一般矩阵为上海森伯格矩阵, 记为 $\mathbf{H}_\mathbf{A}$.
- (4) 对于 $\mathbf{H}_\mathbf{A}$, 采用 Schmidt、Givens、Householder 三种 QR 正交分解法求解矩阵的全部特征值.
- (5) 对于 $\mathbf{H}_\mathbf{A}$, 采用位移 Schmidt、位移 Givens 和位移 Householder 三种 QR 正交分解法求解矩阵的全部特征值.

求解代码如下:

```
# -*- coding: UTF-8 -*-
"""
@file_name: chepter_10_exp2.py
"""

import numpy as np
from matrix_eigenvalue_calculation_10.utils.mat_eig_utils import MatrixEigenvalueUtils
from matrix_eigenvalue_calculation_10.power_method_eig import PowerMethodMatrixEig
from matrix_eigenvalue_calculation_10.rayleigh_quotient_accelerated_power import
RayleighQuotientAcceleratedPower
from matrix_eigenvalue_calculation_10.heisenberg_qr_matrix_eigs import HeisenbergQRMatrixEig
from matrix_eigenvalue_calculation_10.qr_orthogonal_matrix_eigs import QROrthogonalMatrixEigenvalues
from matrix_eigenvalue_calculation_10.displacement_heisenberg_qr_matrix_eigs import
DisplacementHeisenbergQRMatrixEig

A = np.array([[1, 1/2, 1/3, 1/4, 1/5, 1/6], [1/2, 1, 2/3, 1/2, 2/5, 1/3],
              [1/3, 2/3, 1, 3/4, 3/5, 1/2], [1/4, 1/2, 3/4, 1, 4/5, 2/3],
              [1/5, 2/5, 3/5, 4/5, 1, 5/6], [1/6, 1/3, 1/2, 2/3, 5/6, 1]])
v0 = 0.01 * np.ones(6)

# 瑞利商加速幂法
rqap = RayleighQuotientAcceleratedPower(A, u0=v0, eps=1e-16)
eig, eig_vector = rqap.fit_eig()
utils = MatrixEigenvalueUtils(rqap.iter_eigenvalue, rqap.iter_eig_vector)
utils.show_iteration()
utils.plt_matrix_eig("瑞利商加速幂法")
print("归一化后特征向量:", eig_vector)
err = np.dot((A - eig * np.eye(6)), eig_vector)
print(np.linalg.norm(err))
```

```

# 乘幂法与瑞利商加速幂法对比
# pmme = PowerMethodMatrixEig(A, v0=v0, eps=1e-16)
# eig, vector = pmme.fit_eig()
# utils = MatrixEigenvalueUtils(pmme.iter_eigenvalue, pmme.iter_eig_vector)
# utils.show_iteration()
# utils.plt_matrix_eig("乘幂法")
# err = np.dot((A - eig * np.eye(6)), vector)
# print(np.linalg.norm(err))

# 施密特正交化分解 QR 法求解矩阵的全部特征值
qrme = QROrthogonalMatrixEigenvalues(A, eps=1e-16)
qrme.fit_eig()
qrme.plt_eigenvalues()
print("施密特正交化全部特征值: ", qrme.eigenvalues)
eig_vector = np.linalg.eig(A)
print("库函数: ")
for i in range(6):
    print("%.16f" % eig_vector[0][i], end=" ")
print("\n", "=" * 80)

# 采用 Schmidt、Givens、Householder 三种 QR 正交分解法求解矩阵的全部特征值
methods = ["schmidt", "Givens", "householder"]
for mt in methods:
    hqr = HeisenbergQRMatrixEig(A, eps=1e-16, transform=mt)
    hqr.fit_eig()
    print(mt, hqr.eigenvalues)
    hqr.plt_eigenvalues()

# 采用位移 Schmidt、位移 Givens 和位移 Householder 三种 QR 正交分解法求解矩阵的全部特征值.
methods = ["schmidt", "Givens", "householder"]
for mt in methods:
    dhqr = DisplacementHeisenbergQRMatrixEig(A, eps=1e-16, transform=mt)
    dhqr.fit_eig()
    print(mt, dhqr.eigenvalues)
    dhqr.plt_eigenvalues()

```

(1) 瑞利商加速幂法

矩阵的条件数为 29.040428650875786。所求特征值 $\lambda^* = 3.60121153530174531454$ ，归一化后的特征向量为

$$\mathbf{x}^* = (0.50188757, 0.79472568, 0.95011683, 1.0, 0.97066621, 0.88388432)^T.$$

验证结果为 $\|(\mathbf{A} - \lambda^* \mathbf{I})\mathbf{x}^*\|_2 = 3.46408 \times 10^{-9}$ 。如图 10 - 3 所示。而采用幂法求解，需迭代 31 次，故针对对称矩阵，瑞利商加速幂法可提高迭代收敛的速度。

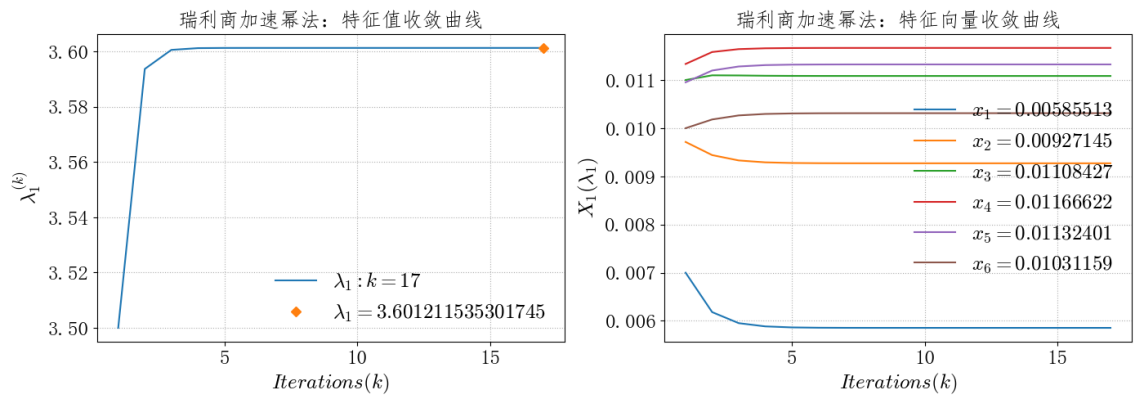


图 10-3 瑞利商加速幂法求特征值与其对应的特征向量收敛曲线 (未归一化)

(2) 施密特正交化分解 QR 法

表 10-4 施密特正交化分解 QR 法求矩阵全部特征值

方法	λ_1^*	λ_2^*	λ_2^*
施密特正交化分解 QR 法	3.6012115353017418	1.156908189230253	0.586913699334691
np.linalg.eig(A)	3.6012115353017444	1.156908189230253	0.586913699334691
方法	λ_4^*	λ_5^*	λ_6^*
施密特正交化分解 QR 法	0.331438482048264	0.1995212628293681	0.1240068312556790
np.linalg.eig(A)	0.331438482048264	0.1995212628293680	0.1240068312556791

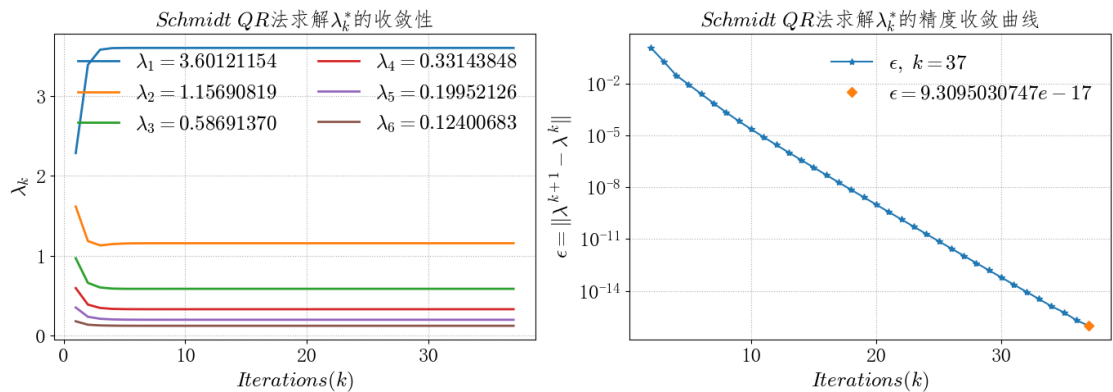


图 10-4 施密特正交化分解 QR 法求解矩阵全部特征值及其精度收敛曲线

(3) 用正交相似变换约化一般矩阵为上海森伯格矩阵.

所求上海森伯格矩阵如下, 由于舍入误差的存在, 某些元素只保留了数量级以及符号.

$$\mathbf{H}_A = \begin{pmatrix} 1.00000000 & -0.70099136 & -10^{-17} & 10^{-17} & 10^{-17} & 10^{-17} \\ -0.70099136 & 2.90163934 & 1.07279015 & 10^{-17} & -10^{-17} & -10^{-17} \\ -10^{-17} & 1.07279015 & 1.33397209 & 0.20747302 & -10^{-17} & -10^{-17} \\ 10^{-17} & -10^{-17} & 0.20747302 & 0.42391411 & 0.05072821 & -10^{-17} \\ 10^{-17} & -10^{-17} & 10^{-17} & 0.05072821 & 0.21476735 & -0.01165770 \\ 10^{-17} & 10^{-17} & -10^{-18} & -10^{-17} & -0.01165770 & 0.12570710 \end{pmatrix}.$$

(4) 对于 $\mathbf{H}_A$, 采用 Schmidt、Givens、Householder 三种 QR 正交分解法求解矩阵的全部特征值.

具体特征值如表 10-5 所示. 图 10-5、图 10-6 和图 10-7 为全部特征以及精度的收敛曲线.

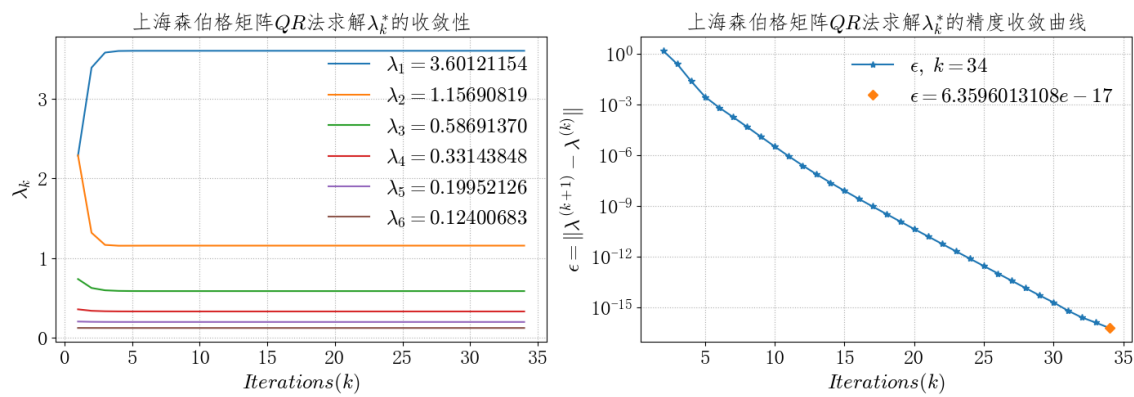


图 10 - 5 针对上海森伯格矩阵采用 Schmidt QR 正交分解法求解矩阵全部特征值以及收敛曲线

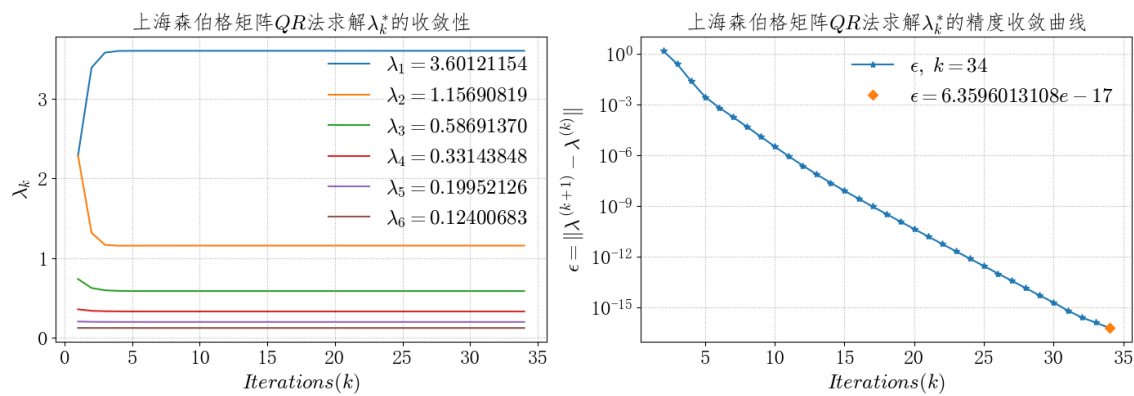


图 10 - 6 针对上海森伯格矩阵采用 Givens QR 正交分解法求解矩阵全部特征值以及收敛曲线

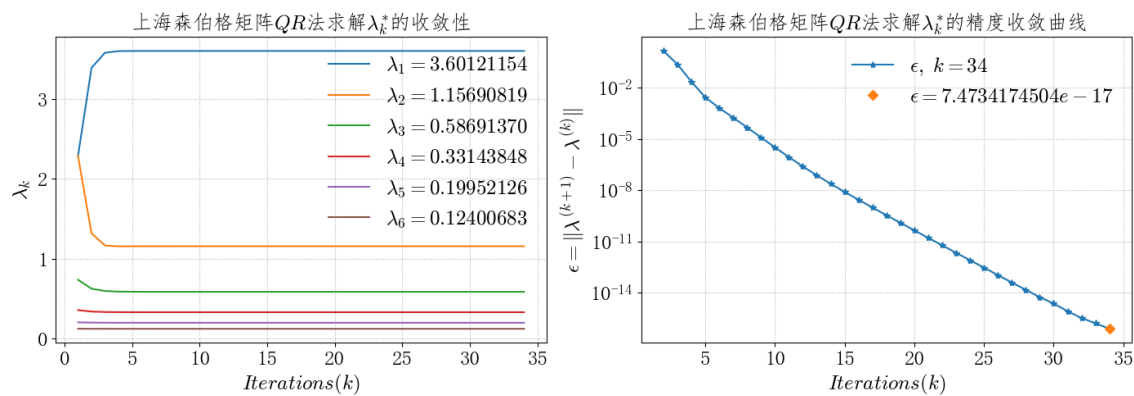


图 10 - 7 针对上海森伯格矩阵采用 Householder QR 正交分解法求解矩阵全部特征值以及收敛曲线

表 10 - 5 三种 QR 正交分解法求解矩阵全部特征值结果

schmidt	迭代 34 次	λ_1^*	λ_2^*	λ_2^*
		3.6012115353017466	1.1569081892302537	0.5869136993346906
		λ_4^*	λ_5^*	λ_6^*
		0.33143848204826487	0.1995212628293676	0.12400683125567899
Givens	迭代 34 次	λ_1^*	λ_2^*	λ_2^*
		3.6012115353017444	1.156908189230253	0.586913699334691
		λ_4^*	λ_5^*	λ_6^*
		0.3314384820482637	0.1995212628293674	0.12400683125567896

householder	迭代 34 次	λ_1^*	λ_2^*	λ_3^*
		3.601211535017435	1.1569081892302502	0.5869136993346904
		λ_4^*	λ_5^*	λ_6^*
		0.33143848204826404	0.1995212628293678	0.1240068312556792

(5) 对于 $\mathbf{H}_A$ ，采用位移 Schmidt、位移 Givens 和位移 Householder 三种 QR 正交分解法求解矩阵的全部特征值. 具体特征值数值可通过算法打印输出，略去，保留小数点后 8 位，可参考图 10 - 8、图 10 - 9 和图 10 - 10 图例，三种方法所求一致.

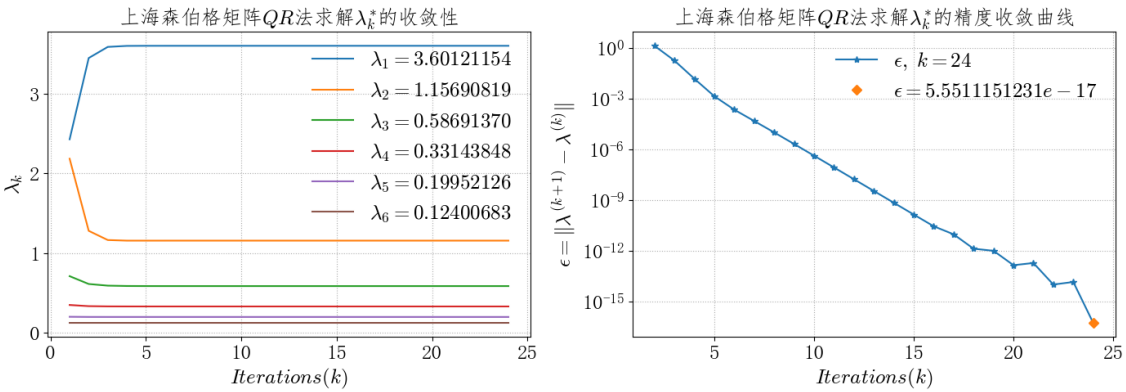


图 10 - 8 针对上海森伯格矩阵采用位移 Schmidt QR 正交分解法求解矩阵全部特征值以及收敛曲线

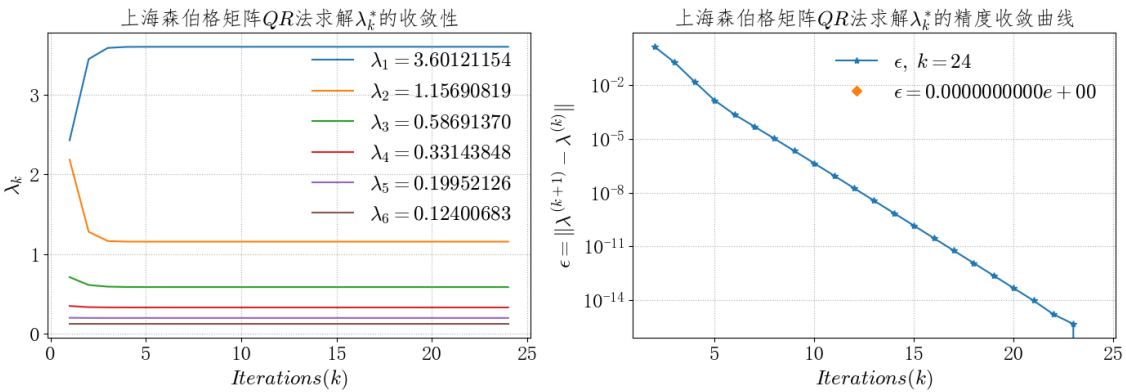


图 10 - 9 针对上海森伯格矩阵采用位移 Givens QR 正交分解法求解矩阵全部特征值以及收敛曲线

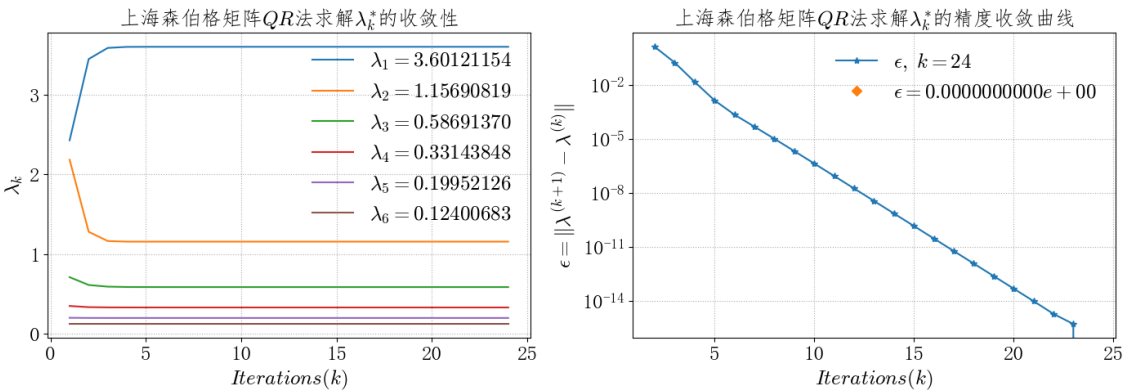


图 10 - 10 针对上海森伯格矩阵采用位移 Householder QR 正交分解法求解矩阵全部特征值以及收敛曲线

第 11 章 实验内容参考答案

1. 采用梯形欧拉法和改进的欧拉法求解如下一阶微分方程, 已知解析解 $y(x) = e^{-x} + x$.

$$y' = -y + x + 1, y(0) = 1, 0 \leq x \leq 1.$$

并求解在给定点 $[0.045, 0.400, 0.487, 0.685, 0.778, 0.923]$ 的微分方程数值解. 设定步长分别为 $h = 0.05$ 和 $h = 0.0001$, 基于解析解分析数值解的误差, 可视化误差曲线和数值解曲线.

求解代码如下:

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_11_exp1.py
"""

import numpy as np
from ode_numerical_solution_11.ode_euler_method import ODEEulerMethod
from ode_numerical_solution_11.utils.ode_utils import ODESolUtils
from Experiment.util_font import *

def ode_fun(x, y):
    """
    微分方程定义
    """
    return -y + x + 1

analytic_ft = lambda x: np.exp(-x) + x # 解析解

ode_method = ["Trapezoid", "PC"]
methods = ["Trapezoid", "PC \ System"]
plt.figure(figsize=(14, 5))
plt.subplot(121)
h = 0.05 # 步长, 此处修改即可
xi = np.arange(0, 1 + h, h)
yi = analytic_ft(xi) # 精确解
x0 = np.array([0.045, 0.4, 0.487, 0.685, 0.778, 0.923]) # 所求任意点的解
y0 = np.zeros((len(x0), 5))
line_style = ["--", "-."]
precision = np.zeros((len(xi), 2))
for i, method in enumerate(ode_method):
    euler = ODEEulerMethod(ode_fun, x0=0, y0=1, x_final=1, h=h, ode_method=method)
    sol = euler.fit_ode()
    utils = ODESolUtils(euler, analytic_ft)
    y0[:, i] = utils.predict_x0(x0) # 任意点的解
    precision[:, i] = yi - sol[:, 1] # 精度误差
    plt.plot(sol[:, 0], sol[:, 1], line_style[i], lw=1.5, label="$%s \ \hat{y}(x)$" % methods[i])
plt.plot(xi, yi, "k-", lw=1.5, label="$Exact \ sol \ y(x)$")
```

```
plt.xlabel("$x$", fontdict={"fontsize": 18})
plt.ylabel("$y(x) \ / \ \hat{y}(x)$", fontdict={"fontsize": 18})
plt.title("欧拉法求解$ODE$初值问题数值解曲线", fontdict={"fontsize": 18})
plt.legend(frameon=False, fontsize=16, loc="best")
plt.grid(ls=":")
plt.tick_params(labelsize=18) # 刻度字体大小 16

# 绘制与解析解对比的精度曲线
plt.subplot(122)
for i in range(2):
    error = np.linalg.norm(precision[:, i])
    label_txt = methods[i] + ", \epsilon = %.2e" % error
    plt.plot(xi, precision[:, i], line_style[i], lw=1.5, label="$%s$" % label_txt)
plt.xlabel("$x$", fontdict={"fontsize": 18})
plt.ylabel("$err = y_k - \hat{y}_k$", fontdict={"fontsize": 18})
plt.title("$ODE$初值问题数值解的误差曲线 $epsilon = \Vert y - \hat{y} \Vert$", fontdict={"fontsize": 18})
plt.legend(frameon=False, fontsize=16, loc="best")
plt.grid(ls=":")
plt.tick_params(labelsize=18) # 刻度字体大小 18
plt.show()

# 输出区间任意点的数值解
y_true = analytic_ft(x0) # 精确解
for i in range(2):
    print(ode_method[i], end=": ")
    print(y0[:, i])
    print(y_true - y0[:, i])
```

(1) 当步长 $h = 0.05$ 时, 梯形公式法与改进的欧拉法求解微分方程初值问题数值解曲线有误差曲线如图 11 - 1 所示, 由于精度较高, 左图不易区分, 右图为误差曲线, 可知具有一定的精度. 具体所求时刻的数值解以及绝对值误差如表 11 - 1 所示.

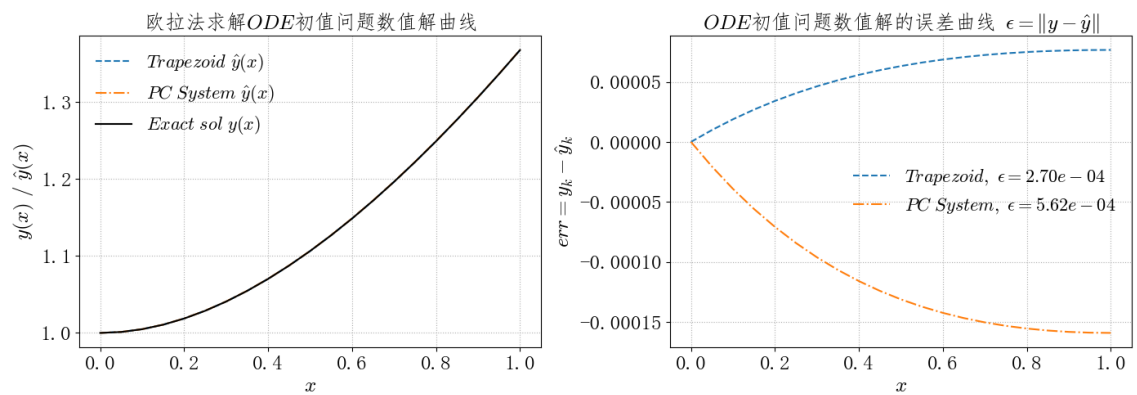


图 11 - 1 梯形公式法与改进的欧拉法求解 ODE 初值问题数值解曲线有误差曲线 ($h = 0.05$)

表 11 - 1 梯形公式法与改进的欧拉法求解 ODE 初值问题数值解以及绝对值误差($h = 0.05, k = 1, 2, \dots, 6$)

x_k	真值	梯形公式法	绝对值误差	改进的欧拉法	绝对值误差
-------	----	-------	-------	--------	-------

0.045	1.00099748	1.00099584	1.64602108e-06	1.0010234	2.59201200e-05
0.400	1.07032005	1.07026417	5.58786315e-05	1.07043605	1.16003256e-04
0.487	1.10146703	1.10145703	1.00027162e-05	1.10164881	1.81779464e-04
0.685	1.18909023	1.1890624	2.78246184e-05	1.18928373	1.93500396e-04
0.778	1.23732374	1.23728627	3.74732224e-05	1.23751533	1.91588819e-04
0.923	1.32032528	1.32027484	5.04317429e-05	1.32050993	1.84656545e-04

(2) 当步长 $h = 0.0001$ 时, 如图 11 - 2 所示, 可见数值解的精度非常高.

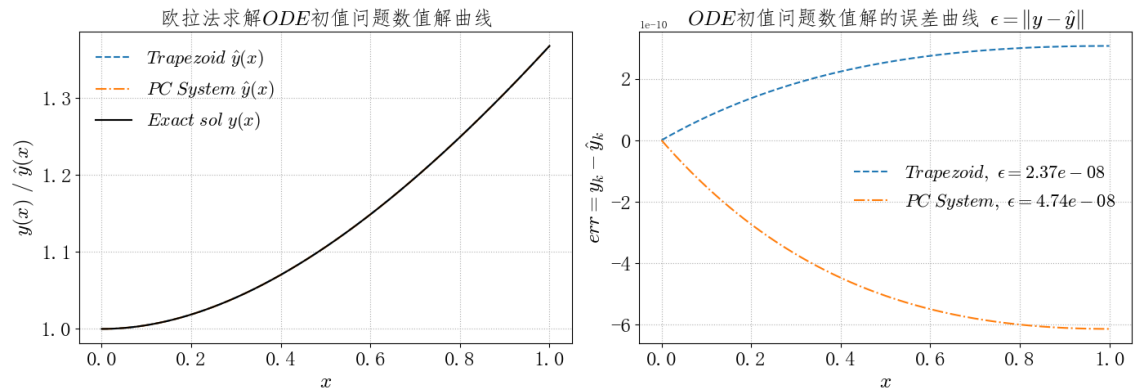


图 11 - 2 梯形公式法与改进的欧拉法求解 ODE 初值问题数值解曲线有误差曲线 ($h = 0.0001$)

表 11 - 2 梯形公式法与改进的欧拉法求解 ODE 初值问题数值解以及绝对值误差($h = 0.0001, k = 1, 2, \dots, 6$)

x_k	真值	梯形公式法	绝对值误差	改进的欧拉法	绝对值误差
0.045	1.00099748	1.00099738	9.88263000e-08	1.00099747	3.57335432e-08
0.400	1.07032005	1.07032005	2.23438379e-10	1.07032005	4.46915172e-10
0.487	1.10146703	1.10144667	5.21009724e-05	1.10150584	1.84840232e-05
0.685	1.18909023	1.18909023	2.87745383e-10	1.18909023	5.75550940e-10
0.778	1.23732374	1.23732374	2.97787572e-10	1.23732374	5.95637317e-10
0.923	1.32032528	1.32032528	3.05603320e-10	1.32032528	6.11267925e-10

2. 采用标准的龙格—库塔法以及芬尔格法求解如下一阶微分方程, 已知解析解 $y(x) = -e^{-x} + x^2 - x + 1$.

$$y' = x^2 + x - y, y(0) = 0, 0 \leq x \leq 1.$$

并求解在给定点 $[0.045, 0.400, 0.487, 0.685, 0.778, 0.923]$ 的微分方程数值解. 设定步长分别为 $h = 0.05$ 和 $h = 0.01$, 基于解析解分析数值解的误差, 可视化误差曲线和数值解曲线.

求解代码如下:

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_11_exp2.py
"""
import numpy as np
from ode_numerical_solution_11.ode_runge_kutta_method import ODERungeKuttaMethod
from ode_numerical_solution_11.utils.ode_utils import ODESolUtils
from Experiment.util_font import *
```



```

def ode_fun(x, y):
    return x ** 2 + x - y

analytic_ft = lambda x: -np.exp(-x) + x ** 2 - x + 1

ode_method = ["rk", "rkf"]
plt.figure(figsize=(14, 5))
plt.subplot(121)
h = 0.05 # 此处修改步长即可
xi = np.arange(0, 1 + h, h)
yi = analytic_ft(xi) # 精确解
x0 = np.array([0.045, 0.4, 0.487, 0.685, 0.778, 0.923]) # 所求任意点的解
y0 = np.zeros((len(x0), 2))
line_style = ["--", "-."]
precision = np.zeros((len(xi), 2))
plt.plot(xi, yi, "k-", lw=1.5, label="$Exact \ sol \ y(x)$")
for i, method in enumerate(ode_method):
    rk = ODERungeKuttaMethod(ode_fun, x0=0, y0=0, x_final=1, h=h, rk_type=method)
    sol = rk.fit_ode()
    utils = ODESolUtils(rk, analytic_ft)
    y0[:, i] = utils.predict_x0(x0) # 任意点的解
    precision[:, i] = yi - sol[:, 1] # 精度误差
    plt.plot(sol[:, 0], sol[:, 1], line_style[i], lw=1.5, label="$%s \ \hat y(x)$" % method)
plt.xlabel("$x$", fontdict={"fontsize": 18})
plt.ylabel("$y(x) \ / \ \hat y(x)$", fontdict={"fontsize": 18})
plt.title("龙格库塔法求解$ODE$初值问题数值解曲线", fontdict={"fontsize": 18})
plt.legend(frameon=False, fontsize=18)
plt.grid(ls=":")
plt.tick_params(labelsize=18) # 刻度字体大小 16

# 绘制与解析解对比的精度曲线
plt.subplot(122)
for i in range(2):
    error = np.linalg.norm(precision[:, i])
    label_txt = ode_method[i] + ", \ \epsilon= %.5e" % error
    plt.plot(xi, precision[:, i], line_style[i], lw=1.5, label="$%s$" % label_txt)
plt.xlabel("$x$", fontdict={"fontsize": 18})
plt.ylabel("$err = y_k - \hat y_k$", fontdict={"fontsize": 18})
plt.title("$ODE$初值问题数值解的误差曲线 $\\epsilon = \\Vert y - \\hat{y} \\Vert$", fontdict={"fontsize": 18})
plt.legend(frameon=False, fontsize=18)
plt.grid(ls=":")
plt.tick_params(labelsize=18) # 刻度字体大小 18
plt.show()

# 输出区间任意点的数值解
y_true = analytic_ft(x0) # 精确解
for i in range(2):

```

```
print(ode_method[i], end=": ")
print(y0[:, i])
print(y_true - y0[:, i])
print("-" * 70)
```

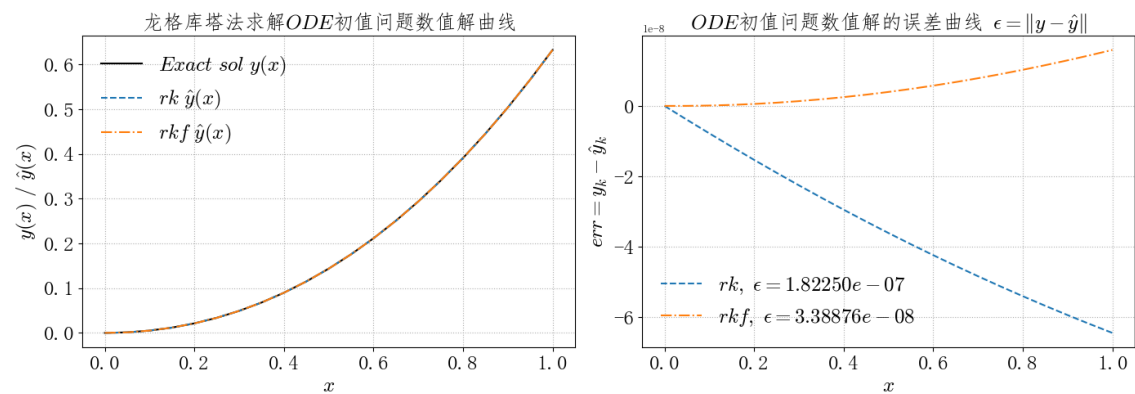


图 11-3 龙格—库塔法和芬尔格法求解 ODE 初值问题的数值解以及误差曲线 ($h = 0.05$)

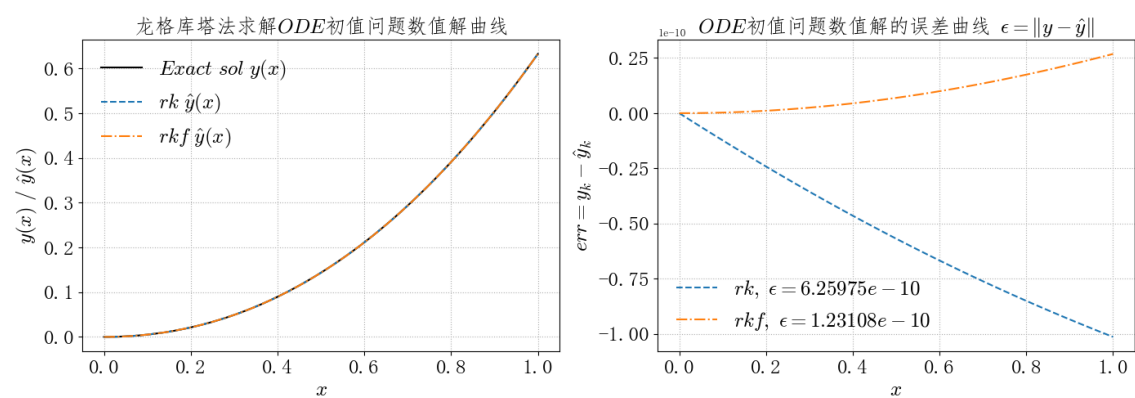


图 11-4 龙格—库塔法和芬尔格法求解 ODE 初值问题的数值解以及误差曲线 ($h = 0.01$)

表 11-3 龙格—库塔法与龙格—库塔—芬尔格法求解 ODE 的数值解以及绝对值误差

x_k	真值	龙格—库塔法	绝对值误差	龙格—库塔—芬尔格法	绝对值误差
0.045	0.00102752	0.00102752	3.06358179e-11	0.00102752	2.47435657e-11
0.400	0.08967995	0.08967995	4.64963207e-11	0.08967995	4.44692894e-12
0.487	0.13570197	0.13570197	6.63309685e-11	0.13570197	5.28707633e-12
0.685	0.28013477	0.28013477	5.03591613e-12	0.28013477	3.91870425e-11
0.778	0.36796026	0.36796026	7.21277482e-11	0.36796026	5.49854051e-11
0.923	0.53160372	0.53160372	4.87868079e-10	0.53160372	2.16091256e-10

3. 用各种线性多步法求解如下二阶微分方程，已知解析解 $y(x)=\frac{\ln x+1}{x}$.

$$y' = \frac{1}{x^2} - \frac{y}{x}, y(1) = 1, 1 \leq x \leq 2.$$

设定步长分别为 $h = 0.01$ 和 $h = 0.001$ ，并对数值解进行可视化，分析各方法的误差。

求解代码如下：

```
# -*- coding: UTF-8 -*-
```

```

"""
@file_name: chapter_11_exp3.py
"""

import numpy as np
from ode_numerical_solution_11.linear_multi_step_method import LinearMultiStepMethod
from Experiment.util_font import *

ode_fun = lambda x, y: 1 / x ** 2 - y / x
analytic_ft = lambda x: (np.log(x) + 1) / x

plt.figure(figsize=(14, 5))
x0, y0, h_s = 1, 1, [0.01, 0.001]
for k, h in enumerate(h_s):
    xi = np.arange(1, 2 + h, h)
    sol_y = np.zeros((len(xi), 6))
    sol_y[:, 0] = analytic_ft(xi) # 解析解
    precision = np.zeros((len(xi), 5))
    ode_methods = ["e_admas", "i_admas", "milne", "simpson", "hamming"]
    for idx, method in enumerate(ode_methods):
        ode_obj = LinearMultiStepMethod(ode_fun, x0, y0, x_final=2, h=h, ode_method=method)
        sol = ode_obj.fit_ode()
        sol_y[:, idx + 1] = sol[:, 1]
        precision[:, idx] = np.abs(sol_y[:, 0] - sol[:, 1])

# 绘制与解析解对比的精度曲线
plt.subplot(121 + k)
line_style = ["-o", "-*", "-+", "-p", "-s"]
labels = ["Adams E", "Adams I", "Milne E", "Simpson I", "Hamming I"]
for i in range(5):
    error_norm = np.linalg.norm(precision[:, i])
    label_txt = labels[i] + ", \epsilon= %.2e" % error_norm
    if h == 0.05:
        plt.semilogy(xi, np.abs(precision[:, i]), line_style[i], lw=1.5, label="%%s" % label_txt)
    else:
        plt.semilogy(xi, np.abs(precision[:, i]), lw=1.5, label="%%s" % label_txt)
plt.xlabel("$x$", fontdict={"fontsize": 18})
plt.ylabel(r"$\text{err} = \text{vert } y_k - \hat{y}_k \text{ vert}$", fontdict={"fontsize": 18})
plt.title("线性多步法$ODE$数值解误差曲线 $h=0.1e$" % h, fontdict={"fontsize": 18})
plt.legend(frameon=False, fontsize=16)
plt.tick_params(labelsize=18) # 刻度字体大小 18
plt.grid(ls=":")
plt.show()

```

五种线性多步法在步长下的数值解误差曲线如图 11-5 所示，整体的精度如图例所示。相对来说，隐式 2 步 4 阶辛普森辛普森法具有不错的精度。

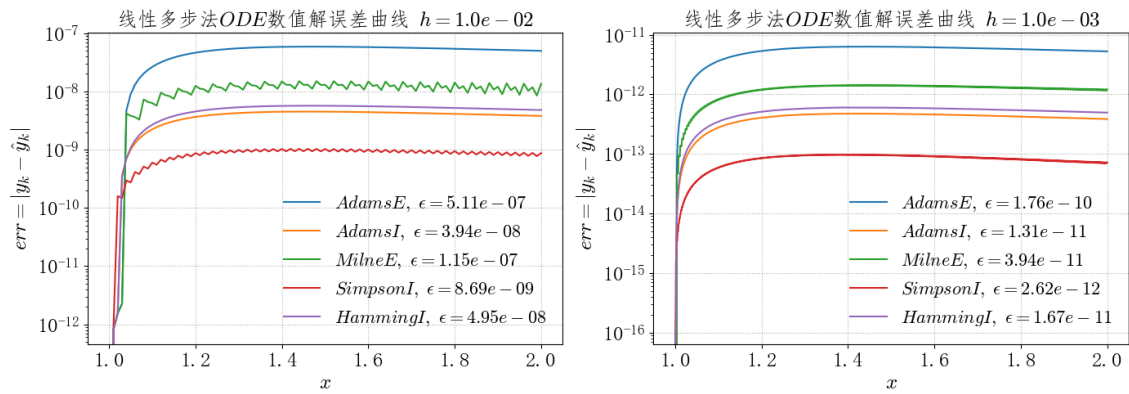


图 11-5 五种线性多步法在不同步长下的数值解误差曲线

4. 洛伦兹方程是描述空气流体运动的一个简化微分方程组，方程描述了三维空间中的一个无质量点 (x, y, z) 的各轴坐标相对于时间 t 的速度矢量。其形式如下：

$$\begin{cases} x'(t) = \sigma(y - x), \\ y'(t) = x(\rho - z) - y, \\ z'(t) = xy - \beta z, \end{cases}$$

其中 $x(t)$ 表示对流的翻动速率， $y(t)$ 正比于上流于下流液体温差， $z(t)$ 是垂直方向的温度梯度。参数 σ, ρ, β 分别为 Prandtl 数、速度阻尼常数和相对瑞利数，假设 $\sigma = 16$ ， $\rho = 45$ ， $\beta = 4$ 。初值 $x(0) = 12$ ， $y(0) = 4$ ， $z(0) = 1$ 。假设步长 $h = 0.001$ ，求解时间 t 范围分别为 $t \in [0, 20]$ 、 $t \in [0, 30]$ 、 $t \in [0, 40]$ 和 $t \in [0, 60]$ 。

求解代码如下：

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_11_exp4.py
"""
import numpy as np
from Experiment.util_font import *
from ode_numerical_solution_11.first_order_ODEs_RK import FirstOrderODEsRK

def ode_funs(t, y):
    """
    微分方程组定义
    :param t: 时间变量
    :param y: 向量, y[0] 表示 x, y[1] 表示 y, y[2] 表示 z
    :return:
    """
    sigma, rou, beta = 16, 45, 4
    return np.array([sigma * (y[1] - y[0]),
                     y[0] * (rou - y[2]) - y[1],
                     y[0] * y[1] - beta * y[2]])

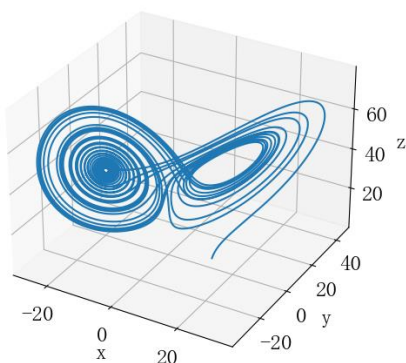
x0, y0, h = 0, np.array([12, 4, 1]), 0.001
fig = plt.figure(figsize=(14, 10))
x_finals = [20, 30, 40, 60]
```

```

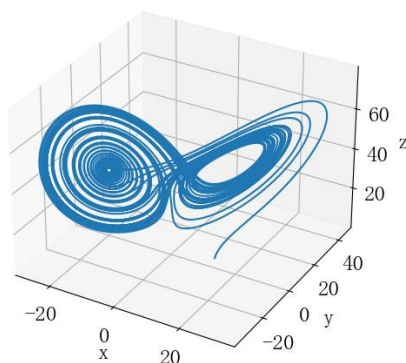
for i, x_f in enumerate(x_finals):
    odes_rk = FirstOrderODEsRK(ode_funs, x0, y0, x_final=x_f, h=h)
    odes_rk.fit_odes()
    # 绘制洛伦兹方程曲线
    ax = fig.add_subplot(221 + i, projection='3d')
    ax.plot(odes_rk.ode_sol[:, 1], odes_rk.ode_sol[:, 2], odes_rk.ode_sol[:, 3])
    ax.set_xlabel("x", fontdict={"fontsize": 18})
    ax.set_ylabel("y", fontdict={"fontsize": 18})
    ax.set_zlabel("z", fontdict={"fontsize": 18})
    plt.title("$Lorenz \ Equation \ by \ Runge \ Kutta \ with \ [0, %d]$" % x_f, fontdict={"fontsize": 18})
    plt.tick_params(labelsize=18) # 刻度字体大小 18
plt.show()

```

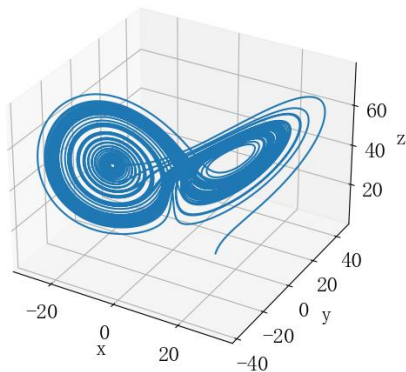
Lorenz Equation by Runge Kutta with [0, 20]



Lorenz Equation by Runge Kutta with [0, 30]



Lorenz Equation by Runge Kutta with [0, 40]



Lorenz Equation by Runge Kutta with [0, 60]

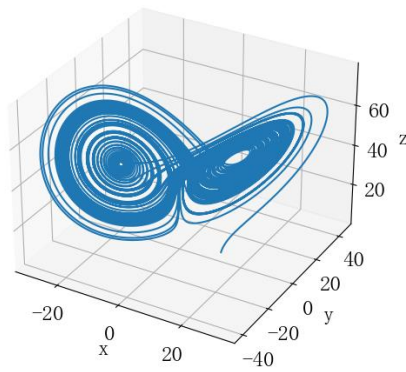


图 11-6 洛伦兹方程方程曲线

5. 采用龙格—库塔法和 GLFIRK 刚性方程组的方法, 求解如下刚性微分方程组.

$$\begin{cases} y'_1 = -0.013y_1 - 1000y_1y_2, \\ y'_2 = -2500y_2y_3, \\ y'_3 = -0.013y_1 - 1000y_1y_2 - 2500y_2y_3, \\ \mathbf{y}(0) = (1, 1, 0)^T. \end{cases}$$

其中 $\mathbf{y}(x) = (y_1(x), y_2(x), y_3(x))^T$. 设置步长 $h = 0.0001$, 求解区间为 $[0, 0.025]$, 观察数值解的收敛性, 并对比两种方法的收敛速度和收敛的精度.

求解代码如下:

```

# -*- coding: UTF-8 -*-
"""
@file_name: chapter_11_exp5.py
"""

import numpy as np
import matplotlib.pyplot as plt
from ode_numerical_solution_11.stiff_ODES_rk_pcs import StiffODEsRKPCS
from ode_numerical_solution_11.first_order_ODEs_RK import FirstOrderODEsRK

def ode_funs(x, y):
    """
    微分方程组定义
    :param x: 自变量
    :param y: y[0] 代表 y1, y[1] 代表 y2, y[2] 代表 y3
    :return:
    """
    dy = np.array([-0.013 * y[0] - 1000 * y[0] * y[1],
                    -2500 * y[1] * y[2],
                    -0.013 * y[0] - 1000 * y[0] * y[1] - 2500 * y[1] * y[2]])
    return dy

x0, y0, h, x_final = 0, np.array([1, 1, 0]), 0.0001, 0.025

plt.figure(figsize=(14, 5))
# 龙格—库塔法
frk = FirstOrderODEsRK(ode_funs, x0, y0, x_final=x_final, h=h)
frk.fit_odes()
plt.subplot(121)
frk.plt_odes_rk(is_show=False)
print(frk.ode_sol[-1, :])

# 3 阶显式龙格 — 库塔公式 + 3 级 6 阶隐式龙格 — 库塔公式构成预测校正系统
s_ode = StiffODEsRKPCS(ode_funs, x0, y0, x_final=x_final, h=h)
s_ode.fit_odes()
plt.subplot(122)
s_ode.plt_odes_rk_pcs(is_show=False)
plt.show()
print(s_ode.ode_sol[-1, :])

```

表 11-4 刚性微分方程组在 $x = 0.005$ 和 $x = 0.025$ 时的数值解

方法 (类名)	x	y_1^*	y_2^*	y_3^*
龙格—库塔法	0.005	1.29729317e-04	1.99978378e+00	-8.64870191e-05
GLFIRK 方法	0.005	3.99176549e-07	1.99999933e+00	2.66120581e-07
龙格—库塔法	0.025	5.51399840e-22	2.00000000e+00	-3.67603876e-22
GLFIRK 方法	0.025	1.85843185e-34	2.00000000e+00	-1.23896799e-34

从表 11 - 4 和图 11 - 7 中可以看出, GLFIRK 方法要比龙格—库塔收敛速度更快, 且最终的收敛精度更高.

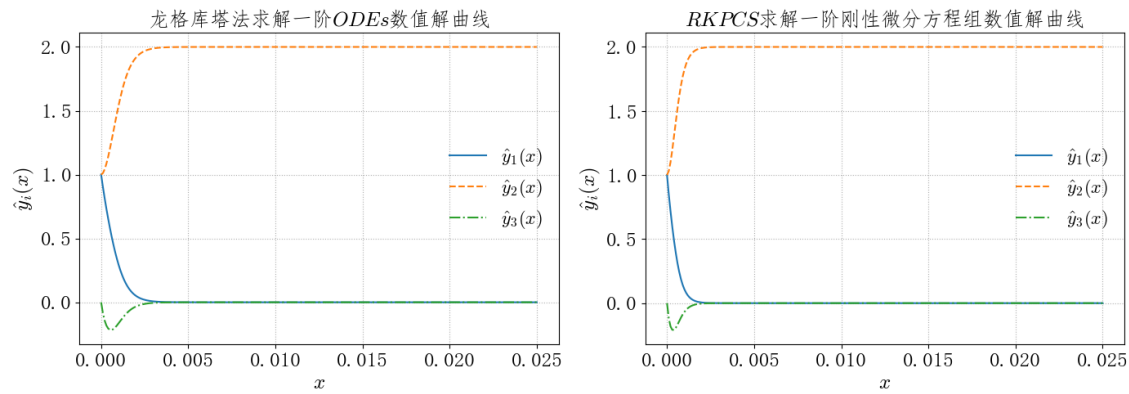


图 11 - 7 刚性微分方程组数值解曲线 ($x \in [0, 0.025]$)

第 12 章 实验内容参考答案

1. 分别采用显式格式和隐式格式求解如下波动方程，解析解为 $u(x, t) = e^{x+t}$ ，设置空间与时间步长为 $h = 0.0005$ 和 $\tau = 0.00005$ 。

$$\begin{cases} u_{tt} - u_{xx} = 0, & 0 < x < 1, & 0 < t \leq 1, \\ u(x, 0) = e^x, & u_t(x, t)|_{t=0} = e^x, & 0 \leq x \leq 1, \\ u(0, t) = e^t, & u(1, t) = e^{1+t}, & 0 < t \leq 1. \end{cases}$$

求解代码如下：

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_12_expl.py
"""
import numpy as np
from partial_differential_equation_12.pde_wave_equation_mixed_boundary import
PDEWaveEquationMixedBoundary
from partial_differential_equation_12.pde_wave_equation_mixed_boundary_implicit \
    import PDEWaveEquationMixedBoundaryImplicit

fun_xt = lambda t, x: 0.0 * t + 0.0 * x # 右端方程
alpha_fun, beta_fun = lambda t: np.exp(t), lambda t: np.exp(1 + t)
u_x0, du_x0 = lambda x: np.exp(x), lambda x: np.exp(x)
pde_model = lambda t, x: np.exp(x + t) # 解析解

x_a, t_T, c, x_h, t_h = 1, 1, 1, 0.0005, 0.0005

# 显式格式
wave = PDEWaveEquationMixedBoundary(fun_xt, alpha_fun, beta_fun, u_x0, du_x0,
                                     x_a, t_T, c, x_h, t_h, pde_model=pde_model)
wave.solve_pde()
wave.plt_pde_wave_surface()
wave.plt_pde_wave_curve_contourf()

# 隐式格式
wave = PDEWaveEquationMixedBoundaryImplicit(fun_xt, alpha_fun, beta_fun, u_x0, du_x0,
                                             x_a, t_T, c, x_h, t_h, pde_model=pde_model)
wave.solve_pde()
wave.plt_pde_wave_surface()
wave.plt_pde_wave_curve_contourf()
```

对于显式格式求解， $r = 1 \leq 1$ ，满足稳定条件，如图 12 - 1 所示，绝对误差均值 $MAE = 1.173 \times 10^{-8}$ ，最大绝对值误差： 3.580×10^{-8} 。对于隐式格式，无条件稳定，如图 12 - 2 所示，绝对误差均值 $MAE = 1.723 \times 10^{-8}$ ，最大绝对值误差 8.875×10^{-8} 。两种方法均具有不错的求解精度。

一维二阶非齐次波动方程数值解曲面

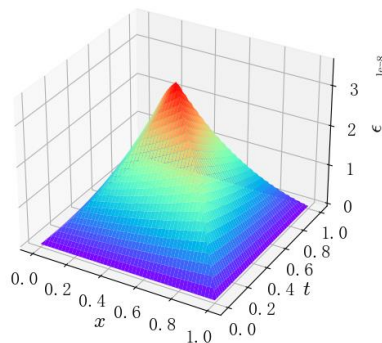
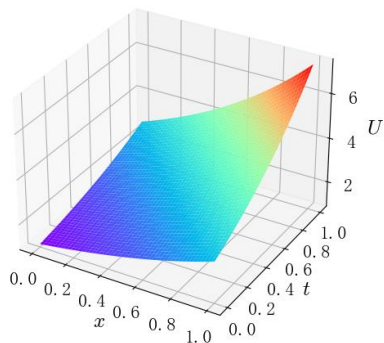
误差曲面 $\epsilon = U(x, t) - \hat{U}(x, t)$, $MAE = 1.173e - 08$ 

图 12-1 显式格式求解波动方程数值解曲面与误差曲面

波动方程隐式差分格式数值解曲面

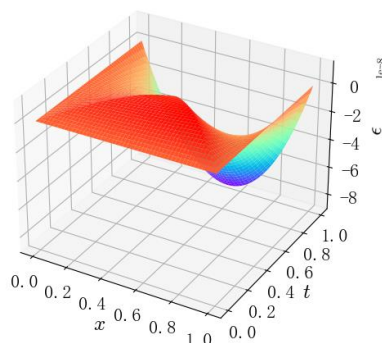
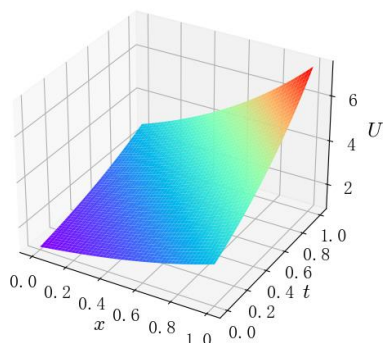
误差曲面 $\epsilon = U(x, t) - \hat{U}(x, t)$, $MAE = 1.723e - 08$ 

图 12-2 隐式格式求解波动方程数值解曲面与误差曲面

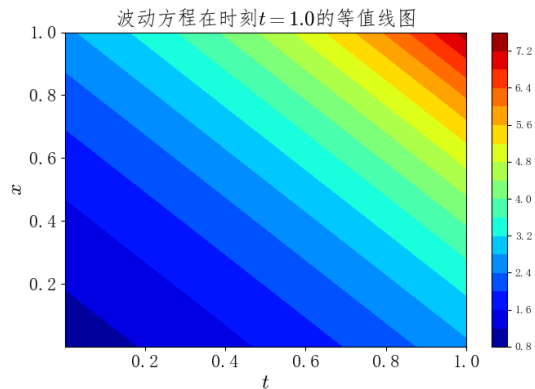
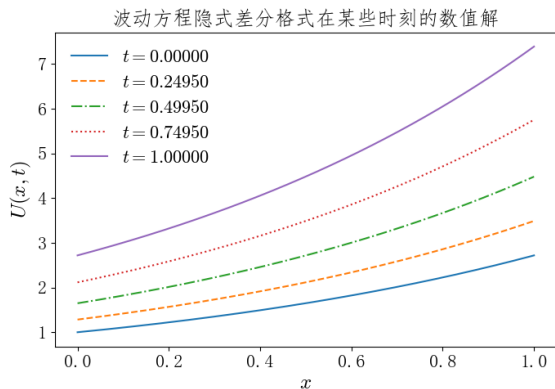


图 12-3 隐式格式求解波动方程某些时刻的数值解以及等值线图

2. 分别采用向前 Euler 差分、向后 Euler 差分、Crank-Nicolson 格式和紧差分格式求解如下一维热传导方程, 解析解为 $u(x, t) = e^x \sin(0.5 - t)$, 设置 $h = 0.01$ 和 $\tau = 0.00001$.

$$\begin{cases} u_t - 2u_{xx} = -e^x [\cos(0.5 - t) + 2\sin(0.5 - t)], & 0 < x < 1, 0 < t \leq 1, \\ u(x, 0) = e^x \sin 0.5, & 0 \leq x \leq 1, \\ u(0, t) = \sin(0.5 - t), u(1, t) = e \cdot \sin(0.5 - t), & 0 < t \leq 1. \end{cases}$$

求解代码如下:

```
#-*- coding: UTF-8 -*-
"""
```

```
@file_name: chapter_12_exp2.py
"""
import numpy as np
from partial_differential_equation_12.pde_heat_conduction_equ_nonhomogeneous \
    import PDEHeatConductionEquationNonhomogeneous

f_fun = lambda x, t: -np.exp(x) * (np.cos(0.5 - t) + 2 * np.sin(0.5 - t))
init_f = lambda x: np.exp(x) * np.sin(0.5)
alpha_f, beta_f = lambda t: np.sin(0.5 - t), lambda t: np.exp(1) * np.sin(0.5 - t)
a, x_a, t_T, x_h, t_h = 2, 1, 1, 0.01, 0.00001
pde_model = lambda x, t: np.exp(x) * np.sin(0.5 - t)

diff_scheme = ["forward", "backward", "crank-nicolson", "compact"]
for scheme in diff_scheme:
    print("方法: ", scheme)
    heat = PDEHeatConductionEquationNonhomogeneous(f_fun, a, init_f, alpha_f, beta_f,
                                                    x_a, t_T, x_h, t_h, pde_model, diff_type=scheme)

    heat.solve_pde()
    heat.plt_pde_heat_surface()
    heat.plt_pde_heat_curve_contourf()
print("=" * 60)
```

如表 12- 1 所示，为四种差分格式求解的平均绝对值误差和最大绝对值误差，求解精度都比较高，其中紧差分格式最好. 如图 12 - 4 所示，为紧差分格式求解一维热传导方程的数值解曲面以及误差曲面，图 12 - 5 为紧差分格式求解一维热传导方程在某些时刻的数值解曲线以及 $t = 1$ 时刻的等值线图.

其他方法的数值解曲面略去，可通过求解代码绘制.

表 12- 1 四种差分格式求解一维热传导方程的误差

方法	平均绝对值误差	最大绝对值误差
向前 Euler 差分格式	6.7051613584e-07	1.7618192756e-06
向后 Eule 差分格式	6.5485267620e-07	1.3891616907e-06
Crank-Nicolson 格式	6.6241354325e-07	1.5736453032e-06
紧差分格式	6.3723893559e-07	1.0579612457e-06

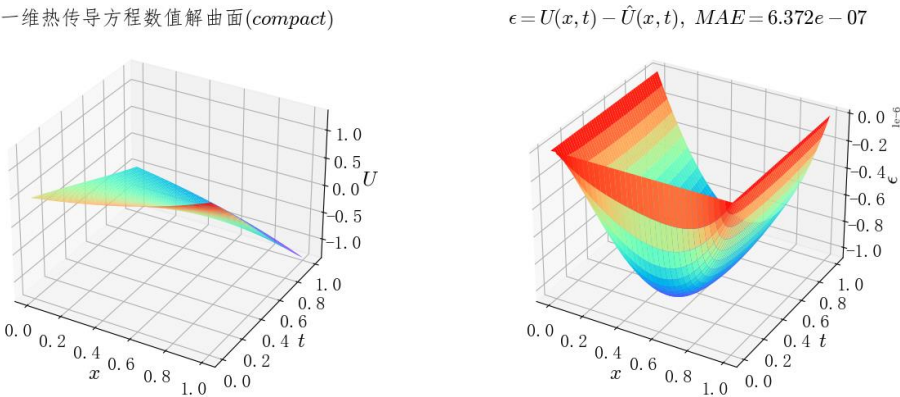


图 12 - 4 紧差分格式求解一维热传导方程数值解曲面以及误差曲面

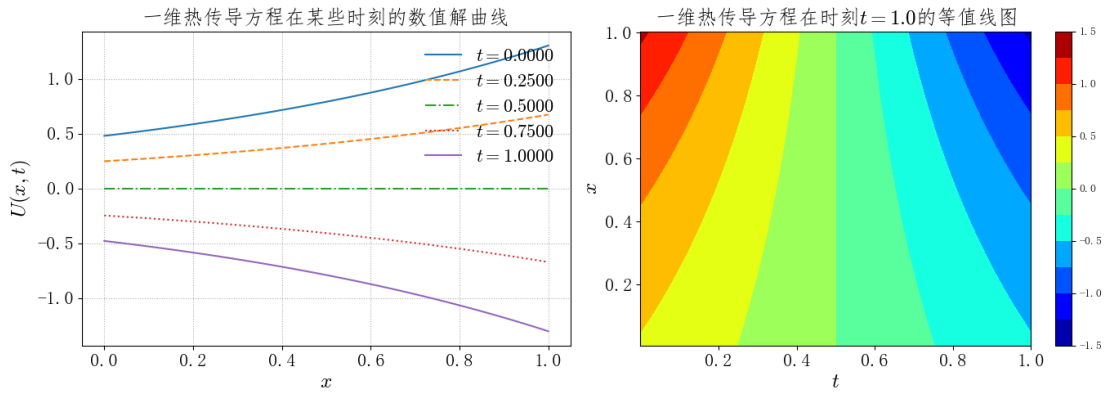


图 12-5 紧差分格式求解一维热传导方程在某些时刻的数值解曲线以及等值线图

3. 采用 Du Fort-Frankel 显式差分格式和 PR 交替方向隐格式方法求解二维热传导问题，解析解为 $u(x,y,t) = \sin(xyt)$ 。对比不同时间和空间步长下的求解精度。

$$\begin{cases} u_t - (u_{xx} + u_{yy}) = (x^2 + y^2)t^2 \sin(xyt) + xy \cos(xyt), & 0 < x, y < 1, & 0 < t \leq 1, \\ u(x, y, 0) = 0, & 0 < x, y < 1, \\ u(0, y, t) = 0, & u(1, y, t) = \sin(yt), & 0 \leq y \leq 1, & 0 < t \leq 1, \\ u(x, 0, t) = 0, & u(x, 1, t) = \sin(xt), & 0 \leq x \leq 1, & 0 < t \leq 1. \end{cases}$$

求解代码如下：

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_12_exp3.py
"""
import numpy as np
from partial_differential_equation_12.pde_heat_conduction_2d_FRADI import
PDEHeatConductionEquation_2D_FRADI
from partial_differential_equation_12.pde_heat_conduction_equ_2d import PDEHeatConductionEquation_2D

f_u0yt = lambda y, t: 0.0 * y + 0.0 * t # u(0,y,t)
f_u1yt = lambda y, t: np.sin(y * t) # u(1,y,t)
f_u0xt = lambda x, t: 0.0 * x + 0.0 * t # u(x,0,t)
f_u1xt = lambda x, t: np.sin(x * t) # u(x,1,t)
f_ut0 = lambda x, y: x * 0 + y * 0 # 初值条件
f_xyt = lambda x, y, t: (x ** 2 + y ** 2) * t ** 2 * np.sin(x * y * t) + x * y * np.cos(x * y * t)
pde_model = lambda x, y, t: np.sin(x * y * t) # 解析解
a_const, x_span, y_span, t_T, tau, h = 1, [0, 1], [0, 1], 1, 0.001, 0.05 # 修改步长

# Du Fort-Frankel 显式差分格式
heat_adi = PDEHeatConductionEquation_2D(a_const, f_xyt, f_u0yt, f_u1yt, f_u0xt, f_u1xt, f_ut0,
                                         x_span, y_span, t_T, h, tau, diff_type="du-fort-frankel",
                                         pde_model=pde_model)

u_xyt = heat_adi.solve_pde()
heat_adi.plt_pde_heat_surface()
```

```
# PR 交替方向隐格式
head_adi = PDEHeatConductionEquation_2D_FRADI(a_const, f_xy, f_ut0, f_u0yt, f_u1yt, f_u0xt, f_u1xt,
                                                x_span, y_span, t_T, tau, h, pde_model)

head_adi.solve_pde()
head_adi.plt_pde_heat_surface()
```

(1) Du Fort-Frankel 显式差分格式

设置 $\tau = 0.001$, $h = 0.05$, 则二维热传导方程的网格比 0.4, 平均绝对值误差: 1.68658×10^{-5} , 最大绝对值误差 5.11289×10^{-5} . 设置 $\tau = 0.0005$, $h = 0.02$, 如图 12-6 所示. 二维热传导方程的网格比 1.25, 平均绝对值误差 9.90453×10^{-6} , 最大绝对值误差: 2.88261×10^{-5} .

二维热传导方程数值解曲面 (du - fort - frankel)

$$\epsilon = U(x, y, t) - \hat{U}(x, y, t), \text{MAE} = 9.905e-06$$

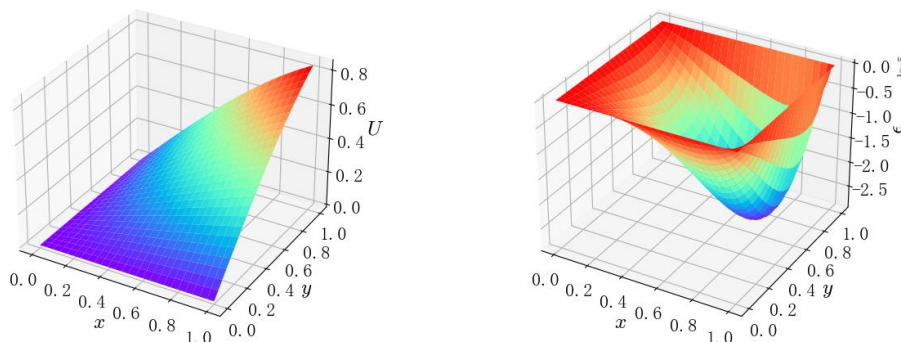


图 12-6 Du Fort-Frankel 显式差分格式求解二维热传导方程数值解曲面以及误差曲面

(2) PR 交替方向隐格式方法

设置 $\tau = 0.001$, $h = 0.05$, 平均绝对值误差: 5.87730×10^{-6} , 最大绝对值误差 1.97863×10^{-6} . 可见在同样步长下, PR 交替方向隐格式求解精度高于 Du Fort-Frankel 显式差分格式. 设置 $\tau = 0.0005$, $h = 0.02$, 如图 12-7 所示. 平均绝对值误差: 1.03567×10^{-7} , 最大绝对值误差 3.24820×10^{-7} . 可见缩小求解步长, 可提高求解精度, 但同样意味着求解时间复杂度的提高.

二维热传导方程 (FR-ADI) 数值解曲面

$$\epsilon = U(x, y, t) - \hat{U}(x, y, t), \text{MAE} = 1.036e-07$$

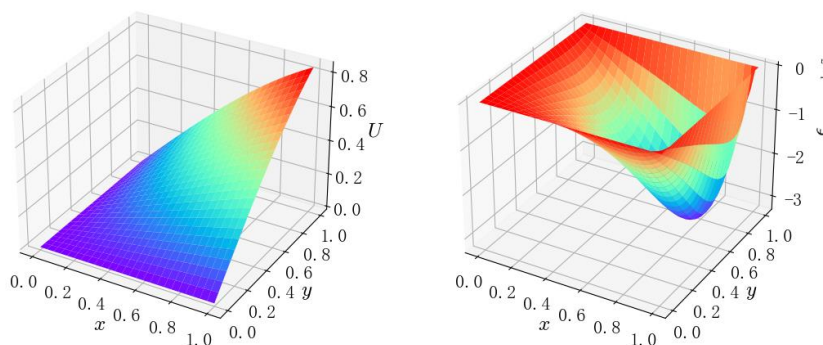


图 12-7 PR 交替方向隐格式求解二维热传导方程数值解曲面以及误差曲面

4. 采用拉普拉斯方程超松弛迭代法, 求解计算矩形区域内的谐波函数 $u(x, y)$ 的近似值. 设置步长 $h = 0.01$ 、精度要求 $\epsilon = 10^{-8}$, 解析解为 $u(x, y) = e^x (\sin y + \cos y)$.

$$\begin{cases} u_{xx} + u_{yy} = 0, & 0 < x < 1, \quad 0 < y < 1, \\ u(x, 0) = e^x, \quad u(x, 1) = e^x (\sin 1 + \cos 1), & 0 \leq x \leq 1, \\ u(0, y) = \sin y + \cos y, \quad u(1, y) = e(\sin y + \cos y), & 0 \leq y \leq 1. \end{cases}$$

求解代码如下:

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_12_exp4.py
"""
import numpy as np
from partial_differential_equation_12.pde_laplace_equation_dirichlet import PDELaplaceEquationDirichlet
import time

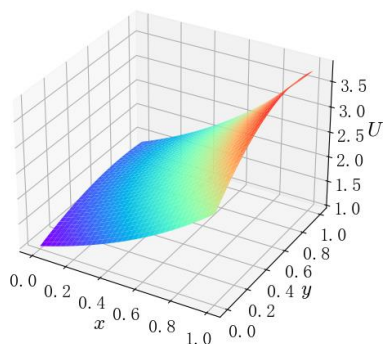
f_u0y = lambda y: np.sin(y) + np.cos(y) # u(0,y)
f_uay = lambda y: np.exp(1) * (np.sin(y) + np.cos(y)) # u(1,y)
f_ux0 = lambda x: np.exp(x) # u(x,0)
f_uxb = lambda x: np.exp(x) * (np.sin(1) + np.cos(1)) # u(x,1)
pde_model = lambda x, y: np.exp(x) * (np.sin(y) + np.cos(y)) # 解析解
x_a, y_b, x_h, y_h, eps = 1, 1, 0.01, 0.01, 1e-8

laplace = PDELaplaceEquationDirichlet(f_ux0, f_uxb, f_u0y, f_uay, x_a, y_b, x_h, y_h, max_iter=1000,
                                       eps=eps, pde_model=pde_model)

time_start = time.time()
laplace.solve_pde()
time_end = time.time()
print("消耗时间: ", time_end - time_start)
laplace.plt_pde_laplace_surface()
```

满足精度要求下共需迭代 401 次, $|r_{ij}| = 3.18862 \times 10^{-9} < 10^{-8}$, 消耗时间 8.84805, 平均绝对值误差 1.28269×10^{-6} , 最大绝对值误差 2.80123×10^{-6} .

Laplace 方程数值解曲面



$\epsilon = U(x, y) - \hat{U}(x, y)$, $MAE = 1.283e-06$

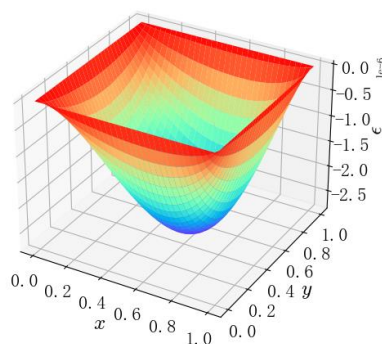


图 12-8 超松弛迭代法求解拉普拉斯方程的数值解曲面以及误差曲面

5. 应用超松弛迭代法和三对角块矩阵求解如下泊松方程问题, 该问题的精确解为 $u(x, y) = x^3 + y^3$. 设置步长 $h = 0.05$, 精度要求 $\epsilon = 10^{-8}$, 其中三对角块矩阵网格划分分别为 10×10 和 20×20 .

$$\begin{cases} u_{xx} + u_{yy} = 6(x + y), & 0 < x < 1, \quad 0 < y < 1, \\ u(0, y) = y^3, \quad u(1, y) = 1 + y^3, & 0 \leq y \leq 1, \\ u(x, 0) = x^3, \quad u(x, 1) = 1 + x^3, & 0 \leq x \leq 1. \end{cases}$$

求解代码如下:

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_12_exp5.py
"""
from partial_differential_equation_12.pde_poisson_equation_matrix import PDEPoissonEquationTriBMatrix
from partial_differential_equation_12.pde_poisson_equation_sor import PDEPoissonEquationSORIteration

g_xy = lambda x, y: -6 * (x + y)
f_u0y, f_uay = lambda y: y ** 3, lambda y: 1 + y ** 3
f_ux0, f_uxb = lambda x: x ** 3, lambda x: 1 + x ** 3
pde_model = lambda x, y: x ** 3 + y ** 3

# 超松弛迭代法
x_a, y_b, x_h, y_h, eps = 1, 1, 0.05, 0.05, 1e-8
poisson = PDEPoissonEquationSORIteration(g_xy, f_ux0, f_uxb, f_u0y, f_uay, x_a, y_b, x_h, y_h, eps=eps,
                                          max_iter=1000, pde_model=pde_model)

poisson.solve_pde()
poisson.plt_pde_poisson_surface()

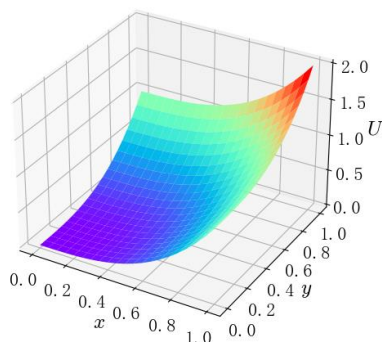
# 三对角块矩阵
x_span, y_span, n_x, n_y = [0, 1], [0, 1], 20, 20
poisson = PDEPoissonEquationTriBMatrix(g_xy, f_ux0, f_uxb, f_u0y, f_uay, x_span, y_span, n_x, n_y,
                                       pde_model=pde_model, is_show=True)

poisson.solve_pde()
poisson.plt_pde_poisson_surface()
```

(1) 超松弛迭代法

满足精度要求下共需迭代 76 次, $|r_{ij}| = 6.81463 \times 10^{-9} < 10^{-8}$, 平均绝对值误差 1.06512×10^{-9} , 最大绝对值误差 5.69187×10^{-9} .

Poisson 方程 (Dirichlet) 数值解曲面



$\epsilon = U(x, y) - \hat{U}(x, y)$, $MAE = 1.065e-09$

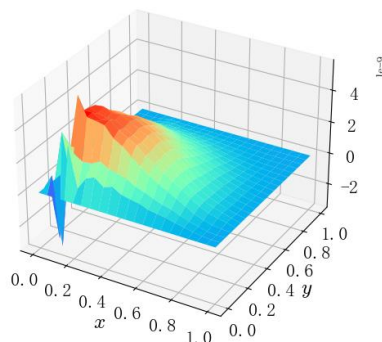


图 12-9 超松弛迭代法求解泊松方程数值解曲面以及误差曲面

(2) 三对角块矩阵

网格划分为 10×10 时, 平均绝对值误差 1.02936×10^{-16} , 最大绝对值误差: 5.55112×10^{-16} . 网格划分为 20×20 时, 平均绝对值误差 1.26506×10^{-16} , 最大绝对值误差: 6.66134×10^{-16} .

Poisson 方程 (Dirichlet) 数值解曲面

$$\epsilon = U(x, y) - \hat{U}(x, y), \text{ MAE} = 1.029e-16$$

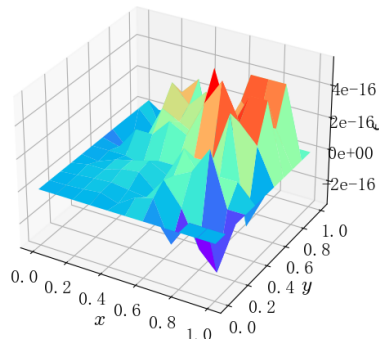
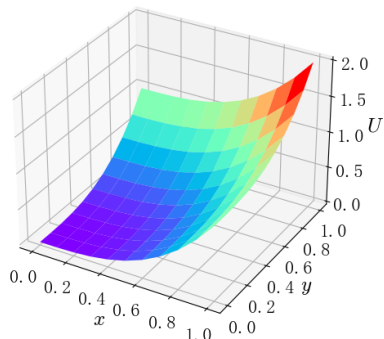


图 12 - 10 三对角块矩阵法求解泊松方程数值解曲面以及误差曲面 (10×10)

Poisson 方程 (Dirichlet) 数值解曲面

$$\epsilon = U(x, y) - \hat{U}(x, y), \text{ MAE} = 1.265e-16$$

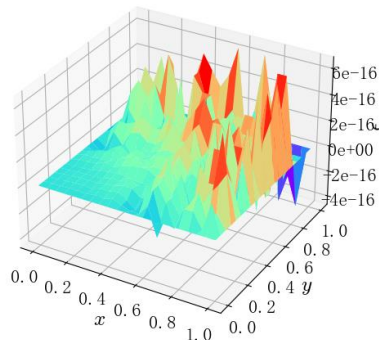
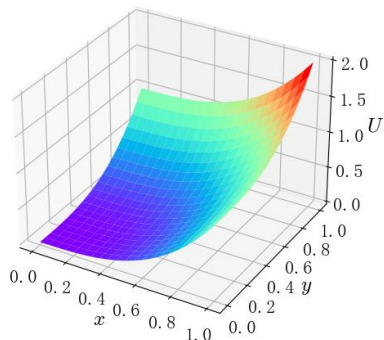


图 12 - 11 三对角块矩阵法求解泊松方程数值解曲面以及误差曲面 (20×20)

第 13 章 实验内容参考答案

1. 采用黄金分割搜索、斐波那契搜索、二次插值逼近和三次逼近算法，求解函数 $f(x) = e^{-x^2+2x} \sin x^2$ 在区间 $[-1, 3]$ 的极小值和极大值，精度要求 10^{-16} 。试对函数进行可视化，以便确定单峰区间。

求解代码如下：

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_13_exp1.py
"""

import numpy as np
import sympy
from numerical_optimization_13.golden_section_search import GoldenSectionSearchOptimization
from numerical_optimization_13.fibonacci_search import FibonacciSearchOptimization
from numerical_optimization_13.interp2_approximation import Interp2ApproximationOptimization
from numerical_optimization_13.cubic_approximation import CubicApproximationOptimization
from Experiment.util_font import *

fun = lambda x: np.exp(-x ** 2 + 2 * x) * np.sin(x ** 2)

# 函数可视化
plt.figure(figsize=(7, 5))
xi = np.linspace(-1, 3, 200)
plt.plot(xi, fun(xi), "-")
plt.xlabel("$x$", fontdict={"fontsize": 18})
plt.ylabel("$y$", fontdict={"fontsize": 18})
plt.title("单变量函数在区间的$[-1, 3]$图像", fontsize=18)
plt.grid(ls=":")
plt.tick_params(labelsize=18)
plt.show()

# 黄金分割法
x_span, eps = [1.5, 2.5], 1e-16 # 极小值
gssso = GoldenSectionSearchOptimization(fun, x_span, eps, is_minimum=True) # 注意 is_minimum 的设置
e_x = gssso.fit_optimize()
print("黄金分割法极小值: (%.15f, %.15f)" % (e_x[0], e_x[1]))
gssso.plt_optimization([-1, 3])
x_span, eps = [0.5, 1.5], 1e-16 # 极大值
gssso = GoldenSectionSearchOptimization(fun, x_span, eps, is_minimum=False) # 注意 is_minimum 的设置
e_x = gssso.fit_optimize()
gssso.plt_optimization([-1, 3])
print("黄金分割法极大值: (%.15f, %.15f)" % (e_x[0], e_x[1]))
print("=" * 60)

# 斐波那契搜索
```



```

x_span, eps = [1.5, 2.5], 1e-16 # 极小值
fso = FibonacciSearchOptimization(fun, x_span, eps, is_minimum=True) # 注意 is_minimum 的设置
e_x = fso.fit_optimize()
print("斐波那契搜索极小值: (%.15f, %.15f)" % (e_x[0], e_x[1]))
fso.plt_optimization([-1, 3])
x_span, eps = [0.5, 1.5], 1e-16 # 极大值
fso = FibonacciSearchOptimization(fun, x_span, eps, is_minimum=False) # 注意 is_minimum 的设置
e_x = fso.fit_optimize()
print("斐波那契搜索极大值: (%.15f, %.15f)" % (e_x[0], e_x[1]))
fso.plt_optimization([-1, 3])
print("=" * 60)

# 二次插值逼近
x_span, eps = [1.8, 2.2], 1e-16 # 极小值
iao = Interp2ApproximationOptimization(fun, x_span, eps, is_minimum=True) # 注意 is_minimum 的设置
e_x = iao.fit_optimize()
print("二次插值逼近极小值: (%.15f, %.15f)" % (e_x[0], e_x[1]))
iao.plt_optimization([-1, 3])
f_max = lambda x: -1 * np.exp(-x ** 2 + 2 * x) * np.sin(x ** 2)
x_span, eps = [1.0, 1.2], 1e-16 # 极大值
iao = Interp2ApproximationOptimization(f_max, x_span, eps, is_minimum=False) # 注意 is_minimum 的设置
e_x = iao.fit_optimize()
print("二次插值逼近极大值: (%.15f, %.15f)" % (e_x[0], e_x[1]))
iao.plt_optimization([-1, 3])
print("=" * 60)

# 三次逼近方法
x = sympy.symbols("x")
fun = sympy.exp(-x ** 2 + 2 * x) * sympy.sin(x ** 2)
x_span, eps = [1.8, 2.2], 1e-16 # 极小值
cao = CubicApproximationOptimization(fun, x_span, eps, is_minimum=True) # 注意 is_minimum 的设置
e_x = cao.fit_optimize()
print("三次逼近极小值: (%.15f, %.15f)" % (e_x[0], e_x[1]))
cao.plt_optimization([-1, 3])
f_max = -1 * sympy.exp(-x ** 2 + 2 * x) * sympy.sin(x ** 2)
x_span, eps = [1.0, 1.2], 1e-16 # 极大值
cao = CubicApproximationOptimization(f_max, x_span, eps, is_minimum=False) # 注意 is_minimum 的设置
e_x = cao.fit_optimize()
print("三次逼近极大值: (%.15f, %.15f)" % (e_x[0], e_x[1]))
cao.plt_optimization([-1, 3])

```

表 13 - 1 四种方法求解单变量函数极值问题结果

方法	极值类型	求解区间	优化次数	x^*	$f(x^*)$
黄金分割法	极小值	[1.5, 2.5]	72	2.058503796110089	-0.788412406203963
	极大值	[0.5, 1.5]	72	1.188816512941634	2.590605758951006
斐波那契搜索	极小值	[1.5, 2.5]	72	2.058503796110089	-0.788412406203963

	极大值	[0.5, 1.5]	72	1.188816512941634	2.590605758951006
二次插值逼近	极小值	[1.8, 2.2]	7	2.058503802062671	-0.788412406203963
	极大值	[1.0, 1.2]	5	1.188816514024227	2.590605758951006
三次逼近	极小值	[1.8, 2.2]	8	2.058503798408582	-0.788412406203963
	极大值	[1.0, 1.2]	8	1.188816514055312	2.590605758951006

(1) 黄金分割搜索法

若搜索区间修改为[1.8, 2.2]，则需要搜索 69 次.

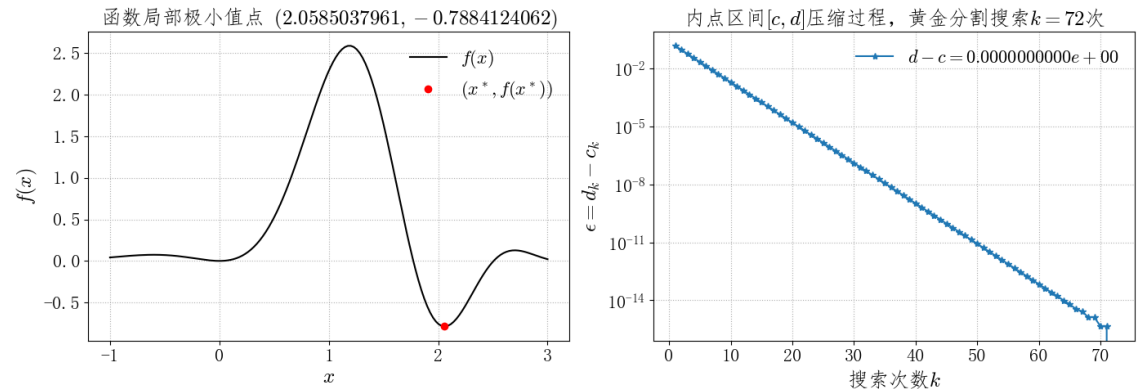


图 13 - 1 黄金分割搜索法求解局部极小值

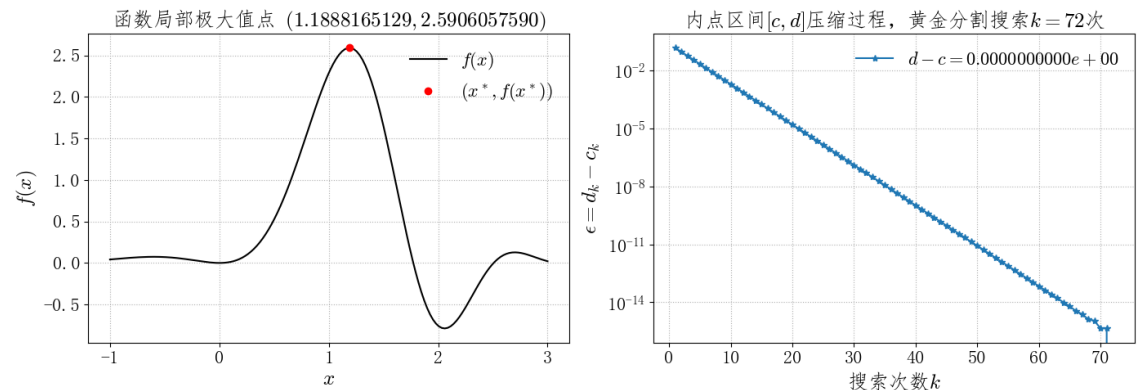


图 13 - 2 黄金分割搜索法求解局部极大值

(2) 斐波那契搜索

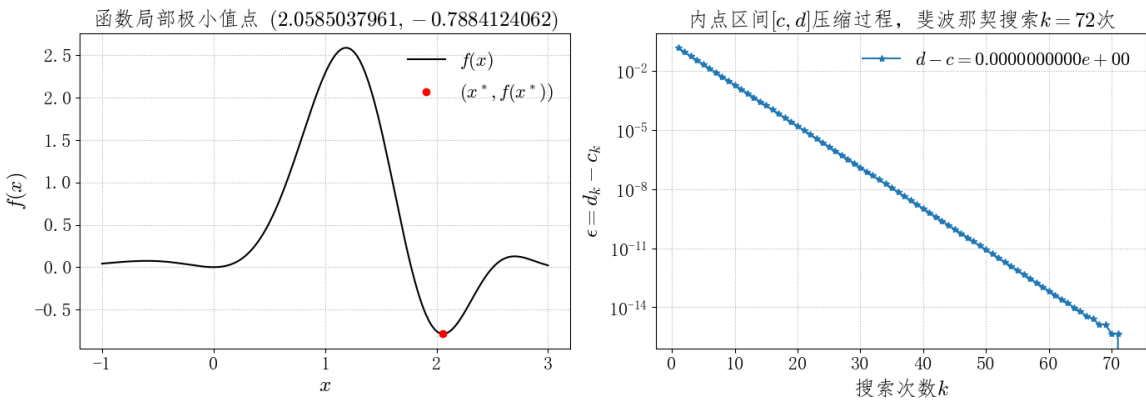


图 13 - 3 斐波那契搜索求解局部极小值

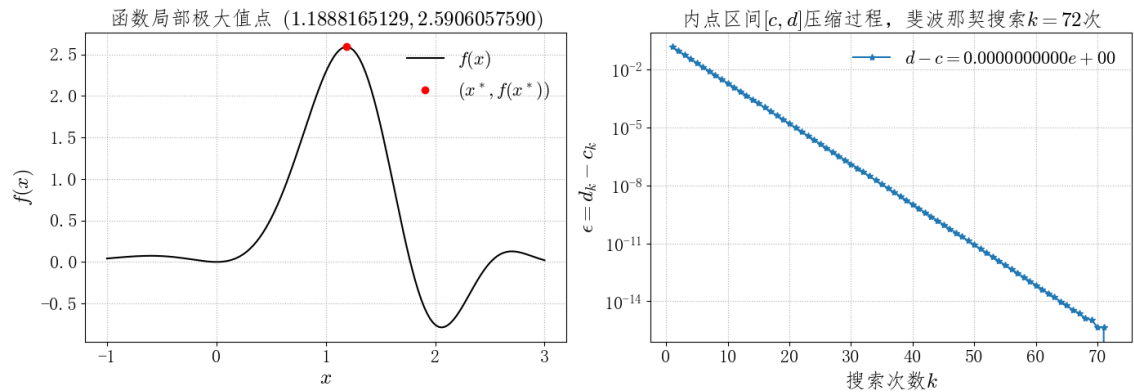


图 13-4 斐波那契搜索求解局部极大值

(3) 二次插值逼近

缩小搜索区间, 较大的区间可能使得求解失败. 优化过程如图 13-5 和图 13-6 所示, 所求极值如图像标题所示, 或如表 13-1 所示.

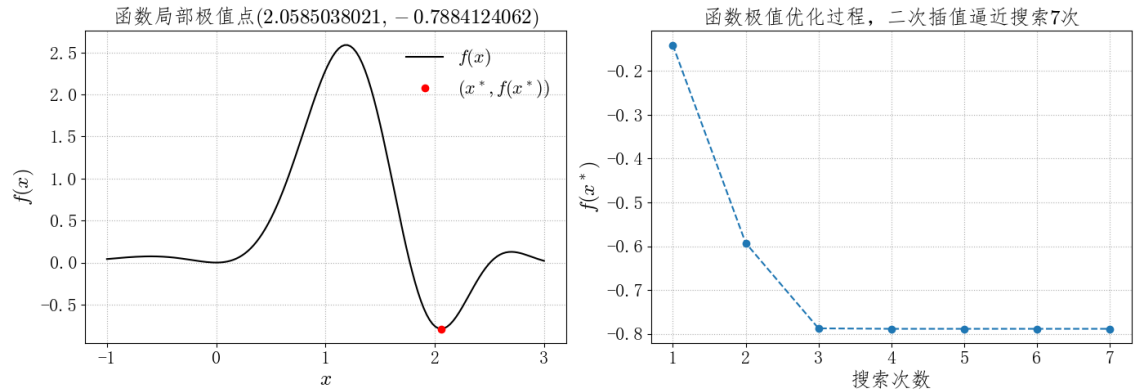


图 13-5 二次插值逼近求解目标函数的极小值

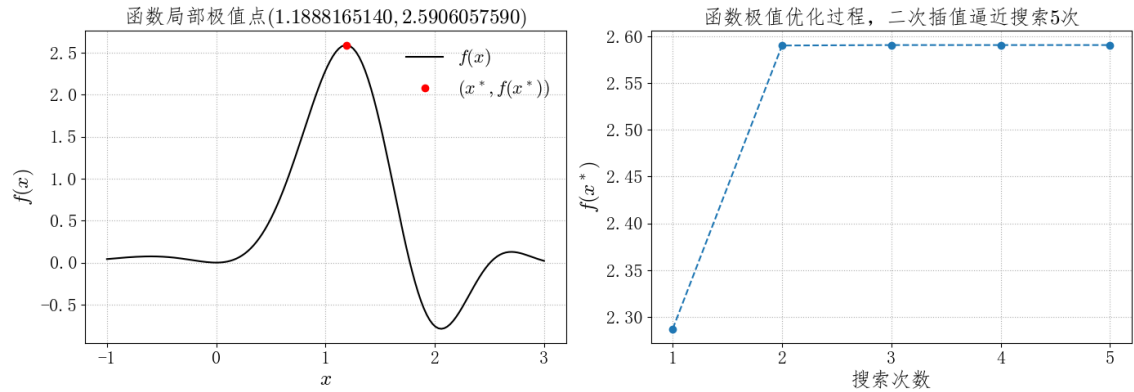


图 13-6 二次插值逼近求解目标函数的极大值

(4) 三次逼近方法

缩小搜索区间, 较大的区间可能使得求解失败. . 优化过程如图 13-7 和图 13-8 所示, 所求极值如图像标题所示, 或如表 13-1 所示.

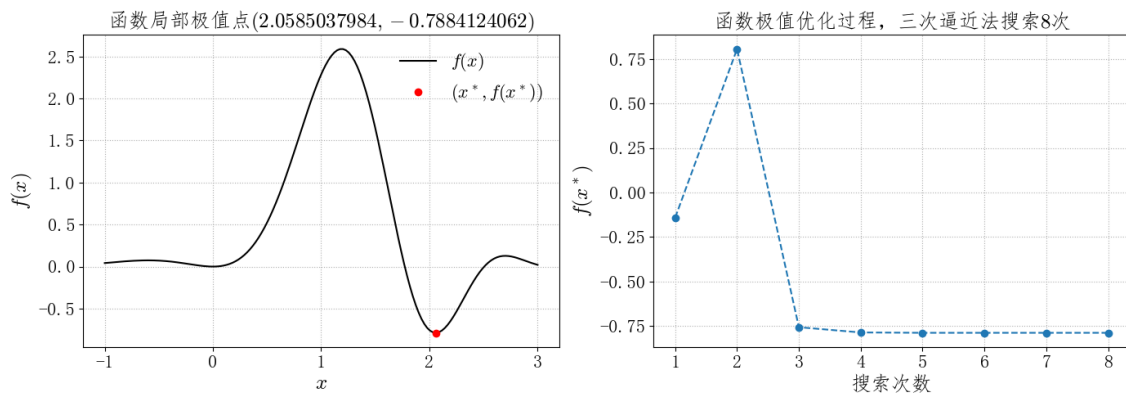


图 13-7 三次逼近求解目标函数的极小值

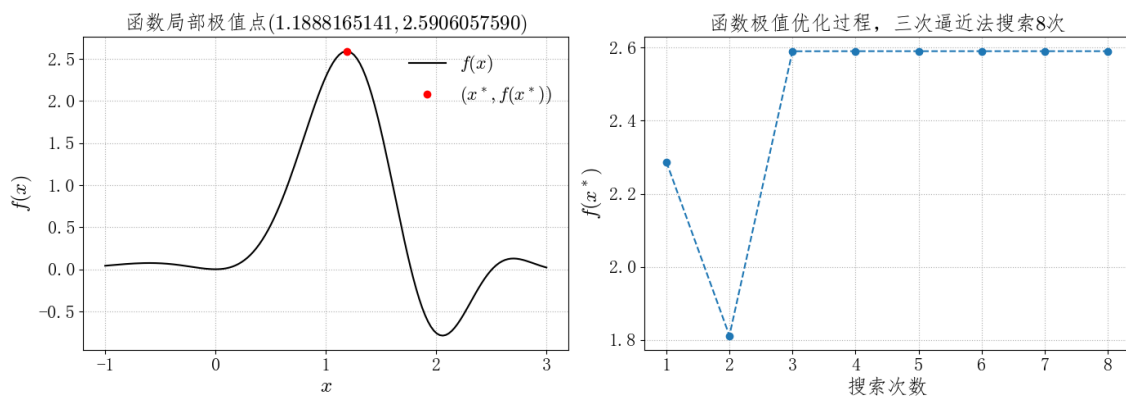


图 13-8 三次逼近求解目标函数的极大值

2. 利用内德—米德和鲍威尔方法求解函数 $f(x, y) = xe^{-x^2 - y^2}$ 在区间 $[-2, 2] \times [-2, 2]$ 内的极小值和极大值, 并通过二元函数等值线的方法确定初始的单纯形的三个顶点或初值点.

求解代码如下:

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_13_exp2.py
"""

import numpy as np
import sympy
from numerical_optimization_13.powell_method import PowellOptimization
from numerical_optimization_13.nelder_mead_2d import NelderMeadOptimization
from Experiment.util_font import *

# 等值线可视化
fh = lambda x, y: x * np.exp(-x ** 2 - y ** 2)
plt.figure(figsize=(7, 5))
x, y = np.linspace(-2, 2, 100), np.linspace(-2, 2, 100)
xi, yi = np.meshgrid(x, y)
zi = fh(xi, yi)
c = plt.contour(xi, yi, zi, levels=15, cmap=plt.get_cmap("jet"))
plt.clabel(c, inline=True, fontsize=10) # 添加等高数值标记
```

```

plt.xlabel("$x$", fontdict={"fontsize": 18})
plt.ylabel("$y$", fontdict={"fontsize": 18})
plt.title("二元函数等值线图", fontdict={"fontsize": 18})
plt.grid(ls=":")
plt.tick_params(labelsize=18)
plt.show()

f_min = lambda x: x[0] * np.exp(-x[0]**2 - x[1]**2) # 极小值
V_k = np.array([[-1.5, 0], [-0.5, 1], [-0.1, -1]])
eps = 1e-15
nmo = NelderMeadOptimization(f_min, V_k, eps, is_minimum=True) # 求极小值
m_x = nmo.fit_optimize()
print("[%.15f, %.15f, %.15f]" % (m_x[0], m_x[1], m_x[2]))
nmo.plt_optimization([-2, 2], [-2, 2])

f_max = lambda x: -1 * (x[0] * np.exp(-x[0]**2 - x[1]**2)) # 极大值
V_k = np.array([[0.2, -1], [1, 1], [1.5, -0.5]])
nmo = NelderMeadOptimization(f_max, V_k, eps, is_minimum=False) # 求极大值
m_x = nmo.fit_optimize()
print("[%.15f, %.15f, %.15f]" % (m_x[0], m_x[1], m_x[2]))
nmo.plt_optimization([-2, 2], [-2, 2])

# 鲍威尔方法
x = sympy.symbols("x_1:3")
f_min = x[0] * sympy.exp(-x[0]**2 - x[1]**2)
x0, eps = [-1.5, 0], 1e-15
po = PowellOptimization(f_min, x0, eps, is_minimum=True) # 极小值
e_x = po.fit_optimize()
po.plt_optimization([-2, 2], [-2, 2])
print("%.15f, %.15f" % (e_x[0], e_x[1]))
print(f_min.subs({x[0]: e_x[0], x[1]: e_x[1]}))

x = sympy.symbols("x_1:3")
f_max = -1 * x[0] * sympy.exp(-x[0]**2 - x[1]**2)
x0, eps = [1.5, 1], 1e-15
po = PowellOptimization(f_max, x0, eps, is_minimum=False) # 极大值
e_x = po.fit_optimize()
po.plt_optimization([-2, 2], [-2, 2])
print("%.15f, %.15f" % (e_x[0], e_x[1]))
print(-1 * f_max.subs({x[0]: e_x[0], x[1]: e_x[1]}))

```

(1) 内德—米德方法

通过等值线图, 对于极小值, 设置三个顶点为 $\{(-1.5, 0), (-0.5, 1), (-0.1, -1)\}$, 对于极大值, 设置三个顶点为 $\{(0.2, -1), (1, 1), (1.5, -0.5)\}$. 设置精度要求 $\varepsilon = 10^{-15}$, 求解结果如表 13-2 所示. 迭代优化过程如图 13-9 和图 13-10 所示.

表 13 - 2 内德—米德方法求解二元函数极值问题结果

极值类型	x^*	y^*	$f(x^*,y^*)$	收敛精度
极小值	-0.707106769037149	-0.000000015138496	-0.428881942480353	9.104505742017336e-16
极大值	0.707106778188015	-0.000000019157076	0.428881942480353	4.577566798522237e-16

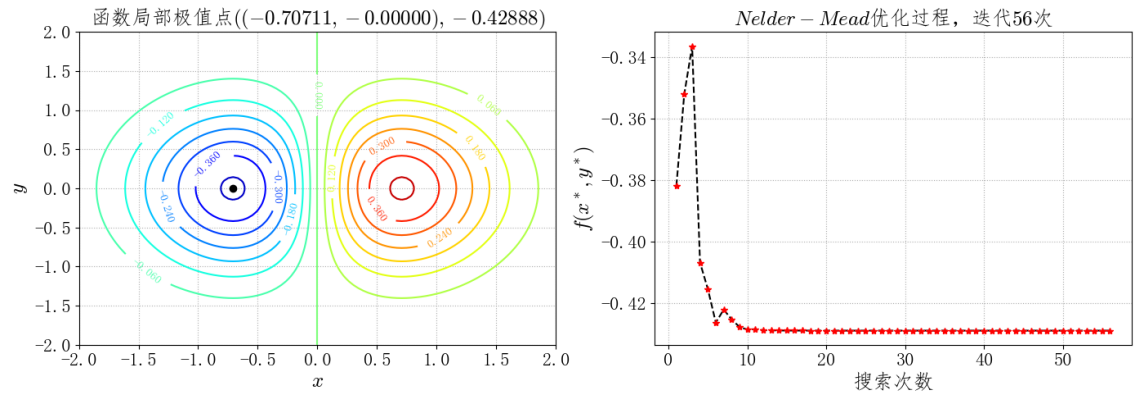


图 13 - 9 内德—米德方法求解二元函数极小值

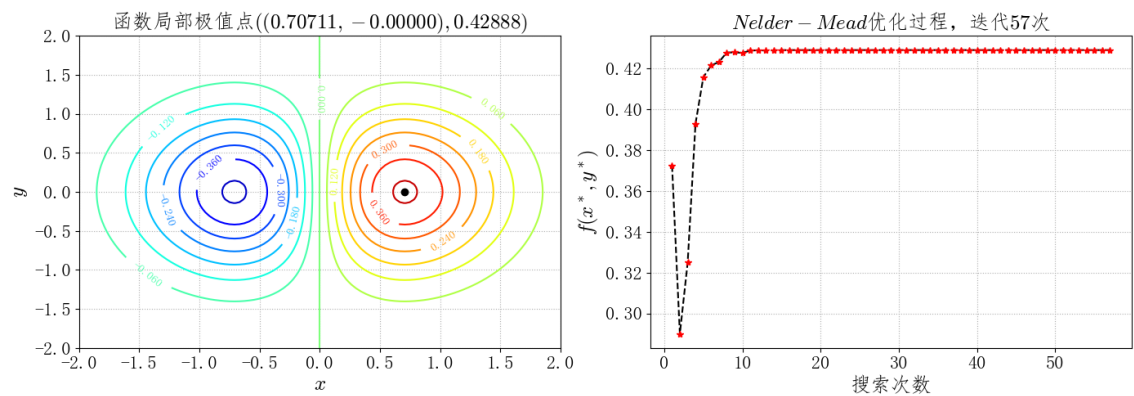


图 13 - 10 内德—米德方法求解二元函数极大值

(2) 鲍威尔方法

对于极小值，设置初值点 $\boldsymbol{x}_0=(-1.5,1)$ ，对于极大值，设置初值点 $\boldsymbol{x}_0=(1.5,1)$ ，求解结果如表 13 - 3 所示. 收敛过程如图 13 - 11 和图 13 - 12 所示.

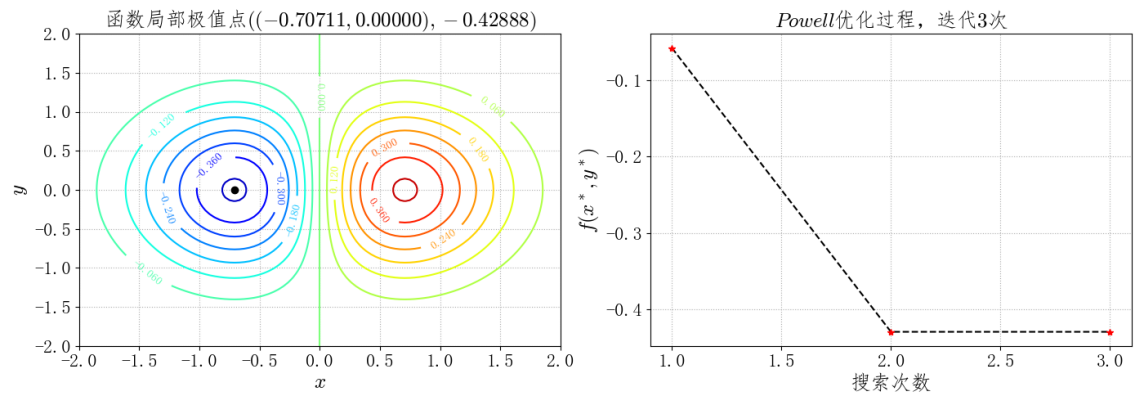


图 13 - 11 鲍威尔方法求解二元函数极小值

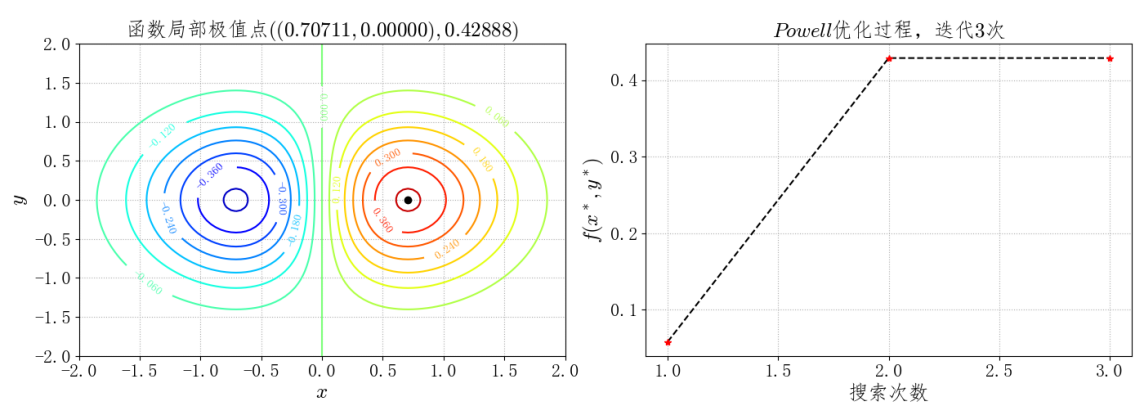


图 13 - 12 鲍威尔方法求解二元函数极大值

表 13 - 3 鲍威尔方法求解二元函数极值问题结果

极值类型	x^*	y^*	$f(x^*, y^*)$
极小值	-0.707106779923930	0.000000007400576	0.428881942480353
极大值	0.707106779923930	0.000000007400576	0.428881942480353

3. 利用梯度法和牛顿法求解函数 $f(x,y)=x^3+y^3-3x-3y+5$ 在区间 $[-2,2]\times[-2,2]$ 内的极小值和极大值，并通过二元函数等值线的方法确定初值点.

求解代码如下:

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_13_exp3.py
"""

import numpy as np
import sympy
from Experiment.util_font import *
from numerical_optimization_13.gradient_descent import GradientDescentOptimization
from numerical_optimization_13.newton_method import NewtonOptimization

# 梯度方法
x = sympy.symbols("x_1:3")
fun = x[0]**3 + x[1]**3 - 3*x[0] - 3*x[1] + 5
x0, eps = [1.5, 1.5], 1e-15
gdo = GradientDescentOptimization(fun, x0, eps, is_minimum=True) # 极小值
e_x = gdo.fit_optimize()
print(fun.subs({x[0]: e_x[0], x[1]: e_x[1]}))
print(gdo.local_extremum)
gdo.plt_optimization([-2, 2], [-2, 2])
x = sympy.symbols("x_1:3")
fun = -1*(x[0]**3 + x[1]**3 - 3*x[0] - 3*x[1] + 5)
x0, eps = [-0.5, -1.5], 1e-15
gdo = GradientDescentOptimization(fun, x0, eps, is_minimum=False) # 极大值
e_x = gdo.fit_optimize()
```



```
print(fun.subs({x[0]: e_x[0], x[1]: e_x[1]}))
print(gdo.local_extremum)
gdo.plt_optimization([-2, 2], [-2, 2])
print("=" * 80)

# 牛顿法
x = sympy.symbols("x_1:3")
fun = x[0]**3 + x[1]**3 - 3*x[0] - 3*x[1] + 5
x0, eps, opt_type = [1.5, 1.5], 1e-16, "improve"
no= NewtonOptimization(fun, x0, eps, opt_type, is_minimum=True) # 极小值
e_x = no.fit_optimize()
no.plt_optimization([-2, 2], [-2, 2])
print(fun.subs({x[0]: e_x[0], x[1]: e_x[1]}))
print(no.local_extremum)
x = sympy.symbols("x_1:3")
fun = -1*(x[0]**3 + x[1]**3 - 3*x[0] - 3*x[1] + 5)
x0, eps, opt_type = [-0.5, -1.5], 1e-16, "improve"
no= NewtonOptimization(fun, x0, eps, opt_type, is_minimum=False) # 极大值
e_x = no.fit_optimize()
no.plt_optimization([-2, 2], [-2, 2])
print(fun.subs({x[0]: e_x[0], x[1]: e_x[1]}))
print(no.local_extremum)
```

(1) 梯度法

对于极小值，设置初值为 $\boldsymbol{x}_0=(1.5, 1.5)$ ，对于极大值，设置初值为 $\boldsymbol{x}_0=(-0.5, -1.5)$ ，求解结果如表 13 - 4 所示，优化过程如图 13 - 13 和图 13 - 14 所示。

表 13 - 4 梯度法求解二元函数极值问题结果

极值类型	x^*	y^*	$f(x^*,y^*)$
极小值	1.0000000127537145	1.0000000127537145	1.000000000000000
极大值	-0.9987217192207317	-1.0021268052551768	8.99998152056150

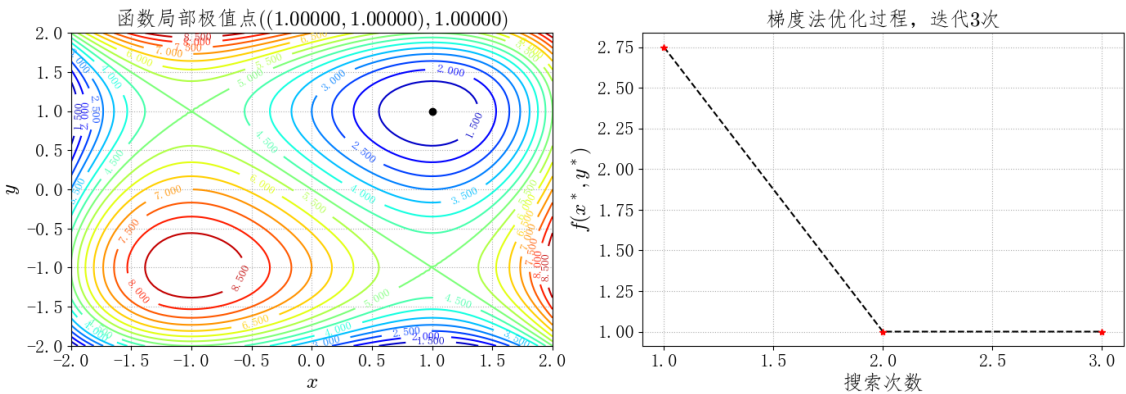


图 13 - 13 梯度法求解二元函数极小值

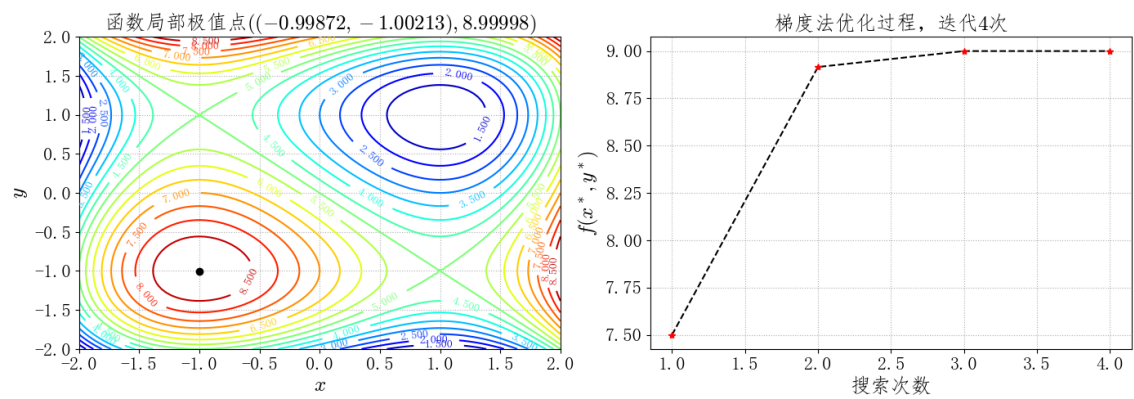


图 13 - 14 梯度法求解二元函数极大值

(2) 牛顿法

对于极小值，设置初值为 $\boldsymbol{x}_0=(1.5, 1.5)$ ，对于极大值，设置初值为 $\boldsymbol{x}_0=(-0.5, -1.5)$ ，求解结果如表 13 - 5 所示，优化过程如图 13 - 15 和图 13 - 16 所示.

表 13 - 5 改进的牛顿法求解二元函数极值问题结果

极值类型	x^*	y^*	$f(x^*, y^*)$
极小值	1.000000003663436	1.000000003663436	1.000000000000000
极大值	-1.0000000051160125	-1.0000000089119785	9.000000000000000

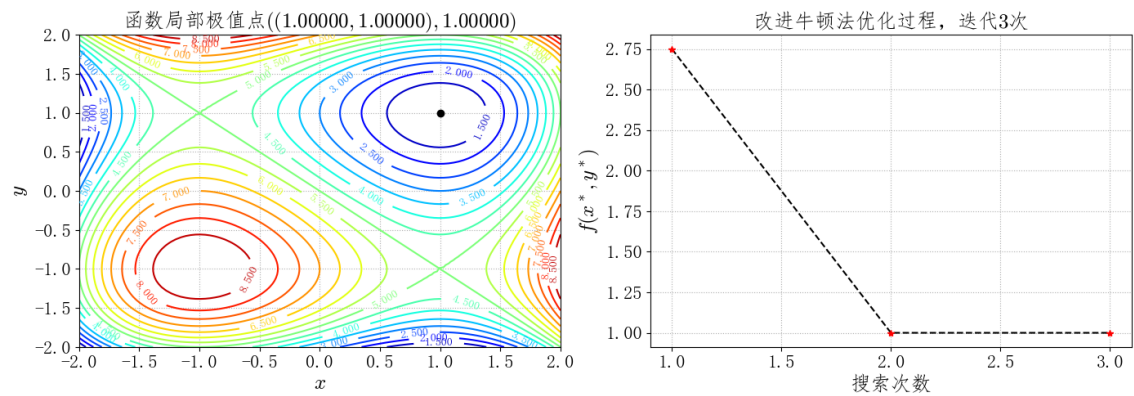


图 13 - 15 改进的牛顿法求解二元函数极小值

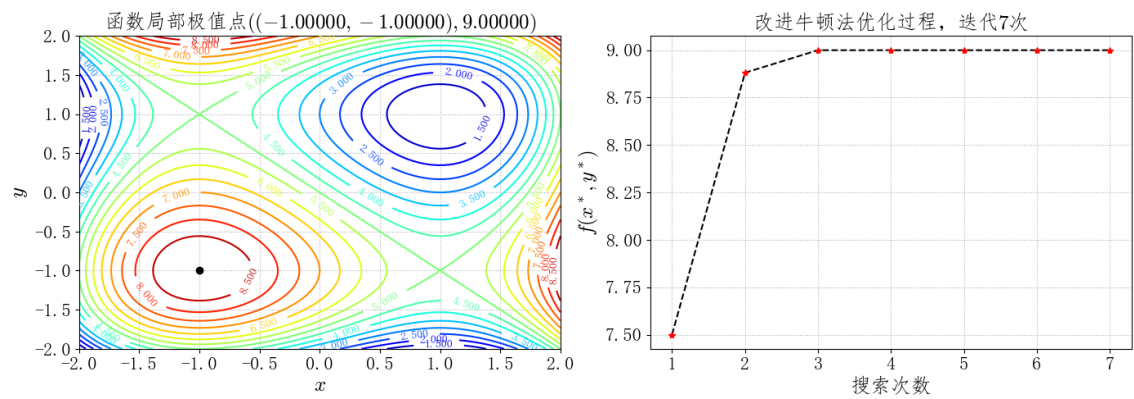


图 13 - 16 改进的牛顿法求解二元函数极大值

4. 采用拟牛顿法求解函数 $f(x, y) = x^2 \sin(x + y^2) + y^2 e^x + 6 \cos(x^2 + y)$ 在初值 $\mathbf{x}_0 = (-2.5, -2.5)$ 附近的极小值和 $\mathbf{x}_0 = (-1.5, -2)$ 附近的极大值, 并可视化 $[-3, -1] \times [-3, -1]$ 区间的图像。

求解代码如下:

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_13_exp4.py
"""

import sympy
from numerical_optimization_13.dfp_quasi_newton import DFPQuasiNewtonOptimization
from numerical_optimization_13.bfgs_quasi_newton import BFGSQuasiNewtonOptimization

# DFP 算法
x = sympy.symbols("x_1:3")
fun = x[0]**2 * sympy.sin(x[0] + x[1]**2) + x[1]**2 * sympy.exp(x[0]) + 6 * sympy.cos(x[0]**2 + x[1])
x0, eps = [-2.5, -2.5], 1e-16 # 极小值
dfp = DFPQuasiNewtonOptimization(fun, x0, eps=eps, is_minimum=True) # 极小值
sol = dfp.fit_optimize()
print(dfp.local_extremum)
print("最优解: %.15f, %.15f, 最优值: %.15f" % (sol[0], sol[1], sol[2]))
dfp.plt_optimization([-3, -1], [-3, -1])
x = sympy.symbols("x_1:3")
fun = -1 * (x[0]**2 * sympy.sin(x[0] + x[1]**2) + x[1]**2 * sympy.exp(x[0]) + 6 * sympy.cos(x[0]**2 + x[1]))
x0, eps = [-1.5, -2], 1e-16 # 极大值
dfp = DFPQuasiNewtonOptimization(fun, x0, eps=eps, is_minimum=False) # 极大值
sol = dfp.fit_optimize()
print(dfp.local_extremum)
print("最优解: %.15f, %.15f, 最优值: %.15f" % (sol[0], sol[1], sol[2]))
dfp.plt_optimization([-3, -1], [-3, -1])
print("=" * 80)

# BFGS
x = sympy.symbols("x_1:3")
fun = x[0]**2 * sympy.sin(x[0] + x[1]**2) + x[1]**2 * sympy.exp(x[0]) + 6 * sympy.cos(x[0]**2 + x[1])
x0, eps = [-2.5, -2.5], 1e-16 # 极小值
bfgs = BFGSQuasiNewtonOptimization(fun, x0, eps=eps, is_minimum=True) # 极小值
sol = bfgs.fit_optimize()
print(bfgs.local_extremum)
print("最优解: %.15f, %.15f, 最优值: %.15f" % (sol[0], sol[1], sol[2]))
bfgs.plt_optimization([-3, -1], [-3, -1])
x = sympy.symbols("x_1:3")
fun = -1 * (x[0]**2 * sympy.sin(x[0] + x[1]**2) + x[1]**2 * sympy.exp(x[0]) + 6 * sympy.cos(x[0]**2 + x[1]))
x0, eps = [-1.5, -2], 1e-16 # 极大值
bfgs = BFGSQuasiNewtonOptimization(fun, x0, eps=eps, is_minimum=False) # 极大值
sol = bfgs.fit_optimize()
print(bfgs.local_extremum)
```

```
print("最优解: %.15f, %.15f, 最优值: %.15f" % (sol[0], sol[1], sol[2]))
bfgs.plt_optimization([-3, -1], [-3, -1])
```

(1) DFP 算法

表 13 - 6 DFP 算法求解二元函数极值问题结果

极值类型	x^*	y^*	$f(x^*, y^*)$
极小值	-2.452666315347257	-2.680759556435951	-11.284043710745076
极大值	-1.391611393212747	-1.793515727143566	8.612916227685943

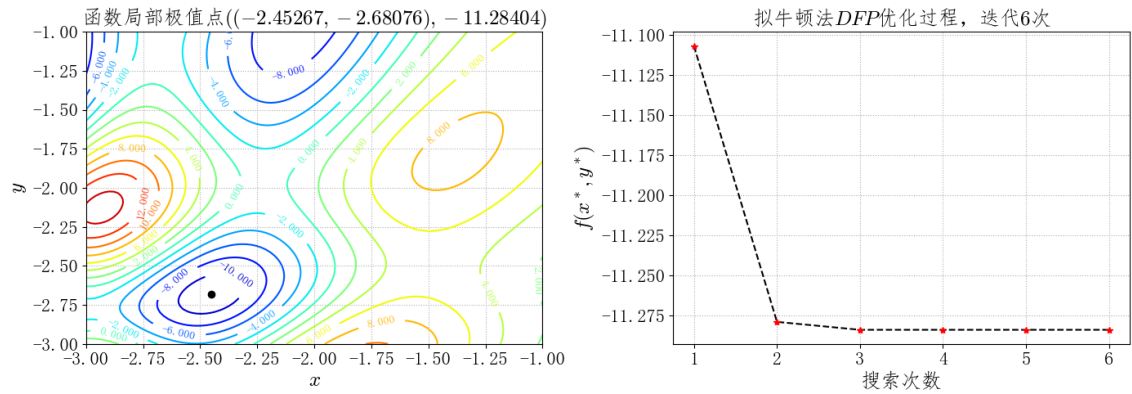


图 13 - 17 DFP 算法求解二元函数极小值

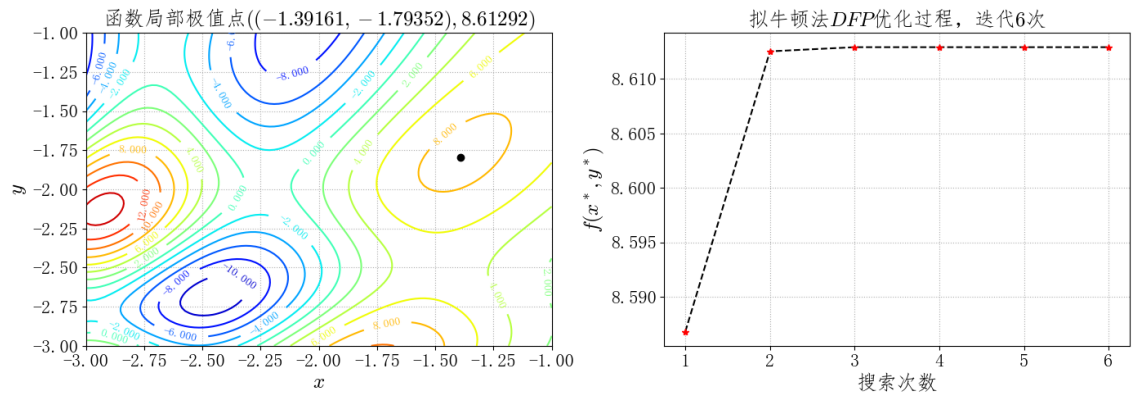


图 13 - 18 DFP 算法求解二元函数极大值

(2) BFGS 算法

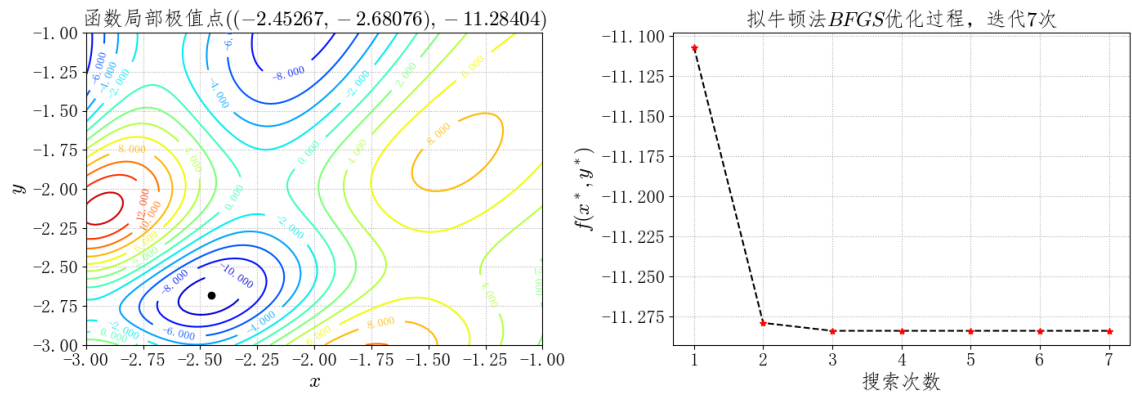


图 13 - 19 BFGS 算法求解二元函数极小值

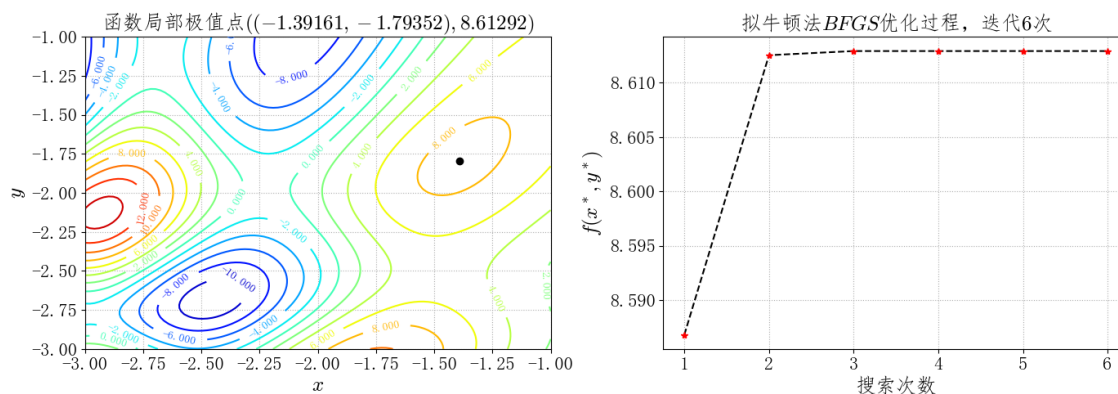


图 13 - 20 BFGS 算法求解二元函数极大值

表 13 - 7 BFGS 算法求解二元函数极值问题结果

极值类型	x^*	y^*	$f(x^*, y^*)$
极小值	-2.452666314143919	-2.680759557666897	-11.284043710745078
极大值	-1.391611391901446	-1.793515724867843	8.612916227685945

5. 采用模拟退火算法、遗传算法和蚁群算法求解二元函数 $f(x, y) = \cos(x^2 - xy) + \sin xy$ 在区间 $[-4, 4] \times [-4, 4]$ 内的最小值和最大值, 并进行可视化.

求解代码如下:

```
# -*- coding: UTF-8 -*-
"""
@file_name: chapter_13_exp5.py
"""

import numpy as np
from numerical_optimization_13.simulate_anneal import SimulatedAnnealingOptimization
from numerical_optimization_13.genetic_algorithm import GeneticAlgorithmOptimization
from numerical_optimization_13.ant_colony_algorithm import AntColonyAlgorithmOptimization
from Experiment.util_font import *

# 函数的三维曲面图
fig = plt.figure(figsize=(7, 5))
x = np.linspace(-4, 4, 100)
y = np.linspace(-4, 4, 100)
xi, yi = np.meshgrid(x, y)
zi = np.cos(xi ** 2 - xi * yi) + np.sin(xi * yi)
ax = fig.add_subplot(111, projection='3d')
ax.plot_surface(xi, yi, zi, cmap=plt.get_cmap("YlGnBu"))
plt.xlabel("$x$", fontdict={"fontsize": 18})
plt.ylabel("$y$", fontdict={"fontsize": 18})
plt.title("目标函数的三维曲面图", fontdict={"fontsize": 18})
plt.tick_params(labelsize=16) # 刻度字体大小 16
plt.show()
```

```

def sa_func_min(X):
    """
    定义目标函数, 求解极小值
    :param X: 向量
    :return:
    """
    x, y = X[0], X[1]
    z = np.cos(x ** 2 - x * y) + np.sin(x * y)
    return z

def sa_func_max(X):
    """
    定义目标函数, 求解极大值
    :param X: 向量
    :return:
    """
    x, y = X[0], X[1]
    z = np.cos(x ** 2 - x * y) + np.sin(x * y)
    return -1 * z

# (1) 模拟退火算法, 默认求最小值
sa = SimulatedAnnealingOptimization(sa_func_min, [[-4, 4], [-4, 4]], epochs=100)
sy, sx = sa.fit_optimize()
print("最优值%.15f, 解: %.15f, %.15f" % (sy, sx[0], sx[1])) # 针对最小值
# print("最优值%.15f, 解: %.15f, %.15f" % (-sy, sx[0], sx[1])) # 针对最大值
fig = plt.figure(figsize=(14, 5))
ax1 = fig.add_subplot(121)
x = np.linspace(-4, 4, 100)
y = np.linspace(-4, 4, 100)
xi, yi = np.meshgrid(x, y)
zi = np.cos(xi ** 2 - xi * yi) + np.sin(xi * yi)
cs = ax1.contour(xi, yi, zi, levels=20)
ax1.clabel(cs, inline=True, fontsize=10)
plt.plot(sx[0], sx[1], "r.", markersize=10)
plt.xlabel("$x$", fontdict={"fontsize": 18})
plt.ylabel("$y$", fontdict={"fontsize": 18})
# plt.title("最大值点: $f(%.8f, %.8f) = %.8f$" % (sx[0], sx[1], -sy), fontdict={"fontsize": 18})
plt.title("最小值点: $f(%.8f, %.8f) = %.8f$" % (sx[0], sx[1], sy), fontdict={"fontsize": 18})
plt.tick_params(labelsize=16) # 刻度字体大小 16
plt.subplot(122)
# 如下为绘制函数优化曲线
y_vals = np.asarray(sa.best_y_optlist)
plt.plot(y_vals) # 最小值
plt.grid(ls=":")
plt.xlabel("迭代次数", fontdict={"fontsize": 18})
plt.ylabel("最优值", fontdict={"fontsize": 18})

```

```

plt.title("模拟退火算法最值的优化过程", fontdict={"fontsize": 18})
plt.tick_params(labelsize=16) # 刻度字体大小 16
plt.show()

def plt_optimization(optimizing_best_f_val, best_x, best_y, title, type=0):
    """
    可视化优化过程
    :param optimizing_best_f_val: 优化过程的函数值, 一维数组
    :param best_x, best_y: 最优值
    :param title: 标题信息, 遗传算法或蚁群算法
    :param type: 最值类型, 0 为最大值, 1 为最小值
    :return:
    """

    fig = plt.figure(figsize=(14, 5))
    ax1 = fig.add_subplot(121)
    x = np.linspace(-4, 4, 100)
    y = np.linspace(-4, 4, 100)
    xi, yi = np.meshgrid(x, y)
    zi = np.cos(xi ** 2 - xi * yi) + np.sin(xi * yi)
    cs = ax1.contour(xi, yi, zi, levels=20)
    ax1.clabel(cs, inline=True, fontsize=10)
    plt.plot(best_x[0], best_x[1], "r.", markersize=10)
    plt.xlabel("$x$", fontdict={"fontsize": 18})
    plt.ylabel("$y$", fontdict={"fontsize": 18})
    if type == 0:
        plt.title("最大值点: $f(%.8f, %.8f) = %.8f$" % (best_x[0], best_x[1], best_y),
                  fontdict={"fontsize": 18})
    else:
        plt.title("最小值点: $f(%.8f, %.8f) = %.8f$" % (best_x[0], best_x[1], -1 * best_y),
                  fontdict={"fontsize": 18})

    plt.tick_params(labelsize=16) # 刻度字体大小 16
    plt.subplot(122)
    y_vals = np.asarray(optimizing_best_f_val)
    plt.plot(y_vals) # 最小值
    plt.grid(ls=":")
    plt.xlabel("迭代次数", fontdict={"fontsize": 18})
    plt.ylabel("最优值", fontdict={"fontsize": 18})
    plt.title("%s 算法最值的优化过程" % title, fontdict={"fontsize": 18})
    plt.tick_params(labelsize=16) # 刻度字体大小 16
    plt.show()

def func_min(X):
    """
    定义目标函数, 求解极小值, 适用于遗传算法和蚁群算法
    :param X: 向量
    """

```

```

x, y = X[:, 0], X[:, 1]
z = np.cos(x ** 2 - x * y) + np.sin(x * y)
return -1 * z

def func_max(X):
    """
    定义目标函数, 求解极大值, 适用于遗传算法和蚁群算法
    :param X: 向量
    """
    x, y = X[:, 0], X[:, 1]
    z = np.cos(x ** 2 - x * y) + np.sin(x * y)
    return z

# (2) 遗传算法, 默认求最大值
args_span = [[-4, 4], [-4, 4]]
# ga = GeneticAlgorithmOptimization(func_max, args_span, [8, 8],
#                                   max_epochs=1000, is_Maximum=True) # 最大值
ga = GeneticAlgorithmOptimization(func_min, args_span, [8, 8],
                                   max_epochs=1000, is_Maximum=False) # 最小值
best_y, best_x = ga.solve()
print("最优值%.15f, 解: %.15f, %.15f" % (best_y, best_x[0], best_x[1]))
plt_optimization(ga.optimizing_best_f_val, best_x, best_y, "遗传算法", type=1)

# (3) 蚁群算法, 默认求最大值
args_span = [[-4, 4], [-4, 4]]
# aco = AntColonyAlgorithmOptimization(func_max, args_span, ant_m=50, step=0.1, rho=0.8, tp_c=0.6,
#                                     is_Maximum=True, max_iter=1000) # 最大值
aco = AntColonyAlgorithmOptimization(func_min, args_span, ant_m=50, step=0.1, rho=0.8, tp_c=0.6,
                                     is_Maximum=False, max_iter=1000) # 最小值
best_x, best_y = aco.fit_optimize()
print("最优值%.15f, 解: %.15f, %.15f" % (best_y, best_x[0], best_x[1]))
plt_optimization(aco.optimizing_best_f_val, best_x, best_y, "蚁群算法", type=1)

```

目标函数的三维曲面图

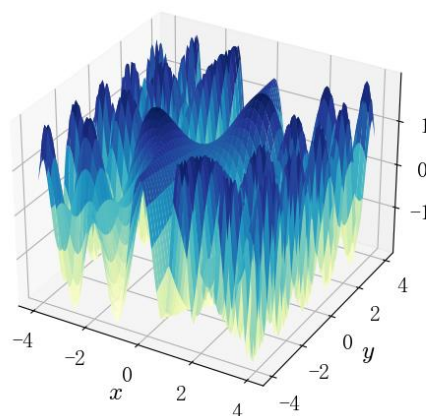


图 13-21 目标函数的三维曲面图

(1) 模拟退火算法

表 13 - 8 模拟退火算法求解最值问题结果

极值类型	试验次数	x^*	y^*	$f(x^*,y^*)$
最小值	1	-1.254632773406411	-3.754166906964755	-1.999981729049725
	2	-1.255391276775046	-3.761836336389136	-1.999939873215859
最大值	1	-1.257801093040669	-1.247763879045793	1.999919386555625
	2	1.244601849934187	1.260074659573532	1.999811442620618

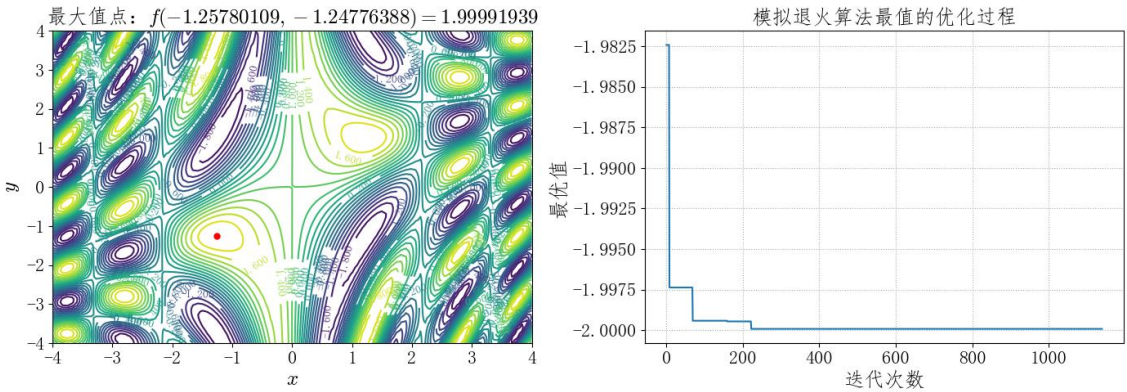


图 13 - 22 模拟退火算法求解目标函数的最大值

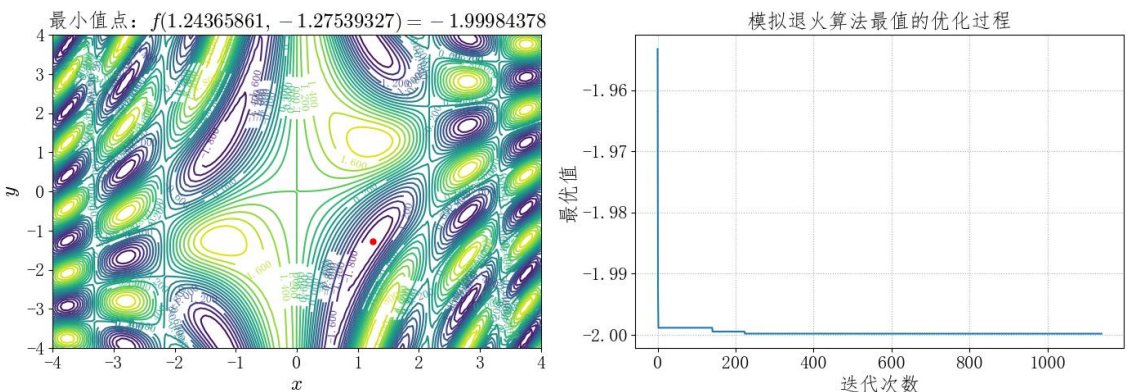


图 13 - 23 模拟退火算法求解目标函数的最小值

(2) 遗传算法

表 13 - 9 遗传算法求解目标函数最值问题结果

极值类型	试验次数	x^*	y^*	$f(x^*,y^*)$
最小值	1	2.802244319396321	-0.560088005438529	-1.999995515692834
	2	1.252198604170418	3.764420971054976	-1.999990144206603
最大值	1	-1.253071879272416	-1.252620031361114	1.999999151287903
	2	1.253901076739562	-3.757503270877062	1.999999445202902

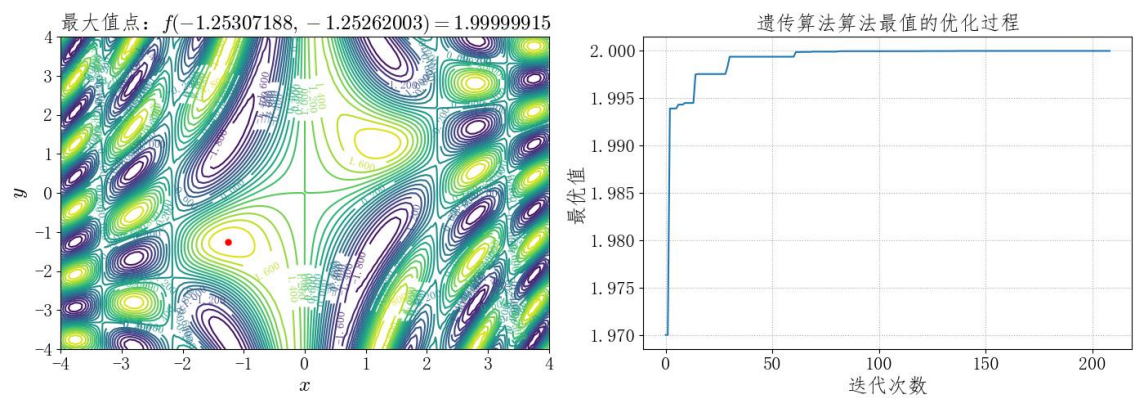


图 13 - 24 遗传算法求解目标函数的最大值

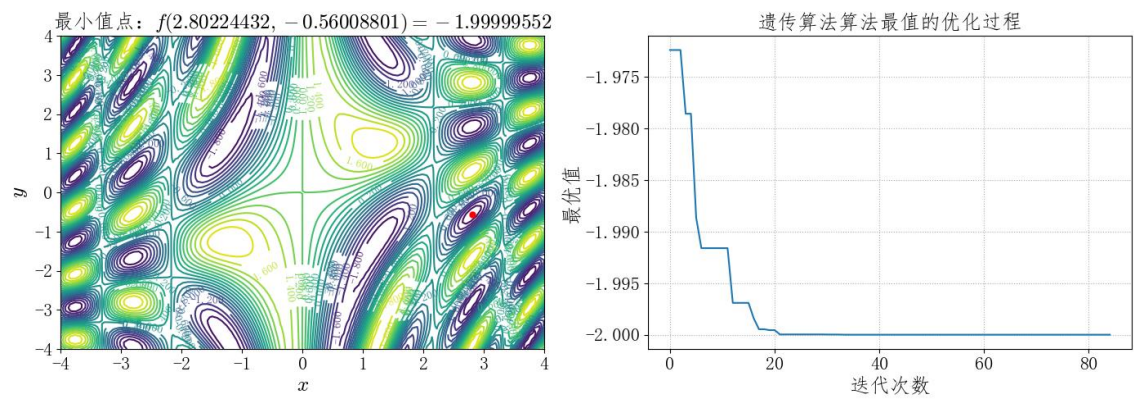


图 13 - 25 遗传算法求解目标函数的最小值

(3) 蚁群算法

表 13 - 10 蚁群算法求解目标函数最值问题结果

极值类型	试验次数	x^*	y^*	$f(x^*, y^*)$
最小值	1	2.802500229913949	3.923475960656166	-1.999999997811005
	2	-1.253296974372040	1.253343427162322	-1.999999999497460
最大值	1	-1.253408464696485	-1.253279875839959	1.999999984178008
	2	1.253320602460965	1.253304963877419	1.99999999802156

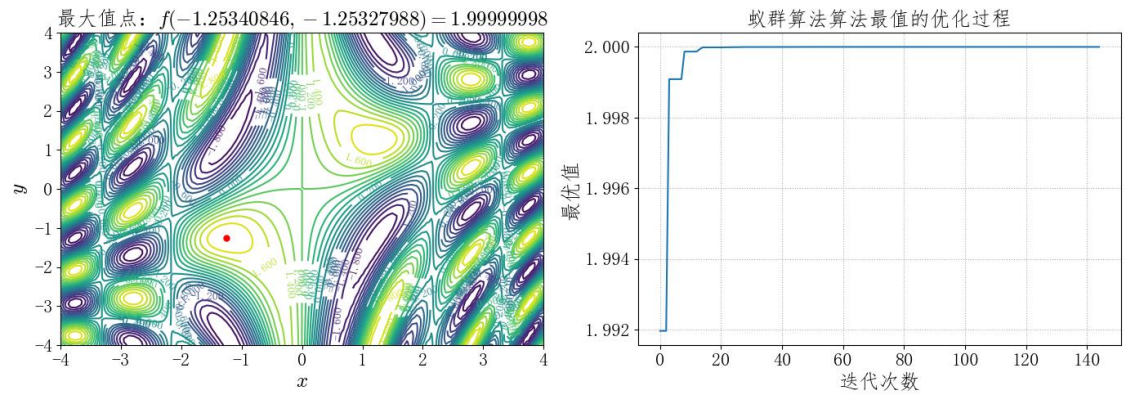


图 13 - 26 蚁群算法求解目标函数的最大值

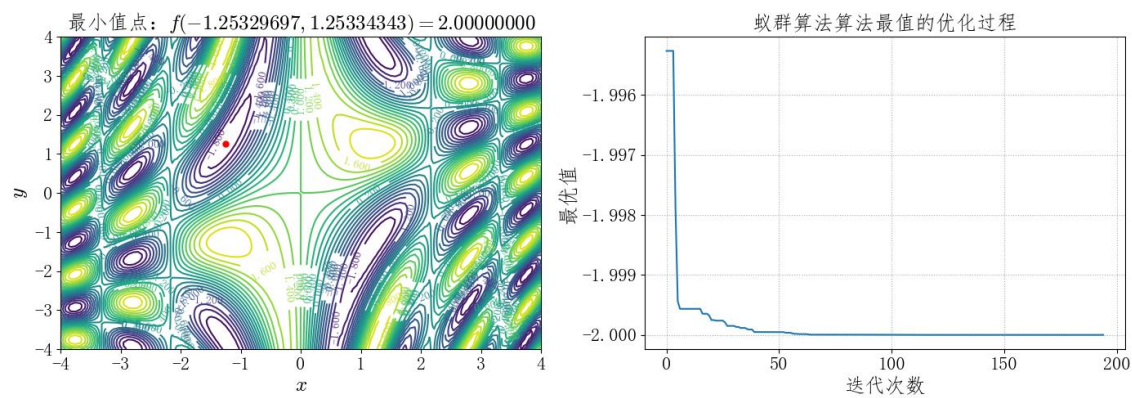


图 13 - 27 蚁群算法求解目标函数的最小值